

Sistemi Intelligenti

Appunti Completi di Teoria

Corso di Sistemi Intelligenti — Cristina Baroglio, Università di Torino

A.A. 2025–2026

Indice

Modulo 1 — Introduzione all'IA e Agenti Intelligenti	8
1. Cos'è l'intelligenza artificiale?	8
1.1 Un punto di partenza informale: cosa sanno fare i sistemi di IA generativa oggi	8
1.2 IA nell'immaginario collettivo vs. IA reale	8
1.3 Definizione di "intelligenza" e problema della IA	9
1.4 Breve storia: le origini	9
1.5 Il Dartmouth Summer Research Project (1956)	9
1.6 Il Test di Turing	10
1.7 La Stanza Cinese di John Searle (1980)	10
1.8 Test di Turing inverso: CAPTCHA	11
1.9 Strong AI vs. Weak AI	11
1.10 Dall'esempio "attraversare la strada" nasce l'astrazione di agente	11
1.11 Le quattro scuole di definizione dell'IA	12
1.12 Quando serve davvero l'IA?	13
1.13 Discipline di fondamento dell'IA	13
1.14 Risoluzione automatica di problemi (anticipazione)	13
2. Agenti e ambienti	13
2.1 Agente e ambiente: il binomio fondamentale	13
2.2 Sequenza percettiva	14
Esempio: il mondo dell'aspirapolvere	14
2.3 Razionalità	14
2.3.1 "Fare la cosa giusta"	14
2.3.2 Definizione formale di comportamento razionale	14
2.3.3 Razionalità vs. onniscienza/chiaroveggenza	15
2.3.4 La percezione non è un atto passivo	15
2.4 Task environment e PEAS	16
2.5 Proprietà dell'ambiente (task environment)	16
Esempi di caratterizzazione di ambienti (dalle slide)	17
Esempio guida: il termostato	17
Esempio guida: robot aspirapolvere (Roomba e simili)	17
2.6 Agente = architettura + programma	18
2.7 Tipologie di agente	18
2.7.1 Agenti reattivi semplici	18
2.7.2 Agenti reattivi basati su modello	19
2.7.3 Agenti basati su obiettivi (goal-driven)	19
2.7.4 Agenti basati sull'utilità (utility-driven)	19
2.7.5 Agenti che apprendono	20
2.8 Tabella riassuntiva delle tipologie di agente	20
3. Paradigmi di programmazione e agenti autonomi	20
3.1 Dall'approccio tradizionale (imperativo/a oggetti) all'approccio dichiarativo	21
3.2 Esempio guida: il mondo dei blocchi (Blocks World)	21

3.3 Dal problema giocattolo al problema reale	22
3.4 Da automazione ad autonomia	22
3.5 Gradi di autonomia: tre esempi concreti a confronto	23
3.6 Ragionare basta?	24
3.7 Approfondimento facoltativo: sistemi multiagente (MAS)	24
Riepilogo e punti chiave	25
Modulo 2 — Risoluzione di problemi mediante ricerca (non informata e informata)	27
Stati, azioni e astrazione	27
Obiettivi e ricerca	27
Definizione formale di problema di ricerca	28
Esempio guida: problema di navigazione	28
Esempio: il mondo dell'aspirapolvere	28
Esempio: il gioco dell'8 (8-puzzle)	28
Toy problem vs Real-world problem	29
Spazio degli stati vs albero/grafico di ricerca	29
Criteri di valutazione delle strategie di ricerca	29
Parte 1 — Ricerca non informata (blind search)	31
1. Ricerca in ampiezza (Breadth-First Search, BFS)	31
Valutazione BFS	31
2. Ricerca a costo uniforme (Uniform-Cost Search)	31
Valutazione ricerca a costo uniforme	32
3. Ricerca in profondità (Depth-First Search, DFS) senza backtracking	32
Valutazione DFS	32
Variante: ricerca a profondità limitata	32
4. Iterative Deepening Depth-First Search (IDDFS)	33
Perché non è così costoso ripartire da zero?	33
Valutazione IDDFS	33
5. Ricerca bidirezionale	33
Valutazione ricerca bidirezionale	34
Grafi di ricerca e nodi già esplorati	34
Esempi classici di problemi di attraversamento	34
Tabella riassuntiva — confronto strategie di ricerca non informata	34
Parte 2 — Ricerca informata (heuristic search)	36
Perché serve conoscenza aggiuntiva	36
Funzioni di valutazione e best-first search	36
1. Ricerca greedy (avida)	36
2. A* (A-star)	37
Nozione di obiettivo preferito	37
Pseudocodice di A*	37
Euristiche ammissibili	37
Ottimalità di A*	38
Dimostrazione (caso albero di ricerca)	38
Caso grafo di ricerca: serve la consistenza (monotonicità)	38
Monotonicità vs ammissibilità	38
Ammissibile non vuol dire informativo	39
Ottimalità in senso forte di A*	39
Ridurre i requisiti di memoria: IDA* e RBFS	39
RBFS in dettaglio (Korf, 1993)	39
Valutazione RBFS	40
Funzioni euristiche: il caso di studio dell'8-puzzle	40
Due euristiche ammissibili classiche	40
Confronto sperimentale fra euristiche	40

Effective branching factor (fattore di ramificazione effettivo) b^*	41
Valutazione teorica: euristiche dominanti	41
Costruzione sistematica di euristiche ammissibili: problemi rilassati	41
Combinare euristiche non comparabili	42
Alternativa: apprendere le euristiche induttivamente	42
Riepilogo e punti chiave	42
MODULO 3 — Ricerca con avversario (giochi)	44
Introduzione: perché studiare i giochi in AI	44
Ambienti competitivi	44
Tassonomia dei giochi	44
Gioco a somma zero	45
Perché il tris è un caso interessante da studiare	45
Differenze rispetto alla ricerca (in)formata "classica"	45
Caratteristiche formali del gioco (a due giocatori)	45
Albero di gioco: definizione	46
Algoritmo Minimax	46
Idea e strategia ottima	46
Definizione ricorsiva: valore-minimax	46
Pseudocodice	46
Esempio passo-passo	47
Complessità	47
Proprietà	47
Estensione a più di due giocatori	48
Teoria delle decisioni: maximax, maximin, minimax regret	48
Scenario di esempio: tabella dei payoff	48
Approccio maximax (ottimistico)	48
Approccio maximin (pessimistico/conservativo)	48
Approccio minimax regret	49
Quando si usa ciascun approccio	49
Collegamento con i giochi con avversario	49
Potatura alfa-beta (Alpha-Beta Pruning)	49
Motivazione	49
Definizione di α e β	50
Quando si pota un ramo	50
Pseudocodice	50
Esempio passo-passo (tracciamento sull'albero delle slide)	51
Equivalenza con Minimax e guadagno in complessità	51
Importanza dell'ordinamento delle mosse	51
Come trovare le killer moves	52
Trasposizioni	52
Ricerca con profondità limitata e funzioni di valutazione euristica	52
Il problema del real-time	52
Funzione di valutazione euristica	53
Alfa-beta con cutoff (versione con profondità limitata)	53
Problema dell'orizzonte	53
Quiescenza	53
Giochi stocastici: cenni (expectiminimax)	53
Programmi storici che giocano	54
DeepBlue	54
AlphaGo	54
Un parallelo con la memoria umana	55
Riepilogo e punti chiave	55

Modulo 4: Constraint Satisfaction Problems (CSP)	56
Definizione formale di CSP	56
Stati e assegnamenti	56
Esempio guida: colorazione della mappa dell'Australia	56
Tipi di domini e di variabili	57
Arità dei vincoli	57
Esempio classico di vincolo n-ario: criptoaritmetica	57
Vincoli vs. criteri di preferenza	57
CSP come problema di ricerca nello spazio degli stati	58
Perché l'ordine di assegnamento non conta: commutatività	58
Rappresentare gli stati: generate-and-test vs. ricerca con backtracking	58
Approccio a forza bruta: generate-and-test	58
Ricerca in profondità con backtracking	59
Pseudocodice del backtracking	59
Limiti del backtracking semplice: il thrashing	59
Euristiche per la scelta della variabile	60
Minimum Remaining Values (MRV, o euristica "fail-first")	60
Degree heuristic (euristica di grado)	60
Euristica per la scelta del valore: Least Constraining Value (LCV)	60
Rendere informata la ricerca: propagazione dei vincoli (inferenza)	61
Forward checking	61
Limiti del forward checking	62
Node consistency	62
Arc consistency	62
Algoritmo AC-3	62
Costo computazionale di AC-3	63
Incompletezza di AC-3 (arc-consistent ma senza soluzione)	63
Path consistency	64
Generalizzazione: k-consistency	64
Vincoli speciali (globali)	64
Alldifferent($X_1, \dots, X_n$)	64
Atmost($N, A_1, \dots, A_k$)	65
Oltre il backtracking cronologico: backjumping	65
Il limite del backtracking cronologico	65
Backjumping: backtracking non cronologico	65
Relazione fra backjumping e forward checking	65
Assegnamenti NOGOOD e conflict-directed backjumping	65
NOGOOD	66
Conflict-directed backjumping	66
Applicazioni ed esempi aggiuntivi	66
Sudoku come CSP	66
N-regine su larga scala	67
Applicazioni reali di tipo ricerca operativa	67
Software e strumenti per CSP	67
Riepilogo e punti chiave	68
Modulo 5: Rappresentazione della conoscenza e Logica proposizionale	69
Perché rappresentare la conoscenza	69
Cosa vuol dire "ragionare"	69
Agenti basati sulla conoscenza	69
Dato, informazione, conoscenza	70
Esempio: bug algorithm	70
Programmazione dichiarativa	70
Esempio: il mondo dei blocchi	70

Rappresentare la conoscenza tramite la logica	71
Modello	71
Conseguenza logica (entailment, $\models$)	71
Equivalenza	71
Validità, insoddisfacibilità, soddisfacibilità	72
Inferenza	72
Grounding	72
Logica proposizionale	73
Sintassi	73
Semantica	73
L'implicazione: come interpretarla	74
Biimplicazione	74
Verificare l'entailment: $KB \models P$?	74
Regole di inferenza (equivalenze logiche)	75
La regola di risoluzione (Resolution)	75
Dall'agente alla necessità della forma clausale	76
Forma Normale Congiuntiva (CNF)	76
Esempio completo: dalle formule alle clausole	76
Algoritmo di risoluzione proposizionale	76
Esempio applicativo: pioggia, atmosfera, strada	77
Clausole di Horn, Forward Chaining, Backward Chaining	77
Clausole di Horn	77
Forward Chaining (concatenazione in avanti)	78
Esempio (rappresentazione a grafo AND-OR)	78
Backward Chaining (concatenazione all'indietro)	78
Esempio 1	79
Esempio 2 (stessa KB del forward chaining)	79
Forward vs Backward chaining: confronto	79
Riepilogo e punti chiave	80
Modulo 6: Logica del primo ordine (FOL)	81
Perché non basta la logica proposizionale	81
Pregi della logica proposizionale (da mantenere)	81
I limiti espressivi	81
Dalla logica proposizionale a FOL	82
Elementi sintattici di FOL	82
Grammatica di FOL	82
Predicati vs funzioni	82
Modelli e interpretazione	83
Soddisfacibilità, validità, insoddisfacibilità (richiamo)	83
Quanti modelli ha una KB FOL?	83
Termini e formule atomiche	83
Quantificatori	84
Quantificatore universale $\forall$	84
Quantificatore esistenziale $\exists$	85
Regola pratica: $\forall$ va con $\Rightarrow$, $\exists$ va con $\wedge$	85
Quantificatori annidati: l'ordine conta	85
Quantificatori e negazione (leggi di De Morgan generalizzate)	86
Uguaglianza e quantificatori	86
Il problema dell'unicità dei nomi	86
Database semantics (semantica alternativa più intuitiva)	86
Inferenza in FOL: due approcci	87
Interrogare una KB FOL	87
Sostituzione	87

Proposizionalizzazione: UI ed EI	87
Il problema delle funzioni e il teorema di Herbrand	88
Inefficienza della proposizionalizzazione	88
Modus Ponens Generalizzato (MPG)	88
Formula	88
Esempio	88
Lifting	89
Unificazione	89
Esempi passo-passo	89
Clausole di Horn del prim'ordine	89
KB di esempio: "Sotto Casa"	90
Forward Chaining in FOL	90
Backward Chaining (BC) in FOL	91
Risoluzione in FOL: lifting e refutazione	91
Procedura per tradurre FOL in CNF	92
Skolemizzazione	92
Perché serve	92
Procedura (regola generale)	92
Esempio più semplice, passo-passo (per fissare l'intuizione)	93
Funzioni di Skolem con più argomenti	93
Caso limite: l'esistenziale "esterno" a tutti gli universali	93
Binary resolution in FOL (lifting della risoluzione)	94
Dimostrazione per refutazione: l'esempio "Curiosity ha ucciso il gatto?"	94
Refutation-completeness	95
Costruire una KB in FOL: ingegneria della conoscenza	96
Riepilogo e punti chiave	96
Modulo 7 — Tassonomie/Ontologie e Pianificazione automatica	97
Parte A: Tassonomie e Ontologie	97
1. Cos'è una tassonomia	97
2. Decomposizioni, disgiunzione, partizione	97
3. Relazioni strutturali: Part-of e Bunch-of	98
4. T-box e A-box	98
5. Problematiche delle tassonomie/ontologie	98
6. Dalla tassonomia all'ontologia	99
7. Esempio applicativo: PROV (provenance)	99
8. Semantic Web e linguaggi per rappresentare ontologie	99
9. Costruire un'ontologia: metodologia pratica	100
10. Relazioni fra ontologie	101
Parte B: Pianificazione automatica	102
1. Dalle ontologie alla pianificazione: perché serve un nuovo formalismo	102
2. Differenza rispetto alla ricerca classica	102
3. Il Situation Calculus: fondamento logico della pianificazione	103
4. Il Frame Problem	104
5. Perseguire goal complessi: scomposizione in sottogoal	104
6. L'anomalia di Sussman: quando i sottogoal interagiscono	104
7. Limiti pratici del Situation Calculus	105
Riepilogo e punti chiave	106
Modulo 8: Machine Learning — Classificazione, Alberi di Decisione, Reti Neurali	107
Il problema della classificazione	107
Le classi non sono "naturali"	107
Definizione del problema	107
Schema generale: training set, modello, test set	107

Modello predittivo vs modello descrittivo	108
Valutazione: matrice di confusione, accuratezza, error rate	108
Matrice dei costi	108
Insidie pratiche nella costruzione del classificatore	109
Rote learning (apprendimento meccanico) — un “non-modello”	109
Alberi di decisione	109
Cosa sono	109
Struttura	110
Algoritmi per apprendere alberi di decisione	110
L’algoritmo di Hunt	110
Strategia greedy e problemi aperti	110
Tipi di split in base al tipo di attributo	111
Quale attributo scegliere per lo split? Misure di impurità	111
Entropia	111
Gini index	111
Errore di classificazione (misclassification error)	111
Calcolo del guadagno (gain) di uno split	112
Information gain	112
Criteri di arresto (cenni)	112
Reti neurali	112
Ispirazione biologica	113
Il neurone artificiale (Perceptron)	113
Passata forward e interpretazione geometrica	113
Caratteristiche del perceptron	114
Apprendimento del perceptron	114
Limiti del singolo perceptron: lo XOR	114
Reti neurali: definizione generale e topologia	115
Multi-Layer Perceptron (MLP)	115
Codifica delle classi nell’output di un MLP	115
Cosa imparano i diversi hidden layer	116
Potere espressivo dell’MLP	116
Apprendimento dell’MLP: backpropagation	116
Discesa del gradiente	116
Errore globale dell’MLP	116
Il problema della distribuzione del credito/biasimo (credit assignment)	117
Oltre la classificazione: la regressione	117
Riepilogo e punti chiave	118

Modulo 1 — Introduzione all'IA e Agenti Intelligenti

Corso di Sistemi Intelligenti, Cristina Baroglio, Università di Torino. Riferimento: Russell & Norvig, *Intelligenza Artificiale: un approccio moderno* (in particolare il cap. 2 per la parte sugli agenti).

1. Cos'è l'intelligenza artificiale?

1.1 Un punto di partenza informale: cosa sanno fare i sistemi di IA generativa oggi

Le slide aprono mostrando esempi reali (DeepSeek, ChatGPT, febbraio 2025) a cui viene posta la stessa domanda ("Chi era Napoleone VII di Baviera?", personaggio storico **inesistente**). I due sistemi rispondono con sicurezza ma con contenuti diversi e **inventati** (allucinazioni): DeepSeek lo descrive come figlio di Napoleone III morto nel 1879, ChatGPT come figlio di Massimiliano I di Baviera morto nel 1837. Nessuna delle due risposte è vera, ma entrambe sono fluente e plausibili.

Nel secondo esempio, allo stesso sistema viene chiesto di scrivere un articolo che metta "in buona luce" e poi "in cattiva luce" le lumache: il sistema produce entrambi i testi senza alcuna difficoltà, adattando tono e argomentazione alla richiesta.

Perché questi esempi aprono il corso? Servono a mostrare, prima ancora di dare definizioni, il problema centrale che il modulo affronterà: questi sistemi producono output linguisticamente ineccepibili e convincenti, ma ciò **non implica che "comprendano"** quello che dicono, né che possiedano conoscenza verificata. È un'anticipazione informale del dibattito Turing/Searle che viene formalizzato più avanti.

1.2 IA nell'immaginario collettivo vs. IA reale

- **Immaginario comune:** robot antropomorfo, capace di risolvere problemi complessi, capace di imparare (fantascienza).
- **IA realmente diffusa intorno a noi** (spesso in modo "invisibile", cioè in **contatto quotidiano ma inconsapevole**):
 - servizi di streaming (raccomandazioni),
 - fotocamere/smartphone (riconoscimento volti, cibo, scene),
 - social network (annunci e suggerimenti personalizzati),
 - assistenti virtuali (chat, voce, mappe),
 - supporto alla decisione (es. concessione di mutui),
 - logistica e consegne a domicilio (calcolo percorsi).

Il punto pedagogico: l'IA che usiamo ogni giorno è quasi sempre **specializzata e invisibile**, non un robot antropomorfo generalista.

1.3 Definizione di “intelligenza” e problema della IA

Il vocabolario definisce l'intelligenza (umana) come il complesso di facoltà psichiche che permettono di pensare, comprendere, spiegare fatti/azioni, elaborare modelli astratti della realtà, farsi intendere dagli altri, giudicare, adattarsi a situazioni nuove e modificarle. È una facoltà propria dell'uomo (con gradazioni riconosciute anche agli animali), che si sviluppa nell'infanzia insieme alla consapevolezza.

Intelligenza artificiale = una forma di intelligenza **non naturale**, ottenuta con procedimenti tecnici (cioè costruita dall'uomo tramite l'informatica, non frutto di sviluppo biologico).

1.4 Breve storia: le origini

Anno	Evento
1936	Alan Turing formalizza la Macchina di Turing , pietra miliare dell'informatica teorica
1940	ENIAC, tra i primi calcolatori
1950	Turing propone il Test di Turing : quando un computer può dirsi intelligente?
1951	UNIVAC
1956	Nasce ufficialmente l'IA come disciplina: Dartmouth Summer Research Project

Anni '40 — automazione del calcolo: il computer esegue una sequenza di istruzioni predefinita, senza operatore umano che intervenga passo-passo. Questo pone subito la domanda: *l'automazione (eseguire automaticamente un programma) equivale a intelligenza?*

Le slide insistono molto su questa domanda mostrando esempi progressivi (una calcolatrice esegue un programma ed è automatica — ma non diremmo che è intelligente) per portare lo studente a capire che **automazione ≠ intelligenza**: eseguire automaticamente istruzioni non basta. Serve qualcos'altro — capacità di adattarsi, di scegliere di fronte a situazioni non previste esplicitamente dal programmatore.

? Domanda d'esame: Automazione è intelligenza? No. Un programma che esegue automaticamente una sequenza di istruzioni predefinite (es. una calcolatrice) è automatico ma non è considerato intelligente: non si adatta, non affronta situazioni nuove, non delibera. L'intelligenza (artificiale) richiede in più la capacità di **adattarsi**, di scegliere l'azione più opportuna in funzione di percezioni e obiettivi, eventualmente anche di **imparare** dall'esperienza. Questo è il punto di svolta che porta dal concetto di “programma automatico” a quello di “agente”.

1.5 Il Dartmouth Summer Research Project (1956)

- Il termine “Artificial Intelligence” nasce nel 1956, proposto da **John McCarthy**.
- Un gruppo di circa venti ricercatori di discipline diverse (Solomonoff, Minsky, McCarthy, Shannon, Rochester, Selfridge, Newell, Simon, ecc.) si riunisce per circa due mesi per fare brainstorming sulle “thinking machines”. Notare: né Turing (morto nel 1954) né von Neumann (gravemente malato) parteciparono.
- **“Carta di identità” dell'IA:**
 - Nascita: Dartmouth Conference, USA, 1956; nome scelto da McCarthy.
 - Prima del 1956 (primi anni '50): si discuteva già se una macchina potesse “pensare” (cibernetica, teoria degli automi, elaborazione dell'informazione complessa; Test di Turing).
 - Dopo il 1956 (primi anni '60): primi tentativi applicativi (scacchi, giochi, dimostrazione automatica di teoremi).

- Contesto tecnologico dell'epoca: niente tastiere, dischi magnetici o monitor come li intendiamo oggi; solo al MIT si sperimentava l'input diretto da tastiera (anziché schede perforate), mentre IBM sviluppava i precursori dei dischi magnetici.

1.6 Il Test di Turing

Proposto da Alan Turing nel 1950 nell'articolo "Computing Machinery and Intelligence" come **The Imitation Game**: sostituisce la domanda mal posta "può una macchina pensare?" con una domanda operativa.

Come funziona: un intervistatore umano comunica per iscritto con due interlocutori nascosti (uno umano, uno macchina) senza sapere quale sia quale. Può fare qualsiasi domanda. Se, al termine, l'intervistatore non riesce a distinguere in modo affidabile la macchina dall'umano (o giudica erroneamente che la macchina sia umana), la macchina **supera il test**.

Esempio dalle slide:

Q: Please write me a sonnet on the subject of the Forth Bridge.

A: Count me out on this one. I never could write poetry.

Q: Add 34957 to 70764.

A: (pausa di 30 secondi) 105621.

Q: Do you play chess?

A: Yes.

Da notare: la macchina finge persino di essere lenta a fare un calcolo (per sembrare più umana) — il test valuta la capacità di **imitare risposte plausibili**, di comportarsi *come se* fosse un essere umano, non un meccanismo interno particolare.

? Domanda d'esame: Il test di Turing cattura davvero l'intelligenza? Questo è uno dei nodi filosofici centrali del modulo. Il test si basa solo sull'osservazione dei comportamenti esterni (input/output): se gli output sono indistinguibili da quelli umani, la macchina "passa". Ma **produrre gli output attesi è sufficiente per dire che c'è comprensione?** Le slide rispondono con l'esempio del quiz sulla geografia: due persone (Valeria e Rossella) danno entrambe la risposta corretta "Piemonte" alla domanda "dove si trova Torino?", ma Valeria la sa perché ha studiato geografia e ha usato conoscenza + ragionamento per collegare la domanda alla risposta, mentre Rossella ha semplicemente tirato a caso ed è stata fortunata. Dall'esterno (stesso input, stesso output) i due comportamenti sono indistinguibili, ma solo uno dei due implica reale comprensione. Questo mostra che **stesso output non implica stessa comprensione**: il Test di Turing, valutando solo il comportamento osservabile, non può distinguere questi due casi, e quindi è un test necessario ma probabilmente non sufficiente per l'intelligenza "vera" (nel senso forte).

1.7 La Stanza Cinese di John Searle (1980)

Argomento filosofico contro l'idea che superare il Test di Turing implichi comprensione reale (Searle, *Minds, brains, and programs*, 1980).

Esperimento mentale: una persona che non conosce il cinese è chiusa in una stanza. Riceve dall'esterno frasi scritte in ideogrammi cinesi (input). All'interno della stanza ha un manuale di istruzioni (equivalente a un programma) che le dice, per ogni sequenza di simboli in ingresso, quali simboli produrre in uscita, puramente in base alla loro forma sintattica, senza capirne il significato. Seguendo meccanicamente le istruzioni, la persona restituisce risposte in cinese perfettamente sensate, tanto che chi sta fuori dalla stanza (un madrelingua cinese) crede di conversare con qualcuno che capisce il cinese.

Domanda: quella persona (o il sistema stanza+persona+manuale) **parla davvero cinese? Lo capisce?**

Secondo Searle, no: la persona sta solo manipolando simboli sintatticamente, senza avere accesso alla semantica, all'intenzionalità, alla comprensione. Questo è esattamente ciò che fa un computer che esegue un programma: manipola simboli seguendo regole sintattiche, senza "capire" nulla nel senso in cui lo capisce un umano. Da qui la tesi di Searle: *"Instantiating a computer program is never by itself a sufficient condition of*

intentionality” — eseguire un programma non è mai di per sé condizione sufficiente per avere intenzionalità (cioè stati mentali “rivolti verso” un significato).

Questo argomento si collega direttamente all’apertura del modulo (DeepSeek/ChatGPT): questi sistemi possono essere impressionanti nelle risposte, ma la domanda “capiscono quello che dicono, oppure no?” resta aperta e filosoficamente delicata.

1.8 Test di Turing inverso: CAPTCHA

CAPTCHA = “Completely Automated Public Turing-test to tell Computers and Humans Apart”. A differenza del test di Turing classico (dove un umano cerca di riconoscere la macchina), qui è **un computer** che pone un test per distinguere se dall'altra parte c'è un umano o un bot, tipicamente per impedire l'accesso automatizzato a form o dati. Sviluppato nel 1997 nel contesto del motore di ricerca AltaVista.

1.9 Strong AI vs. Weak AI

Due modi diversi di intendere l'obiettivo della disciplina:

	Strong AI (IA forte)	Weak AI (IA debole)
Obiettivo	Riprodurre realmente l'intelligenza umana	Trovare metodi per risolvere problemi che, se risolti da un umano, richiederebbero intelligenza
Approccio	Studio del pensiero e comportamento umano (scienze cognitive)	Task-oriented: studio del pensiero e comportamento <i>razionale</i> (non necessariamente umano)
Criterio di successo	La macchina deve effettivamente pensare/capire come un umano	Basta che il sistema <i>funzioni</i> producendo la soluzione corretta, indipendentemente dal meccanismo interno

Questa distinzione anticipa le “quattro scuole” di definizione dell'IA (§1.11) e chiarisce perché il corso, seguendo Russell & Norvig, adotterà come riferimento pratico l'approccio del **comportamento razionale** (weak AI, task-oriented), più operativo e ingegnerizzabile.

1.10 Dall'esempio “attraversare la strada” nasce l'astrazione di agente

Le slide usano un esempio guida ricorrente: *quanto è difficile attraversare una strada?*

Un **uomo** che attraversa la strada deve: identificare il passaggio pedonale, rilevare possibili ostacoli, rilevare oggetti in movimento, rilevare segnali significativi (es. semaforo), costruire un piano d'azione. L'ambiente in cui opera è **complesso, parzialmente prevedibile, parzialmente collaborativo** (altri conducenti/pedoni possono cooperare o no).

La domanda diventa: **come si programma un agente artificiale** capace di fare le stesse cose, nello stesso tipo di ambiente? Da qui emergono le prime astrazioni fondamentali del corso: il binomio **(agente, ambiente)**.

Definizione di agente: un agente è un'astrazione che rappresenta un qualsiasi sistema che percepisce il proprio ambiente tramite dei sensori e agisce su di esso tramite degli attuatori.

Punti chiave impliciti in questa definizione: - Non esistono agenti che non siano **situati in un ambiente**: agente e ambiente sono un **binomio inscindibile**, non ha senso parlare di un agente “nel vuoto”. - Tra la percezione (input dai sensori) e l'azione (output verso gli attuatori) c'è una **funzione deliberativa** (rappresentata spesso nei diagrammi con un punto interrogativo “?” dentro l'agente) che decide quale azione eseguire sulla base di ciò che è stato percepito. È proprio il “contenuto” di questo punto interrogativo — come viene implementata

la deliberazione — a differenziare i vari tipi di agente che vedremo più avanti (agenti reattivi, basati su modello, su obiettivi, sull'utilità, che apprendono).

Questo schema **agente** ↔ **ambiente** (percezione tramite sensori → deliberazione → azione tramite attuatori, in un ciclo continuo) è lo schema concettuale centrale attorno a cui ruota tutto il resto del modulo.

1.11 Le quattro scuole di definizione dell'IA

Russell & Norvig classificano le definizioni storiche di IA lungo due assi: *pensiero vs. comportamento* e *fedeltà umana vs. razionalità*.

	Riproduce il pensiero	Riproduce il comportamento
come l'uomo	Sistemi che pensano come esseri umani Haugeland 1985: "far sì che i computer arrivino a pensare... macchine dotate di mente"Bellman 1978: automazione di decisione, risoluzione problemi, apprendimento	Sistemi che agiscono come esseri umani Kurzweil 1990: "creare macchine che eseguono attività che richiedono intelligenza se svolte da persone"Rich & Knight 1991: far eseguire ai computer ciò in cui, al momento, gli umani sono più bravi
in modo razionale	Sistemi che pensano razionalmente Charniak & McDermott 1985: studio delle facoltà mentali tramite modelli computazionaliWinston 1992: studio dei processi computazionali che rendono possibile percepire, ragionare, agire	Sistemi che agiscono razionalmente Poole et al. 1998: progettazione di agenti intelligentiNilsson 1998: comportamento intelligente negli artefatti

Confronto sintetico con l'esempio dell'attraversamento pedonale:

	Pensiero	Comportamento
Umano (approccio forte)	Ragionamento logico esplicito sul fatto che c'è un passaggio pedonale, sgombro, senza auto in arrivo (Modellazione Cognitiva, es. <i>General Problem Solver</i> di Newell & Simon)	Guardo a sinistra e a destra prima di attraversare (valutato col Test di Turing: il comportamento è "umano" se un esaminatore non lo distingue da quello di un umano)
Razionale (approccio debole)	Una rete neurale che decide se ci sono le condizioni per attraversare (codifica formale del ragionamento, es. inferenza logica)	Un robot con sonar schiva passanti e auto in modo diverso da un umano, ma funzionalmente efficace (codifica di comportamenti che "fanno la cosa giusta" senza necessariamente imitare meccanismi umani)

Il corso adotta prevalentemente l'ottica "**agire razionalmente**" (in linea con Poole/Nilsson e con Russell & Norvig), perché è la più operativa per progettare e valutare agenti artificiali: non serve che l'agente "pensi come un umano", basta che scelga sempre l'azione che massimizza il risultato atteso.

1.12 Quando serve davvero l'IA?

- **Non è adatta** dove esistono già modelli matematici precisi o metodi algoritmici specifici (in quei casi conviene usare un algoritmo tradizionale).
- **È utile o necessaria** quando si hanno: problemi non deterministici, molteplicità di soluzioni possibili, preferenze tra soluzioni diverse, dati di natura simbolica, conoscenza ampia e incompleta, informazione parzialmente strutturata, necessità di interazione con l'ambiente e con esseri umani.

1.13 Discipline di fondamento dell'IA

L'IA è un campo intrinsecamente interdisciplinare: **Filosofia, Matematica, Economia, Neuroscienze, Psicologia, Informatica, Teoria del controllo e cibernetica, Linguistica.**

Due esempi approfonditi dalle slide:

- **Filosofia — Aristotele** (*Etica Nicomachea*, Libro III): Aristotele osserva che non deliberiamo sui fini (un medico non delibera se debba guarire) ma sui **mezzi** per raggiungerli, una volta posto il fine. Se il fine è raggiungibile con più mezzi, si valuta quale sia il più facile/migliore; si procede a ritroso dai mezzi fino a una "causa prima" (ciò che è ultimo nell'analisi è primo nella costruzione). Questo è considerato il **primo esempio storico di algoritmo di deliberazione backward** (dal goal ai mezzi), concettualmente identico alla ricerca all'indietro usata nella risoluzione di problemi in IA — implementato circa 2300 anni dopo nel **General Problem Solver (GPS)** di Newell & Simon.
- **Psicologia — Ivan Pavlov**: nasce nel XIX secolo lo studio scientifico del comportamento animale/umano. Pavlov studia l'apprendimento per **riflesso condizionato**: la salivazione del cane è un riflesso naturale (incondizionato) davanti al cibo (stimolo incondizionato); associando ripetutamente un campanello (stimolo neutro) alla presentazione del cibo, il campanello da solo diventa uno **stimolo condizionato** che induce salivazione. Questo mostra come un comportamento possa essere modificato dall'esperienza — idea alla base dell'apprendimento automatico.

1.14 Risoluzione automatica di problemi (anticipazione)

Le slide chiudono anticipando i temi successivi del corso: definire cosa sia un problema e una soluzione (distinguendo soluzione da soluzione *ottima*), con tre categorie principali di approcci: 1. ricerca nello spazio degli stati, 2. ricerca in spazi con avversario (giochi a informazione completa), 3. risoluzione di problemi mediante soddisfacimento di vincoli (CSP).

2. Agenti e ambienti

2.1 Agente e ambiente: il binomio fondamentale

Ribadendo la definizione già introdotta: un **agente** è un'astrazione che rappresenta un qualsiasi sistema che **percepisce** il proprio ambiente tramite **sensori** e **agisce** su di esso tramite **attuatori**. Non esistono agenti scollegati da un ambiente: **agente e ambiente formano un binomio inscindibile**. Il simbolo "?" nel diagramma dell'agente rappresenta la **funzione deliberativa**, cioè il meccanismo interno (non ancora specificato) che, sulla base di ciò che è stato percepito, determina quale azione eseguire.

Esempi di sensori fisici reali (per dare concretezza al concetto): sensori di luce/ottici, infrarossi, suono (microfoni), accelerazione (accelerometri), temperatura, calore, radiazione (contatori Geiger), resistenza/corrente/tensione elettrica, magnetismo, pressione, gas e flusso di liquidi, movimento, orientamento, forza (celle di carico, estensimetri), prossimità, distanza, biometrici, chimici.

2.2 Sequenza percettiva

Definizione: la **sequenza percettiva** è la storia completa di tutte le percezioni che un agente ha ricevuto fino a un certo istante.

Un agente sceglie la prossima azione in base alla **situazione in cui si trova**, cioè in base a tutto ciò che ha percepito fino a quel momento (non solo alla percezione istantanea). Il modo più generale (ma anche più oneroso) di realizzare il modulo deliberativo è una **tabella percepito → azione** che associa ad ogni possibile sequenza percettiva un'azione. In alcuni casi è possibile prescindere dalla storia intera e basarsi sulla sola percezione corrente; in altri casi serve davvero l'intera sequenza (o una sintesi di essa, come uno "stato interno").

Esempio: il mondo dell'aspirapolvere

Un piccolo mondo giocattolo con due sole posizioni, A e B, ciascuna delle quali può essere pulita o sporca. Una possibile tabella percepito-azione:

Sequenza percettiva	Azione
[A, pulito]	destra
[A, sporco]	aspira
[B, pulito]	sinistra
[B, sporco]	aspira
[A, pulito],[A, pulito]	destra
[A, pulito],[A, sporco]	aspira
... sequenze di lunghezza crescente	...

Le slide mostrano poi una **seconda tabella diversa** per lo stesso ambiente (dove, ad esempio, [A, pulito] produce "aspira" invece di "destra"): rappresenta un **agente diverso**, con un comportamento differente pur nello stesso ambiente. Questo porta alla domanda cruciale: **due aspirapolvere che si comportano diversamente sono confrontabili? Sulla base di cosa?** Serve un criterio oggettivo per stabilire quale dei due comportamenti sia "migliore" — da qui nasce la nozione di misura di prestazione (§2.3).

2.3 Razionalità

2.3.1 "Fare la cosa giusta"

In termini informali, un agente **razionale** è un agente che "fa la cosa giusta", cioè opera per conseguire il "successo". Per rendere questa idea precisa serve una **misura di prestazione (performance measure)**: un criterio che valuta la bontà degli **stati** attraversati dall'ambiente, definito in funzione dell'effetto che si desidera ottenere (non della singola azione in sé, ma delle sue conseguenze sullo stato del mondo).

Esempio nel mondo dell'aspirapolvere — una possibile misura di bontà degli stati:

Stato	Bontà
[A pulito], [B pulito]	1
[A pulito], [B sporco]	0.5
[A sporco], [B pulito]	0.5
[A sporco], [B sporco]	0

2.3.2 Definizione formale di comportamento razionale

Il comportamento razionale di un agente dipende da **quattro fattori**: 1. le **azioni** nelle facoltà dell'agente (il repertorio di azioni disponibili), 2. la **misura di prestazione** che definisce il criterio di successo, 3. la **conoscenza dell'ambiente** posseduta dall'agente, 4. la **sequenza percettiva** fino a quel momento.

Definizione: un agente razionale dovrebbe scegliere sempre, per ogni possibile sequenza percettiva, l'azione che **massimizza la misura di prestazione attesa**, date la sequenza percettiva stessa e le informazioni derivabili dalla conoscenza dell'ambiente in possesso dell'agente.

Nota il termine "attesa": la razionalità si valuta rispetto a ciò che l'agente può ragionevolmente prevedere dato ciò che sa, **non** rispetto all'esito effettivo (che dipende anche da fattori che l'agente non poteva conoscere).

2.3.3 Razionalità vs. onniscienza/chiaroveggenza

	Razionalità	Onniscienza / chiaroveggenza
Ottimizza Fattori ignoti/imprevedibili	il risultato atteso possono intercorrere e impedire di raggiungere il risultato ideale	il risultato reale per definizione non esistono: si conosce già tutto

Esempio (torta di compleanno): voglio che al compleanno di mio figlio ci sia la torta più bella e meno costosa. Conosco le offerte dei pasticceri della zona, i gusti di mio figlio, ho abbastanza denaro e so fare acquisti: sulla base di tutto ciò, scelgo la torta migliore che conosco. Due cose però potevano sfuggirmi: (a) la nonna, a mia insaputa, porta in regalo una torta identica (informazione che non potevo conoscere), e (b) esiste una nuova pasticceria con un'offerta migliore di cui non ero a conoscenza. In entrambi i casi il mio comportamento **resta razionale**, anche se il risultato non è quello ottimale in assoluto: la mia sequenza percettiva non includeva quelle informazioni, quindi non potevo tenerne conto. Un agente **onnisciente/chiaroveggente**, che sapesse tutto in anticipo, avrebbe invece ottenuto il risultato migliore in assoluto (es. non comprare nessuna torta sapendo che arriva quella della nonna).

Punto fondamentale: **razionale non significa "di successo" né "onnisciente"**. Significa fare la scelta migliore possibile *dato ciò che si sa*.

Razionalità e apprendimento: se un anno dopo, sapendo ormai che la nonna tende a fare la torta perfetta per il compleanno, io comprassi comunque la torta senza prima verificare, il mio comportamento **non sarebbe più razionale**, perché ora quella conoscenza fa parte della mia sequenza percettiva/esperienza pregressa. Questo mostra che **un agente razionale è tanto più efficace quanto più ha capacità di imparare dall'esperienza** e di modificare il proprio comportamento futuro di conseguenza.

2.3.4 La percezione non è un atto passivo

Esempio (attraversare la strada): è razionale che un robot attraversi la strada senza prima guardare a destra? **No**. Prima di eseguire un'azione, un agente deve valutare se ha raccolto informazioni sufficienti oppure se è necessario compiere ulteriori **atti percettivi** (es. guardare) prima di agire. Questo mostra che la **razionalità dipende dalla sequenza percettiva, ma può anche influenzarla**: un agente razionale può scegliere di percepire di più prima di decidere.

? Domanda d'esame: quale aspirapolvere è più razionale? Sono definibili comportamenti ancora più razionali? Non si può rispondere senza fissare prima una misura di prestazione esplicita (es. quella della tabella con bontà degli stati) e senza conoscere l'ambiente e le azioni disponibili. Rispetto a una misura che premia il numero di celle pulite nel tempo, un agente che aspira solo quando percepisce "sporco" ed evita mosse inutili è più razionale di uno che, ad esempio, spreca energia muovendosi anche quando la cella corrente è già pulita senza una strategia. Esistono comportamenti "ancora più razionali", ad esempio agenti che tengono conto anche del consumo energetico, del tempo impiegato, o che imparano il layout dell'ambiente per pianificare percorsi più efficienti: dipende sempre da **quali effetti desiderati** vengono inclusi nella misura di prestazione.

2.4 Task environment e PEAS

Task environment: il contesto in cui l'agente è inserito. Può essere **fisico** (reale o simulato, tipico per agenti robotici) oppure, ad esempio, **sociale**, comprendendo le relazioni con altri agenti (tipico per agenti software).

PEAS è l'acronimo che identifica i quattro elementi che definiscono formalmente un task environment, e va sempre specificato quando si progetta un agente: - **Performance measure** — la misura di prestazione rispetto a cui valutare il successo dell'agente, - **Environment** — l'ambiente in cui l'agente opera, - **Actuators** — gli attuatori a disposizione dell'agente per agire, - **Sensors** — i sensori a disposizione dell'agente per percepire.

Specificare il PEAS è il primo passo, sistematico e imprescindibile, nella progettazione di qualunque agente: obbliga a chiarire esplicitamente cosa l'agente deve ottenere, dove opera, come può agire e cosa può percepire.

2.5 Proprietà dell'ambiente (task environment)

Ogni ambiente può essere caratterizzato lungo sette dimensioni indipendenti. Questa classificazione è fondamentale perché **determina quali tecniche di progettazione dell'agente sono necessarie o sufficienti** (ambienti più "difficili" richiedono agenti più sofisticati).

Proprietà	Valore 1	Valore 2
Osservabilità	Completamente osservabile: in ogni istante i sensori danno accesso a tutti gli aspetti dell'ambiente rilevanti per scegliere l'azione	Parzialmente osservabile: i sensori danno accesso solo a parte dell'informazione rilevante (per sensori imprecisi o incapaci di rilevare certi dati)
Determinismo	Deterministico: lo stato successivo è determinato univocamente dallo stato corrente e dall'azione eseguita	Stocastico: applicando più volte la stessa azione nello stesso stato si possono raggiungere stati diversi. Caso particolare: strategico se lo stocasticismo dipende solo dalle azioni di altri agenti (non dal caso puro)
Episodicità	Episodico: l'esperienza è divisa in episodi atomici indipendenti, ciascuno = una percezione seguita da una singola azione (es. classificazione di immagini)	Sequenziale: l'attività è composta da più passi collegati, ognuno dei quali influenza in generale i successivi
Dinamicità	Statico: l'ambiente non cambia mentre l'agente "pensa" (cioè mentre decide quale azione eseguire)	Dinamico: l'ambiente può cambiare mentre l'agente sta ancora decidendo
Discretezza	Discreto: stato, tempo, percezioni e azioni sono discreti (es. gli scacchi: stati, percezioni e azioni discreti)	Continuo: stato, tempo, percezioni o azioni sono continui (es. gli scacchi hanno tempo continuo, se cronometrati)
Numero di agenti	Singolo agente: viene modellata come agente una sola entità	Multiagente: vengono modellate come agenti più entità distinte

Note importanti (chiarimenti impliciti nelle slide):

- **Connessione tra parzialmente osservabile e stocastico:** spesso un ambiente appare stocastico proprio perché è parzialmente osservabile — non percependo tutti gli aspetti rilevanti, l'agente non riesce a prevedere con certezza l'esito delle proprie azioni, anche se "in realtà" il mondo sottostante

è deterministico. Esempio: una batteria la cui carica reale si consuma in modo continuo, ma l'agente percepisce solo tre stati discreti (bassa/media/alta). La stessa azione "passo", eseguita quando la carica percepita è "media", può risultare in "carica bassa" oppure "rimane media", perché l'agente non vede il livello esatto e continuo di carica residua.

- **Come decidere cosa modellare come agente (singolo/multiagente):** il progettista deve decidere quali entità del mondo trattare come agenti e quali come parte dell'ambiente. Criterio: vanno modellate come agenti le entità il cui comportamento tenta di **massimizzare una propria misura di prestazione che dipende anche dal comportamento di altri agenti** (interazione strategica). Se un'entità si comporta in modo puramente meccanico/prevedibile senza obiettivi propri, può essere trattata come parte dell'ambiente.
- **Il caso più complesso e generale** (e più realistico per problemi reali) è un ambiente **parzialmente osservabile, stocastico, sequenziale, dinamico, continuo e multiagente**.

Esempi di caratterizzazione di ambienti (dalle slide)

Ambiente	Osservabilità	Agenti	Determinismo	Episodicità	Dinamicità	Discretezza
Parole crociate	totale	singolo	deterministico	sequenziale	statico	discreto
Guidare un taxi	parziale	multiagente	stocastico	sequenziale	dinamico	continuo
Tutor di inglese interattivo	parziale	multiagente	stocastico	sequenziale	dinamico	discreto
Analizzare di immagini	totale	singolo	deterministico	episodico	statico	continuo

Questi esempi mostrano bene lo spettro di difficoltà: le parole crociate sono l'ambiente "più semplice" possibile (tutte le proprietà nella colonna "facile"), guidare un taxi è vicino al caso più complesso possibile (quasi tutte le proprietà nella colonna "difficile") — coerentemente col fatto che guidare in un ambiente reale è un problema molto più arduo da automatizzare rispetto a risolvere cruciverba.

Esempio guida: il termostato

- **Percepisce:** la temperatura della stanza.
- **Delibera:** decide quale azione eseguire.
- **Agisce:** accende/spegne il riscaldamento, oppure non fa nulla (comportamento predefinito).

È l'esempio più semplice possibile di ciclo percezione→delibera→azione: mostra il funzionamento essenziale di un agente anche senza nessuna sofisticazione interna.

Esempio guida: robot aspirapolvere (Roomba e simili)

- **Percepisce:** ostacoli (oggetti, scale) e livello della batteria.
- **Delibera:** decide quale azione eseguire.
- **Agisce:** si muove, si ferma, gira, oppure va alla stazione di ricarica (comportamento predefinito quando la batteria è scarica).

Da notare: sia il termostato sia il Roomba hanno un "comportamento predefinito" per i casi non altrimenti gestiti (non fare nulla / andare a ricaricarsi) — un modo semplice per garantire che l'agente abbia sempre un'azione definita anche in assenza di condizioni specifiche riconosciute.

2.6 Agente = architettura + programma

- **Architettura:** la specifica degli elementi strutturali e funzionali su cui il programma agente viene eseguito (l'hardware/piattaforma, es. un robot fisico o una macchina virtuale).
- **Programma:** la funzione che mette in relazione le percezioni con le azioni.

Distinzione importante tra due nozioni spesso confuse: - **Funzione agente:** un'astrazione matematica che ha come input **l'intera sequenza percettiva** (la storia completa delle percezioni) e restituisce un'azione. È una descrizione ideale, esaustiva, di come l'agente *dovrebbe* comportarsi per ogni possibile storia di percezioni. - **Programma agente:** l'implementazione concreta, che tipicamente ha come input solo la **percezione corrente** (eventualmente insieme a uno stato interno che riassume la storia passata, per non dover ricordare l'intera sequenza percettiva esplicitamente).

2.7 Tipologie di agente

Le slide presentano cinque tipologie di agente, in ordine di sofisticazione crescente. Questa è una delle tassonomie più importanti del modulo (corrisponde alla struttura del cap. 2 di Russell & Norvig).

2.7.1 Agenti reattivi semplici

Basano la scelta dell'azione **solo sulla percezione corrente**, ignorando la storia passata. Il programma codifica direttamente delle **regole condizione-azione** ("se percepisco X, faccio Y"). È un agente scritto ad hoc per un problema specifico.

Esempio (aspirapolvere reattivo):

```
Aspirapolvere-reattivo(posizione, stato) return azione
{
  if (stato = sporco) then return aspira
  else if (posizione = A) then return destra
  else if (posizione = B) then return sinistra
}
```

Schema generale:

```
Agente-reattivo-semplce(percezione) return azione
{
  stato ← interpreta(percezione)
  regola ← individua-regola(stato, regole)
  azione ← regola-azione(regola)
  return azione
}
```

La percezione viene interpretata per identificare uno stato; lo stato individua la regola applicabile; la regola determina l'azione da eseguire. L'insieme delle regole definisce interamente il comportamento dell'agente.

Limiti degli agenti reattivi semplici: - Funzionano correttamente solo in ambienti **completamente osservabili**. - Esempio: se l'aspirapolvere non ha un sensore di posizione, e la cella corrente è pulita, non sa in che direzione muoversi per essere efficace (non sa dove si trova nel mondo). Questo può generare **loop infiniti** (es. [pulito] → destra → [pulito] → destra → ...) perché l'agente ripete sempre la stessa scelta in presenza della stessa percezione. Per rompere questi loop occorre introdurre comportamenti **casuali (random)**. - L'azione deve essere determinabile dalla sola percezione corrente: se, ad esempio, l'aspirapolvere potesse leggere un solo sensore alla volta (o sporco/pulito, o posizione, ma non entrambi insieme), non saprebbe come comportarsi in modo affidabile.

2.7.2 Agenti reattivi basati su modello

In caso di **osservabilità parziale**, un agente puramente reattivo non basta: serve un **modello del mondo**, cioè una rappresentazione interna di **come il mondo evolve** e di **quali effetti hanno le azioni**. L'agente mantiene uno **stato interno** che viene aggiornato combinando: (1) la percezione corrente, (2) lo stato interno precedente, (3) l'ultima azione eseguita, (4) il modello del mondo.

Schema:

```

Agente-reattivo-con-modello(percezione) return azione
{
    stato ← aggiorna-stato(stato, azione, percezione)
    regola ← individua-regola(stato, regole)
    azione ← regola-azione(regola)
}

```

Esempio (batteria): il modello dice che "tre passi consumano una tacca di batteria". Se la sequenza percettiva ha mostrato "2 tacche" per tre volte consecutive dopo tre azioni "passo", il modello permette di **prevedere** che dopo il prossimo "passo" la batteria scenderà a 1 tacca, anche se il sensore, in un dato istante, non fornisce l'informazione precisa e continua sul livello di carica. Avere un modello permette quindi di **prevedere gli effetti delle azioni** e scegliere l'azione anche sulla base di questa previsione, superando il non-determinismo apparente dovuto all'osservabilità parziale (si ricollega alla nota del §2.5 su parziale osservabilità/stocasticità).

2.7.3 Agenti basati su obiettivi (goal-driven)

L'agente sceglie l'azione da eseguire in base ai propri **obiettivi**: l'azione scelta deve avvicinarlo all'obiettivo o farglielo raggiungere. La decisione può riguardare solo il prossimo singolo passo, oppure può richiedere di guardare più avanti nel tempo, costruendo un **piano** di più passi.

Meccanismo tipico: il **ragionamento ipotetico** — l'agente "simula" mentalmente l'effetto delle azioni possibili per valutare se e quanto lo avvicinano all'obiettivo, prima di eseguirle realmente.

Vantaggio chiave: separando esplicitamente l'obiettivo dal meccanismo di ragionamento, basta **cambiare l'obiettivo** per ottenere comportamenti completamente diversi dallo stesso agente, senza riscrivere le regole di comportamento (a differenza dell'agente reattivo, dove il comportamento è "cablato" nelle regole condizione-azione).

2.7.4 Agenti basati sull'utilità (utility-driven)

Alla nozione di obiettivo (raggiunto/non raggiunto, binario) si aggiunge una **misura di prestazione più fine**, calcolata da una **funzione di utilità**, che valuta quanto "buona" sia una scelta tenendo conto di più fattori contemporaneamente: costo, velocità, difficoltà attuativa, tempo, risorse utilizzate, ecc.

Esempio: per raggiungere una località possono esserci più percorsi alternativi (strade sterrate, percorso cittadino, strada veloce a pagamento). Il problema non è solo *trovare una soluzione* (un qualunque percorso che arrivi a destinazione) ma trovarne una che **massimizzi la soddisfazione** ("ci rende felici") secondo criteri come rapidità, costo, comodità.

Distinzione obiettivo/utilità: - **Obiettivo (goal):** uno stato particolare che si vuole raggiungere, tipicamente descritto in termini di proprietà da soddisfare (es. "essere arrivato a destinazione"). - **Utilità:** una funzione che, dato uno stato (o una sequenza di stati), restituisce una misura numerica di bontà. - L'agente ha quindi un **duplice problema**: (1) raggiungere l'obiettivo, e (2) farlo massimizzando l'utilità. Da qui la distinzione tra **soluzioni** (qualunque sequenza di azioni che raggiunge il goal) e **buone soluzioni** (quelle che, tra le soluzioni possibili, massimizzano l'utilità).

2.7.5 Agenti che apprendono

Aggiungono all'agente "classico" (che percepisce, decide, agisce) un ulteriore componente dedicato all'**apprendimento**, composto da tre elementi: 1. **Critico**: valuta il livello di prestazione dell'agente e decide se e quando attivare l'apprendimento (confrontando il comportamento con uno standard esterno di prestazione). 2. **Modulo di apprendimento**: modifica effettivamente la conoscenza/il comportamento dell'agente sulla base del feedback del critico. 3. **Generatore di problemi**: suggerisce/causa l'esecuzione di **azioni esplorative**, il cui scopo è esporre l'agente a nuove esperienze (anche non immediatamente ottimali) da cui poter imparare qualcosa di utile per il futuro.

Le slide precisano che non si fanno assunzioni su **quali segnali o tecniche specifiche** l'agente usi per apprendere: il concetto di "agente che apprende" è uno schema architetturale generale, indipendente dall'algoritmo di apprendimento concreto (che sarà oggetto di moduli successivi del corso, es. machine learning).

2.8 Tabella riassuntiva delle tipologie di agente

Tipo di agente	Cosa usa per decidere	Punti di forza	Limiti
Reattivo semplice	Solo la percezione corrente + regole condizione-azione	Semplice, veloce, poco costoso da implementare	Funziona solo se l'ambiente è completamente osservabile; rischio di loop infiniti; nessuna "memoria"
Reattivo basato su modello	Percezione corrente + stato interno + modello di come evolve il mondo	Gestisce l'osservabilità parziale, "compensa" i limiti dei sensori con la conoscenza	Il modello va costruito e mantenuto correttamente; ancora nessun concetto esplicito di obiettivo
Basato su obiettivi	Percezione + modello + un obiettivo esplicito da raggiungere (eventuale pianificazione/ricerca)	Flessibile: basta cambiare l'obiettivo per cambiare comportamento; permette ragionamento ipotetico "what-if"	Non distingue tra soluzioni diverse che raggiungono comunque l'obiettivo (nessuna nozione di "quanto bene")
Basato sull'utilità	Come sopra + una funzione di utilità che valuta la qualità degli stati/soluzioni	Permette di scegliere la <i>migliore</i> tra più soluzioni possibili, bilanciando più criteri (costo, tempo, rischio...)	Richiede di saper definire e calcolare una funzione di utilità adeguata, spesso non banale
Che apprende	Tutto quanto sopra + un ciclo di apprendimento (critico, modulo di apprendimento, generatore di problemi)	Migliora nel tempo, si adatta a condizioni non previste in fase di progettazione	Maggiore complessità architetturale; richiede segnali di valutazione (feedback) adeguati

3. Paradigmi di programmazione e agenti autonomi

3.1 Dall'approccio tradizionale (imperativo/a oggetti) all'approccio dichiarativo

Le slide confrontano due modi opposti di strutturare il software, per far capire perché il software di IA richiede spesso un cambio di paradigma rispetto alla programmazione tradizionale.

Approccio tradizionale — paradigma imperativo / a oggetti (non-AI software): - Risolve **un singolo compito specifico**. - È tipicamente strutturato come una **sequenza di passi** che descrivono esplicitamente **come (how)** ottenere il risultato. - Esempio dalle slide: una funzione C che inserisce un elemento in una lista ordinata (`ins_ord`), scritta come una sequenza esplicita di operazioni sui puntatori — il programmatore ha già codificato *passo passo* l'algoritmo che produce il risultato.

Approccio dichiarativo (AI software): - Separa una **descrizione dichiarativa** del problema (**cosa (what)** si vuole/si sa) da un **programma generale** (motore di inferenza/ricerca) che sa elaborare tale descrizione. - Lo **stesso programma generale** può essere applicato a **descrizioni diverse** per risolvere **problemi diversi**, senza essere riscritto. - Architettura tipica: una **base di conoscenza (Knowledge Base)** contiene la rappresentazione dichiarativa di un corpo di conoscenze applicabile a molteplici situazioni; le **percezioni** vengono via via integrate in questa conoscenza; un **modulo deliberativo** generale interroga (query) la base di conoscenza per decidere l'azione.

Perché questo cambio di paradigma è importante per l'IA? Perché nei problemi "intelligenti" tipicamente non conosciamo a priori la sequenza esatta di passi che risolve il problema (a differenza dell'inserimento ordinato in una lista, che ha un algoritmo noto e fisso): vogliamo invece che sia il sistema stesso, dato un obiettivo e una descrizione del mondo, a **costruire** la sequenza di passi giusta. Per questo, come si vede nell'esempio del mondo dei blocchi, "il programma dell'agente costruisce un programma" — cioè elabora la conoscenza (le mosse possibili) per generare esso stesso la soluzione, invece di eseguire una soluzione già scritta dal programmatore.

3.2 Esempio guida: il mondo dei blocchi (Blocks World)

Il **mondo dei blocchi** è un classico problema giocattolo (*toy problem*) dell'IA, usato per illustrare in modo semplice il funzionamento di un agente dichiarativo/deliberativo.

Setup: alcuni blocchi (etichettati A, B, C, D, E, F, G nelle slide) sono disposti impilati su un piano diviso in postazioni (1, 2, 3, 4). L'agente percepisce la configurazione (stato) **iniziale** dei blocchi.

Passaggi concettuali: 1. **Percezione:** l'agente percepisce la situazione iniziale. Di per sé, in questo momento, **non fa nulla:** la sola percezione non genera azione. 2. **Definizione del goal:** per agire, occorre prima descrivere un **obiettivo** (goal), cioè uno stato di cose che si vuole rendere vero (es. una certa configurazione finale desiderata dei blocchi, diversa da quella iniziale). 3. **Costruzione del piano:** l'agente deve costruire autonomamente la sequenza di passi/azioni ($a_1, a_2, \dots, a_k$) che porta dallo stato iniziale (I) allo stato goal (G): $I \rightarrow a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_k \rightarrow G$. 4. **Conoscenza delle azioni:** per farlo, l'agente deve avere/usare conoscenza su quali azioni sono disponibili (es. "Prendi(blocco)", "Impila(blocco, blocco)", "Metti(blocco, posizione)"), su quando ciascuna azione è **applicabile** (precondizioni) e quali sono i suoi **effetti** sul mondo. 5. **Ragionamento:** l'agente deve ragionare per determinare quali azioni, tra tutte quelle applicabili, effettivamente avvicinano (o portano) al goal.

Perché serve ricerca: localmente, nello stato iniziale, potrebbero esserci molte mosse possibili (es. Prendi(B), Prendi(C), Prendi(E)), ma solo alcune di esse sono davvero utili per raggiungere lo stato finale desiderato. Il problema di **come scegliere** tra le mosse possibili è esattamente il problema della **ricerca nello spazio degli stati**, che verrà approfondito nei moduli successivi.

Soluzione trovata dall'agente (esempio dalle slide): una sequenza concreta come Prendi(C), Metti(C,4), Prendi(B), Impila(B,D), Prendi(C), Impila(C,A), ... ecc. Il punto concettuale fondamentale è che **il programma dell'agente costruisce un programma:** elabora la conoscenza sulle mosse possibili per generare autonomamente la sequenza di azioni risolutiva, che il progettista non aveva scritto esplicitamente in anticipo.

Domande aperte lasciate dalle slide (che anticipano temi futuri): possono esistere più soluzioni alternative allo stesso problema? Tra più soluzioni possibili, qual è la migliore? (Questo si ricollega alla distinzione soluzione/soluzione ottima e agli agenti basati sull'utilità.)

3.3 Dal problema giocattolo al problema reale

Il mondo dei blocchi è deliberatamente un **toy problem**: stati discreti, ambiente completamente osservabile, deterministico, un solo agente. Le slide chiedono esplicitamente: *cosa succede se ci spostiamo nel mondo reale?* — riprendendo l'esempio dell'attraversamento della strada, che è invece un ambiente **complesso, parzialmente prevedibile, parzialmente collaborativo**.

Esempio: identificare un passaggio pedonale da un'immagine reale. Quando una telecamera cattura un'immagine di strisce pedonali, non "vede" delle strisce come le vediamo noi: cattura solo **pixel**, cioè codifiche numeriche di colori con cui l'immagine viene approssimata digitalmente. Fattori che complicano ulteriormente il problema: prospettiva, colore, qualità dell'immagine, necessità di estrarre l'oggetto dal contesto, modi alternativi con cui la stessa scena può essere rappresentata.

Punto concettuale chiave: "vedere" (per un agente artificiale) non significa incamerare passivamente dei dati grezzi, ma **elaborare quei dati fino a trasformarli in informazioni**, secondo modelli che noi umani (e molti animali) sappiamo costruire autonomamente e quasi senza sforzo cosciente. Le slide notano anche un dettaglio sottile: "vediamo" le strisce pedonali (cioè le notiamo, le processiamo come rilevanti) **solo quando ci interessa attraversare** — la percezione utile è già guidata da un obiettivo/contesto, non è un processo neutro.

Questo esempio mostra la distanza enorme tra un toy problem come il mondo dei blocchi (dati già simbolici e puliti: "il blocco A è sopra B") e un problema reale (dati grezzi, numerici, ambigui, da interpretare) — ed è uno dei motivi per cui l'IA moderna dedica un'enorme quantità di sforzi alla percezione (visione artificiale, elaborazione del linguaggio, ecc.), argomenti trattati in moduli successivi.

3.4 Da automazione ad autonomia

- **Automazione:** ormai uno standard consolidato in moltissime attività. Richiede di programmare il dispositivo per compiere esplicitamente **ogni singolo passo**. È applicabile bene in domini fortemente **ripetitivi** e prevedibili (es. un robot di saldatura in una catena di montaggio industriale, che esegue sempre la stessa sequenza fissa di movimenti).
- **Autonomia:** un agente artificiale autonomo riceve **compiti ad alto livello** (goal), e l'utente/operatore demanda all'agente stesso il compito di trovare **come** risolverli concretamente — non gli vengono più specificati i singoli passi.

Agente autonomo — caratteristiche: - In quanto agente, possiede capacità di azione. - Essendo *autonomo*, in particolare: - riceve solo compiti ad alto livello (non istruzioni passo-passo), - ragiona ed esplora alternative (spesso in numero esponenziale, dato che a ogni istante possono esserci molte mosse possibili), - riconosce quando una linea di azione non è più percorribile (vicolo cieco), - riconosce se è già stato in una situazione già esplorata (evitando cicli/ripetizioni inutili), - semplificando: **prima ragiona e poi agisce**, cioè costruisce un piano (un "programma" per sé stesso) e poi lo esegue.

Autonomia e controllabilità — un equivoco comune: al di fuori della comunità di IA, gli agenti autonomi vengono talvolta percepiti come "*a self-conscious, uncontrollable entity whose autonomy emerges as a property extra-program*" — cioè come entità semi-coscienti e incontrollabili la cui autonomia "emerge" al di fuori/al di sopra del programma che le governa, quasi per magia, e che potrebbero prendere iniziative mai codificate.

Le slide **correggono esplicitamente** questo fraintendimento: in IA, gli agenti autonomi sono semplicemente **un modo di concepire i programmi**, in cui il **controllo** (la logica di alto livello, il modello del mondo e degli obiettivi) è chiaramente **separato** dai dettagli implementativi di basso livello. **Un agente fa sempre e soltanto ciò che è stato programmato a fare** — semplicemente, ciò che è stato programmato è "perseguire questo obiettivo trovando tu stesso i passi necessari", non "eseguire questi passi specifici". Non c'è nulla di

misterioso o “emergente extra-programma”: l'autonomia è essa stessa un costrutto di progettazione, non un'evasione dal programma.

? Domanda d'esame: un agente autonomo può fare cose che non erano previste dal suo programma? No, nel senso tecnico usato in IA. Un agente autonomo esegue sempre il proprio programma; la differenza rispetto a un sistema automatizzato tradizionale è che il programma stesso è scritto per **ragionare e generare piani** in risposta a obiettivi di alto livello, invece di eseguire una sequenza di passi fissata a priori dal programmatore. Le azioni “nuove” che l'agente compie sono il risultato del ragionamento interno (esplorazione di alternative, pianificazione), non un'evasione dal codice: rimangono sempre “ciò che il programma, dato l'input, calcola di fare”. La percezione popolare di un'autonomia “incontrollabile ed emergente” è quindi un fraintendimento da correggere.

3.5 Gradi di autonomia: tre esempi concreti a confronto

Le slide usano tre sistemi reali per mostrare che **autonomia non è un concetto binario**, ma una questione di **grado**, che dipende dalla complessità dell'ambiente in cui l'agente opera.

Sistema	Ambiente	Sensori	Azioni	Grado di autonomia
Metropolitana automatica	“Semplice”: una rotaia fissa, un punto di partenza e uno di arrivo definiti	velocità, accelerazione, posizione, stato porte	accelera / decelera	Bassa: l'azione successiva è calcolata in modo deterministico dato lo stato; percorso fisso, nessuna vera decisione strategica
Rover su Marte	Ancora relativamente “semplice”: una piana più o meno regolare, qualche roccia, ma niente altri agenti né fenomeni atmosferici complessi da gestire	distanza, contatto, telecamera, velocità, accelerazione, posizione	accelera/decelera, gira	Media/parziale: introduce la nozione di “goal programmabile” — un operatore umano assegna solo la destinazione, non i singoli movimenti. Necessaria per via del ritardo di comunicazione radio (~14 minuti in media): il rover deve poter decidere da solo come muoversi verso il goal senza attendere istruzioni in tempo reale

Sistema	Ambiente	Sensori	Azioni	Grado di autonomia
Auto senza guidatore	Complesso: strada affollata di altri agenti (auto, pedoni, animali), condizioni atmosferiche variabili (pioggia), giorno/notte, comportamento altrui imprevedibile	distanza, contatto, telecamera, velocità, accelerazione, posizione	accelera/decelera, gira	Alta: il goal (destinazione) è fornito dall'operatore, ma l'agente deve gestire in autonomia un ambiente multiagente e fortemente imprevedibile

Tutti e tre condividono lo stesso **ciclo agente** di base:

Agent loop:

- 1) Leggi i sensori
- 2) Applica la funzione di controllo alle letture (+ eventualmente il goal / la prossima tappa) e
 ↪ calcola la prossima azione (accelerazione/decelerazione, sterzata...)
- 3) Applica l'azione calcolata

Ciò che cambia drasticamente tra i tre casi non è la struttura del ciclo, ma la **complessità dell'ambiente** (numero di altri agenti, prevedibilità, dinamicità) e di conseguenza quanto la "funzione di controllo" al passo 2 debba essere sofisticata: da un calcolo deterministico semplice (metro) fino a un vero e proprio ragionamento su un ambiente multiagente incerto (auto autonoma).

3.6 Ragionare basta?

No. Il ragionamento da solo non è sufficiente per un agente reale, perché tipicamente si opera: - in presenza di **incertezza**, - in un mondo **non completamente conosciuto**, - in presenza di **altri agenti** (persone o altri robot), i cui comportamenti non sono del tutto prevedibili o controllabili.

Per questo occorre **combinare** tre ingredienti insieme, in un ciclo continuo: **azioni** (che modificano concretamente il mondo), **percezioni** (che informano sullo stato del mondo, anche dopo che l'agente ha agito), e **ragionamento** (che decide cosa fare sulla base di percezioni e conoscenza). Nessuno dei tre, da solo, è sufficiente: il ragionamento senza percezione aggiornata lavorerebbe su informazioni obsolete; la percezione senza ragionamento non produrrebbe decisioni; senza azione, nulla di quanto deciso avrebbe effetto sul mondo.

Esempio applicativo reale — miniere automatizzate: aziende come Rio Tinto Group (Australia), la miniera di Bingham Canyon (Utah), o EEP Elektro-Elektronik Pranjic (azienda tedesca che automatizza miniere in Cina) utilizzano camion autonomi (es. Komatsu driverless trucks) per attività come trivellazione e brillamento autonomi, controllo di flotte di veicoli, trasporto autonomo del materiale (*autonomous haulage*), evitamento di ostacoli e navigazione. È un esempio concreto di dominio industriale reale in cui l'autonomia (nel senso tecnico appena definito) è già impiegata su larga scala.

3.7 Approfondimento facoltativo: sistemi multiagente (MAS)

(Contrassegnato "FACOLTATIVO" nelle slide — riportato per completezza.)

- Un **sistema multiagente (MAS)** è costituito da un insieme di agenti che operano in uno stesso ambiente, potendo **competere o collaborare** nell'uso delle risorse condivise e nel perseguimento dei propri obiettivi (individuali o comuni).

- Per interagire efficacemente, agenti anche implementati in modo indipendente ed eterogeneo devono condividere: (1) un'**ontologia** del dominio del discorso (per attribuire lo stesso significato ai termini usati), (2) un **linguaggio di comunicazione** comune, (3) un **protocollo di interazione** condiviso (uno schema che coordina lo scambio di messaggi tra due o più agenti).
- **FIPA** (Foundation for Intelligent Physical Agents) ha standardizzato, a inizio anni 2000, una semantica formale per gli **atti comunicativi** (speech acts, es. Request, Agree, Confirm, Cancel, Inform, ecc.) e diversi protocolli di interazione standard (es. Contract Net, Brokering, Query Interaction Protocol).
- Esempio di atto comunicativo (request): l'agente *i* chiede all'agente *j* di consegnare una scatola (box017) in una certa posizione; *j* può rispondere con un agree, specificando eventualmente condizioni (es. bassa priorità).
- Strumenti software citati per l'implementazione di MAS: **JADE**, **Jason**, **CartAgO**, **JaCaMo**.

Riepilogo e punti chiave

- **IA** = intelligenza non naturale, ottenuta con procedimenti tecnici. Nasce ufficialmente nel 1956 (Dartmouth Conference, John McCarthy), ma affonda le radici nei lavori di Turing (Macchina di Turing 1936, Test di Turing 1950).
- **Automazione** ≠ **intelligenza**: eseguire automaticamente un programma non basta; serve capacità di adattarsi, decidere, eventualmente imparare.
- **Test di Turing**: valuta solo il comportamento osservabile (indistinguibilità umano/macchina); non garantisce comprensione reale (vedi esempio Valeria/Rossella: stesso output, comprensione diversa).
- **Stanza cinese (Searle)**: manipolare simboli sintatticamente secondo regole (come fa un programma) non implica comprensione semantica/intenzionalità — critica filosofica al comportamentismo del Test di Turing.
- **Strong AI** (riprodurre davvero l'intelligenza/il pensiero umano) vs. **Weak AI** (costruire sistemi che risolvono task "intelligenti" in modo task-oriented, senza doverne replicare i meccanismi interni umani); il corso adotta l'ottica "agire razionalmente" (weak AI).
- Le **quattro scuole** di definizione dell'IA nascono dall'incrocio di due assi: pensiero/comportamento × fedeltà umana/razionalità.
- **Agente** = astrazione che percepisce l'ambiente tramite sensori e agisce tramite attuatori; agente e ambiente sono un binomio inscindibile; la funzione deliberativa ("??") decide l'azione in base a ciò che è stato percepito.
- **Sequenza percettiva** = storia completa delle percezioni ricevute; la scelta dell'azione può dipendere dall'intera sequenza o solo dalla percezione corrente.
- **Razionalità**: un agente razionale sceglie sempre l'azione che massimizza la **misura di prestazione attesa**, data la sequenza percettiva e la conoscenza dell'ambiente disponibili — dipende da 4 fattori: azioni disponibili, performance measure, conoscenza dell'ambiente, sequenza percettiva.
- **Razionale** ≠ **onnisciente**, **razionale** ≠ **"di successo"**: la razionalità si valuta rispetto a ciò che si poteva ragionevolmente sapere, non rispetto al risultato ideale con informazione completa. Un agente razionale migliora se impara dall'esperienza. La percezione stessa è parte della decisione razionale (es. guardare prima di attraversare).
- **PEAS** (Performance, Environment, Actuators, Sensors) = i quattro elementi che definiscono un task environment; primo passo obbligato nella progettazione di ogni agente.
- Le **7 proprietà dell'ambiente**: osservabilità (completa/parziale), determinismo (deterministico/stocastico), episodicità (episodico/sequenziale), dinamicità (statico/dinamico), discretezza (discreto/continuo), numero di agenti (singolo/multiagente). Il caso più complesso: parzialmente osservabile + stocastico + sequenziale + dinamico + continuo + multiagente.
- **Agente** = **architettura** + **programma**; distinzione tra **funzione agente** (input: intera sequenza percettiva, astrazione ideale) e **programma agente** (input: percezione corrente + eventuale stato interno, implementazione reale).

- **Cinque tipi di agente**, crescenti in sofisticazione: reattivo semplice → reattivo basato su modello → basato su obiettivi → basato sull'utilità → che apprende. Ognuno risolve i limiti del precedente (osservabilità parziale, scelta tra soluzioni alternative, miglioramento nel tempo).
- **Paradigma dichiarativo** (tipico del software di IA): separa la descrizione *cosa si sa/si vuole* (knowledge base) da un motore generale di ragionamento/ricerca che genera esso stesso la soluzione, a differenza del paradigma imperativo che codifica esplicitamente *come* risolvere un singolo problema specifico.
- **Mondo dei blocchi**: esempio classico di toy problem per illustrare come un agente, dati uno stato iniziale, un goal e la conoscenza delle azioni disponibili, debba esso stesso costruire (tramite ricerca/ragionamento) la sequenza di passi risolutiva — “un programma che costruisce un programma”.
- **Automazione vs. autonomia**: l'automazione richiede di specificare ogni passo (adatta a domini ripetitivi); l'autonomia consiste nel ricevere solo obiettivi di alto livello e nel dedurre da sé i passi necessari, tramite ragionamento ed esplorazione di alternative. L'autonomia **non** significa comportamento incontrollabile o “extra-programma”: un agente autonomo esegue sempre e solo il proprio programma, che è però progettato per pianificare, non per eseguire passi fissi.
- L'autonomia è **graduale**, non binaria: dipende dalla complessità dell'ambiente (esempi: metropolitana automatica → rover marziano → auto a guida autonoma, con complessità e imprevedibilità crescenti).
- Il ragionamento da solo non basta in presenza di incertezza, mondo parzialmente conosciuto e altri agenti: serve sempre combinare **azioni + percezioni + ragionamento** in un ciclo continuo.

Modulo 2 — Risoluzione di problemi mediante ricerca (non informata e informata)

Questo modulo affronta il primo dei tre grandi approcci alla risoluzione automatica di problemi in IA: 1. **ricerca nello spazio degli stati** (oggetto di questo modulo), 2. ricerca in spazi con avversario (giochi ad informazione completa), 3. risoluzione di problemi mediante soddisfacimento di vincoli (CSP).

L'idea di fondo è che moltissimi problemi si possano modellare come un insieme di **stati** collegati da **azioni**, e che risolvere il problema equivalga a trovare un **percorso** (sequenza di azioni) che porti da uno stato iniziale a uno stato che soddisfa un obiettivo.

Stati, azioni e astrazione

Alla base di tutto c'è l'idea di astrarre la realtà in un insieme discreto di **stati**, che transitano l'uno nell'altro tramite l'esecuzione di **azioni** (operazioni). Esempio minimale: stato {dentro, fuori}, azione {attraversa_soglia} — rappresentabile come un grafo di transizione di stato.

Caratteristiche tipiche di questa astrazione: - **stati discreti**: anche se nel mondo fisico un fenomeno è continuo (es. il passaggio graduale attraverso una soglia), lo si discretizza in un numero finito di valori (idea analoga alla discretizzazione di una funzione continua in un istogramma); - **effetto deterministico delle azioni** (nel caso base — esistono varianti non deterministiche, es. l'azione "passo" che dallo stato "dentro" può portare a due esiti diversi a seconda della posizione non rappresentata nella stanza, oppure azioni a valori continui come "passo(spinta)"); - **dominio statico**: il mondo non cambia se non per effetto delle azioni dell'agente.

Il principio guida dell'astrazione è **rappresentare solo l'informazione rilevante al problema**: per uno stato conta la posizione ma non il panorama; per un'azione conta l'esito raggiunto ma non, ad esempio, l'inquinamento prodotto.

Obiettivi e ricerca

Un **obiettivo** è un risultato verso cui sono diretti gli sforzi, definito da: 1. una **situazione** (es. "essere in un luogo"), 2. eventualmente una **prestazione** associata (es. "...entro un certo tempo").

L'insieme di tutti gli stati che soddisfano la condizione obiettivo è detto **insieme degli stati obiettivo** (o stati *goal/target*).

Un algoritmo di ricerca determina una soluzione che, partendo dallo stato iniziale, raggiunge uno stato obiettivo, utilizzando: 1. una descrizione del problema; 2. un metodo di ricerca nello spazio degli stati.

Una soluzione è un percorso nello spazio degli stati.

Definizione formale di problema di ricerca

Un problema di ricerca è definito formalmente come una **tupla di 4 elementi**:

Elemento	Significato
1. Stato iniziale	cattura la situazione da cui si parte per calcolare la soluzione
2. Funzione successore (o funzione azioni + modello di transizione)	dato uno stato e un'azione legale in esso, calcola lo stato risultante dall'esecuzione di quell'azione
3. Test obiettivo	determina se uno stato è quello goal: verifica una proprietà o l'appartenenza all'insieme degli stati target
4. Funzione di costo del cammino	assegna un costo numerico a un percorso possibile

L'insieme di tutti gli stati possibili (**spazio degli stati**) è spesso definito in modo implicito/generativo (tramite stato iniziale + funzione successore) piuttosto che enumerato esplicitamente.

Esempio guida: problema di navigazione

- **Stato iniziale:** `in(Alessandria)`.
- **Funzione successore:** applicando l'azione `go(Ovada)` allo stato `in(Alessandria)` si ottiene lo stato `in(Ovada)` (transizione possibile solo se Ovada è raggiungibile direttamente da Alessandria).
- **Test obiettivo:** se la destinazione è Genova, verifica se lo stato corrisponde a `in(Genova)`.
- **Funzione di costo del cammino:** ad esempio il numero di km percorsi, oppure una misura più complessa che combina km e pedaggi.

Esempio: il mondo dell'aspirapolvere

Due stanze (sinistra S, destra D), ciascuna sporca o pulita; un aspirapolvere che si trova in una delle due stanze e può muoversi o aspirare.

- Ogni stanza può assumere 4 configurazioni: `<pulita, vuota>`, `<pulita, con aspirapolvere>`, `<sporca, vuota>`, `<sporca, con aspirapolvere>`.
- **Azioni:** applicabili a tutti gli stati (muoviti a sinistra/destra, aspira).
- **Stato iniziale:** qualunque stato può esserlo (dipende da dove si trova l'aspirapolvere e dallo sporco presente).
- **Funzione successore:** restituisce lo stato prodotto applicando l'azione scelta allo stato corrente.
- **Test obiettivo:** entrambe le stanze pulite.
- **Costo del cammino:** ogni azione costa 1, quindi il costo è il numero di azioni eseguite.

Esempio: il gioco dell'8 (8-puzzle)

- **Stato:** posizione di ciascuna delle 8 tessere numerate e dello spazio vuoto in una griglia 3x3. Esistono **181.440 = $9!/2$** stati raggiungibili (metà delle 9! permutazioni, perché solo metà sono raggiungibili da una configurazione data — l'altra metà richiederebbe "sollevare" le tessere).
- **Stato iniziale:** qualunque configurazione può esserlo.
- **Funzione successore:** sposta nello spazio vuoto una delle tessere ad esso adiacenti.
- **Test obiettivo:** la disposizione dei numeri corrisponde alla configurazione obiettivo prefissata.
- **Costo del cammino:** ogni mossa costa 1; il costo del cammino è il numero di mosse.

Toy problem vs Real-world problem

- **Toy problem:** problema artificiale, con formulazione precisa e univoca, usato per illustrare o confrontare metodi di risoluzione (8-puzzle, 8 regine, mondo dell'aspirapolvere, attraversamento del fiume...).
- **Real-world problem:** problema concreto (configurazione VLSI, itinerari stradali, navigazione robotica...), spesso privo di una formulazione unica e standard.

? **Domanda d'esame:** Formulare il problema delle 8 regine (stati, stato iniziale, funzione successore, test obiettivo, costo). **Risposta ragionata:** occorre posizionare 8 regine su una scacchiera 8x8 in modo che nessuna ne attacchi un'altra (stessa riga, colonna o diagonale vietate). Una formulazione efficiente (*incrementale*) è: **stati** = configurazioni con 0-8 regine sulla scacchiera, una per colonna, nessuna in conflitto; **stato iniziale** = scacchiera vuota; **funzione successore** = aggiungi una regina in una qualsiasi casella libera della colonna successiva che non sia attaccata dalle regine già posizionate; **test obiettivo** = 8 regine sulla scacchiera senza conflitti; **costo del cammino** = 1 per ogni regina piazzata (irrilevante, interessa solo raggiungere lo stato goal). Questa formulazione riduce enormemente lo spazio di ricerca rispetto a generare tutte le disposizioni di 8 regine su 64 caselle e poi filtrare.

Spazio degli stati vs albero/grafico di ricerca

È fondamentale distinguere due livelli:

- **Spazio degli stati:** l'insieme (astratto, spesso implicito) di tutti gli stati raggiungibili e le transizioni fra essi, definito dal problema stesso (stato iniziale + funzione successore). Esiste indipendentemente dal fatto che venga esplorato.
- **Albero (o grafo) di ricerca:** struttura dati *costruita dall'algoritmo* durante l'esplorazione per trovare una soluzione. Ogni **nodo** dell'albero corrisponde a uno stato, ma un nodo contiene anche altra informazione (riferimento al nodo genitore, azione che lo ha generato, profondità, costo del cammino accumulato). I **nodi figli** sono generati applicando la funzione successore. L'albero diventa **grafo** quando lo stesso stato può essere raggiunto da più percorsi diversi (nodi diversi che corrispondono allo stesso stato, o — se si marcano gli stati visitati — un unico nodo con più genitori).

Formalmente, un grafo di ricerca è una coppia $G = (\{n_i\}, \{e_{ij}\})$: un insieme di nodi e un insieme di archi, dove l'arco e_{ij} rappresenta che n_j è successore di n_i , con un costo c_{ij} associato. L'esistenza di e_{ij} non implica quella di e_{ji} (i grafi sono orientati); se esiste e_{ji} in generale $c_{ji} \neq c_{ij}$. Attenzione: nella struttura dati, ogni nodo ha un puntatore al proprio **genitore** (arco figlio→genitore, opposto rispetto al verso di espansione).

Durante la ricerca ogni nodo si trova in uno di questi stati: - **frontiera:** nodi generati ma non ancora espansi (detti anche nodi APERTI); - **espanso/chiuso:** nodo di cui sono già stati generati tutti i successori; - non ancora generato.

Il criterio con cui si sceglie **quale nodo della frontiera espandere per primo** è ciò che distingue le diverse strategie di ricerca.

Criteri di valutazione delle strategie di ricerca

Per confrontare le strategie si usano quattro criteri:

1. **Completezza:** garanzia di trovare una soluzione, se esiste.
2. **Ottimalità:** garanzia di trovare la soluzione di costo minimo.
3. **Complessità temporale:** quanto tempo (= quanti nodi generati/espansi) serve per trovare una soluzione.
4. **Complessità spaziale:** quanta memoria (= quanti nodi mantenuti) serve.

Poiché gli algoritmi sono entità astratte (indipendenti dal calcolatore su cui girano), tempo e spazio si misurano in modo **parametrico**, non in secondi/byte: si conta il numero di nodi generati o visitati, usando la **notazione O-grande**. $T(n)$ è $O(f(n))$ se esiste n_0 e una costante $c > 0$ tali che per ogni $n > n_0$ si abbia $T(n) \leq c \cdot f(n)$.

I parametri standard usati per gli algoritmi di ricerca sono: - **b** = fattore di ramificazione (branching factor): numero massimo di figli di un nodo; - **d** = profondità della soluzione più superficiale (meno costosa in numero di passi); - **m** = profondità massima dello spazio degli stati (può essere infinita).

Gli algoritmi si dividono in: - **approcci blind (non informati)**: usano solo la struttura del problema (stato iniziale, successori, test obiettivo); - **approcci informati**: usano anche conoscenza aggiuntiva (euristiche) per guidare la ricerca — monoagente (questo modulo), multiagente/giochi, CSP (moduli successivi).

Parte 1 — Ricerca non informata (blind search)

Lista delle strategie trattate: ricerca in ampiezza, ricerca a costo uniforme, ricerca in profondità (con variante a profondità limitata), iterative deepening, ricerca bidirezionale.

1. Ricerca in ampiezza (Breadth-First Search, BFS)

Idea: si espande prima il nodo radice, poi tutti i suoi successori (livello 1), poi tutti i discendenti di livello 2, e così via — si esplora un livello di profondità alla volta.

Struttura dati: la frontiera è gestita come una **coda FIFO** (First In First Out): i nodi generati per primi vengono espansi per primi.

Nota sulla memoria: tutti i nodi della frontiera *e tutti i loro antenati* devono restare in memoria, per poter ricostruire il percorso soluzione quando si trova il nodo obiettivo (grazie ai puntatori al genitore).

Valutazione BFS

- **Completezza:** sì, se il fattore di ramificazione b è finito e la soluzione si trova a profondità finita d .
- **Ottimalità:** sì, **solo se il costo del cammino è funzione monotona non decrescente della profondità** (in particolare se tutte le azioni hanno lo stesso costo — in tal caso ottimalità = trovare la soluzione con meno passi).
- **Complessità temporale:** $O(b^{(d+1)})$. Deriva dal fatto che nel caso peggiore si generano tutti i b^i nodi di ogni livello i fino al livello d , più i nodi di livello $d+1$ necessari a verificare il test obiettivo sull'ultimo nodo del livello d (in totale $b^{(d+1)} - b$ nodi al livello $d+1$ nel caso peggiore).
- **Complessità spaziale:** $O(b^{(d+1)})$, perché occorre mantenere in memoria tutta la frontiera e tutti gli antenati.

La complessità spaziale è il vero tallone d'Achille della BFS: con $b=10$, generando 10.000 nodi/sec e un nodo che occupa 1000 byte, a profondità 12 servirebbero circa **10 petabyte** di memoria e 35 anni di tempo — la memoria esaurisce le risorse ben prima del tempo.

2. Ricerca a costo uniforme (Uniform-Cost Search)

Idea: quando i costi dei passi non sono tutti uguali, non basta contare i livelli: occorre espandere sempre il nodo il cui **cammino dalla radice ha costo minimo**.

Struttura dati: la frontiera è una **coda con priorità** ordinata per costo del cammino accumulato $g(n)$. Ad ogni iterazione si espande il nodo di costo minimo.

Punti di attenzione: - **costo** $\neq$ **numero di passi**: non conta quante azioni sono state eseguite, ma la somma dei loro costi; - quando si genera per la prima volta un nodo obiettivo **l'algoritmo non si ferma subito**: prima verifica che non vi siano altri cammini aperti di costo ancora inferiore da espandere (perché quello trovato potrebbe non essere il più economico). - quando tutti i costi sono uguali, la ricerca a costo uniforme **coincide** con la BFS.

Valutazione ricerca a costo uniforme

- **Completezza**: garantita solo se tutti i passi hanno **costo** > 0 (costo minimo delle operazioni positivo, altrimenti si rischiano loop infiniti a costo zero che non fanno mai terminare la ricerca — es. un'azione "noOp" a costo 0).
- **Ottimalità**: garantita, sempre a patto che i costi siano tutti > 0 . Il motivo è che i costi dei cammini aumentano monotonamente con l'aggiunta di passi e l'algoritmo espande sempre il cammino di costo minimo attualmente aperto: non può "saltare" per errore un percorso più economico non ancora scoperto.
- **Complessità** (temporale e spaziale): non dipende direttamente da b e d , ma dal **costo della soluzione ottima C^*** e dal costo minimo delle azioni ϵ : $O(b^{(1+Lc^*/\epsilon J)})$. Intuitivamente, equivale al branching factor elevato al numero di passi necessari a raggiungere il costo C^* se tutti i passi costassero ϵ . Quando i costi sono tutti uguali, ricade nel caso della BFS.

3. Ricerca in profondità (Depth-First Search, DFS) senza backtracking

Idea: si espande sempre uno dei nodi **più profondi** (più lontani dalla radice) della frontiera. L'espansione genera **tutti** i successori di un nodo. Quando si cerca di espandere un nodo privo di successori, il nodo viene scartato e si torna indietro (backtrack) per esplorare altre alternative rimaste in frontiera.

Struttura dati: la frontiera è gestita come una **coda LIFO** (Last In First Out), cioè uno **stack**.

Valutazione DFS

- **Completezza**: garantita solo se **tutti i cammini sono finiti** (altrimenti la ricerca può incamminarsi lungo un ramo infinito senza mai tornare indietro a esplorare la soluzione).
- **Ottimalità**: **non garantita in generale**, anche a parità di costo dei passi: la DFS può restituire una soluzione più lunga/costosa semplicemente perché l'ha incontrata per prima esplorando in profondità un ramo "sbagliato", mentre una soluzione più corta si trovava in un altro ramo della frontiera non ancora visitato.
- **Complessità temporale**: $O(b^m)$ nel caso peggiore (m = profondità massima dell'albero), perché nel caso peggiore si visitano tutti i nodi dell'albero.
- **Complessità spaziale**:
 - versione che genera **tutti** i successori di un nodo espanso: $O(b \cdot m)$ — occorre mantenere il cammino corrente più tutti i "fratelli" generati lungo il percorso ($b \cdot m + 1$ nodi nel caso peggiore);
 - variante **con backtracking classico**, in cui si genera un solo successore alla volta (si torna al genitore per generarne un altro solo se il ramo corrente fallisce): la complessità spaziale scende a $O(m)$.

Il vantaggio della DFS rispetto alla BFS è enormemente in termini di spazio: con lo stesso esempio ($b=10$, $d=12$) la DFS richiederebbe solo ~118 KB contro i 10 petabyte della BFS.

Variante: ricerca a profondità limitata

Per evitare che la DFS si perda in cammini infiniti (o semplicemente troppo lunghi) che non portano alla soluzione, si introduce un **limite artificiale l** : un nodo viene espanso solo se la sua profondità $p \leq l$; oltre quel limite viene trattato come se non avesse successori (si taglia il ramo).

- Tutti i cammini esplorati avranno lunghezza al più 1.
- **Problema 1:** si perde completezza se la profondità reale dell'obiettivo d supera l (l'obiettivo non verrà mai trovato).
- **Problema 2:** se invece $1 > d$, si perde efficienza esplorando inutilmente rami oltre la profondità della soluzione.

Il problema pratico è che **d non è quasi mai noto a priori**, quindi scegliere un buon valore di l è difficile — da qui l'idea dell'iterative deepening.

4. Iterative Deepening Depth-First Search (IDDFS)

Idea: esegue ripetutamente una ricerca a **profondità limitata**, aumentando progressivamente il limite l (0, 1, 2, 3, ...) fino a trovare la soluzione. Ad ogni iterazione l'albero di ricerca viene **ricostruito da zero**. Se una ricerca fallisce per raggiungimento del limite, si dice che è avvenuto un "taglio".

In pratica è come esplorare in sequenza tanti alberi diversi (con $l = 0$, poi $l = 1$, poi $l = 2$, ...), ciascuno tramite una normale DFS a profondità limitata.

Perché non è così costoso ripartire da zero?

Sembra controintuitivo che ricostruire l'albero ad ogni iterazione sia efficiente, ma la maggior parte dei nodi si trova **vicino alla frontiera** (ai livelli più profondi), quindi i nodi vicini alla radice vengono rigenerati più volte ma sono relativamente pochi rispetto a quelli generati una sola volta al livello più profondo.

Il numero totale di nodi generati dall'iterative deepening è:

$$N_{id} = d \cdot b + (d-1) \cdot b^2 + \dots + (d-i) \cdot b^{(i+1)} + \dots + 1 \cdot b^d = \sum_{i=1}^d (d-i) \cdot b^i$$

che risulta **minore** del numero di nodi generati dalla BFS nel caso peggiore:

$$N_{amp} = b + b^2 + \dots + b^i + \dots + b^d + (b^{(d+1)} - b)$$

perché la BFS, a differenza dell>IDDFS, genera anche (quasi tutti) i nodi del livello $d+1$.

Esempio numerico ($b=10$, $d=5$): $N_{id} \approx 123.456$ nodi contro $N_{amp} \approx 1.111.100$ nodi — la BFS genera quasi 9 volte più nodi, principalmente a causa del costoso livello $d+1$.

Valutazione IDDFS

Combina i pregi di BFS e DFS: - **Completezza:** sì, se b è finito (come la BFS). - **Ottimalità:** sì, se il costo è funzione non decrescente della profondità (come la BFS). - **Complessità spaziale:** $O(b \cdot d)$, modesta come la DFS. - **Complessità temporale:** $O(b^d)$, simile (nell'ordine di grandezza) alla BFS.

È la strategia **preferita quando lo spazio di ricerca è ampio e la profondità della soluzione non è nota a priori** — combina la garanzia di trovare la soluzione più superficiale (come BFS) con requisiti di memoria contenuti (come DFS).

5. Ricerca bidirezionale

Idea: eseguire simultaneamente **due ricerche**: una **forward** dallo stato iniziale, una **backward** dallo stato obiettivo. La ricerca termina quando le due frontiere si intersecano (un nodo compare in entrambe).

Per determinare la ricerca all'indietro occorre saper calcolare i **predecessori** di uno stato — operazione non sempre semplice o efficiente quanto calcolare i successori. Se gli operatori hanno forma IF antecedente THEN conseguente, la ricerca forward verifica quali antecedenti sono soddisfatti dallo stato corrente, mentre

la ricerca backward verifica quali conseguenti sono soddisfatti (usando la conoscenza degli antecedenti per risalire ai possibili predecessori).

Problema aggiuntivo: se lo stato obiettivo non è unico, da dove far partire la ricerca backward? Soluzione: introdurre uno **stato "di comodo"** raggiungibile in un passo da tutti gli stati obiettivo reali, e farla partire da lì.

Le due ricerche possono procedere: - **a fronte d'onda** (esplorando tutte le operazioni possibili, quando non c'è conoscenza aggiuntiva), oppure - **a cono** (esplorando solo una parte delle operazioni, se si dispone di conoscenza aggiuntiva/euristica) — in questo secondo caso è importante che i due coni si incontrino "a metà strada", altrimenti si rischia di raddoppiare il lavoro.

Valutazione ricerca bidirezionale

- **Complessità temporale e spaziale:** $O(b^{(d/2)})$ (più precisamente $2 \cdot O(b^{(d/2)})$), un netto vantaggio rispetto a $O(b^d)$: esempio con $b=2$, $d=6$, si passa da $b^d = 64$ a $b^{(d/2)} = 8$.
- **Completezza:** garantita se b è finito e si usa la BFS in entrambe le direzioni.
- **Ottimalità:** garantita se i costi dei passi sono identici e si usa la BFS in entrambe le direzioni.

Grafi di ricerca e nodi già esplorati

In molti problemi lo stesso stato è raggiungibile tramite percorsi diversi: questo causa **ripetizione di lavoro** (si riespande più volte la stessa porzione di spazio degli stati). Marcare i nodi già visitati e controllare, prima di espandere un nodo, se lo stato corrispondente è già stato visitato, rende la ricerca molto più efficiente (si passa da una visita ad albero a una visita a grafo).

Esempi classici di problemi di attraversamento

- **Capra (pecora), cavolo e lupo:** un uomo deve traghettare pecora, cavolo e lupo con una barca che ne trasporta uno alla volta, senza mai lasciare incustodite sulla stessa riva pecora+cavolo o lupo+pecora. Stati rappresentati con variabili $\langle U, C, P, L \rangle$ a valori a/b (le due rive); stato iniziale $\langle a, a, a, a \rangle$, stato goal $\langle b, b, b, b \rangle$.
- **Missionari e cannibali:** 3 missionari e 3 cannibali devono attraversare un fiume con una barca da 1-2 posti, senza che in nessuna riva i cannibali siano mai in maggioranza rispetto ai missionari. Rappresentazioni possibili: $\langle Ma, Ca, Mb, Cb, B \rangle$ oppure più compattamente $\langle Ma, Ba, B \rangle$.
- **Torre di Hanoi:** spostare una torre di dischi dal primo al terzo piolo (usando il secondo come appoggio), potendo sovrapporre solo dischi più piccoli su dischi più grandi.

Questi problemi sono tipici esercizi per esercitarsi nella formulazione formale (stato iniziale, azioni, test obiettivo) e nel disegno esplicito dello spazio degli stati/albero di ricerca.

Tabella riassuntiva — confronto strategie di ricerca non informata

Strategia	Struttura dati frontiera	Completezza	Ottimalità	Complessità temporale	Complessità spaziale
Ampiezza (BFS)	coda FIFO	Sì (se b finito)	Sì, se costo non decrescente con la profondità (es. costi uniformi)	$O(b^{(d+1)})$	$O(b^{(d+1)})$
Costo uniforme	coda a priorità (su $g(n)$)	Sì (se costi passi > 0)	Sì (se costi passi > 0)	$O(b^{(1+LC*/\epsilon J)})$	$O(b^{(1+LC*/\epsilon J)})$

Strategia	Struttura dati frontiera	Completezza	Ottimalità	Complessità temporale	Complessità spaziale
Profondità (DFS)	coda LIFO / stack	No, solo se cammini finiti	No	$O(b^m)$	$O(b \cdot m)$ [$O(m)$ con backtracking]
Profondità limitata	stack, con limite l	No, solo se $l \geq d$	No	$O(b^l)$	$O(b \cdot l)$
Iterative deepening (IDDFS)	stack ricostruito ad ogni iterazione	Sì (se b finito)	Sì, se costo non decrescente con la profondità	$O(b^d)$	$O(b \cdot d)$
Bidirezionale	due frontiere (FIFO se BFS in entrambi i sensi)	Sì (se b finito, BFS in entrambe le direzioni)	Sì (se costi uniformi, BFS in entrambe le direzioni)	$O(b^{(d/2)})$	$O(b^{(d/2)})$

(b = fattore di ramificazione; d = profondità della soluzione più superficiale; m = profondità massima dello spazio degli stati; l = limite di profondità; C^* = costo della soluzione ottima; ϵ = costo minimo di un'azione)

Parte 2 — Ricerca informata (heuristic search)

Perché serve conoscenza aggiuntiva

? **Domanda d'esame:** Le strategie di ricerca blind non sono efficienti. È possibile rendere la ricerca più efficiente utilizzando conoscenza sul problema? La conoscenza permette di focalizzare la ricerca verso le direzioni più promettenti? **Risposta ragionata:** sì. Le strategie non informate esplorano lo spazio degli stati "alla cieca", basandosi solo sulla struttura del problema (chi sono i successori, qual è il costo dei passi). Se disponiamo di conoscenza aggiuntiva sul problema — tipicamente una **stima** di quanto uno stato sia vicino all'obiettivo — possiamo usarla per **ordinare la frontiera** e dare priorità agli stati più promettenti, riducendo drasticamente il numero di nodi generati/espansi rispetto a BFS, DFS o costo uniforme. Questa stima si chiama **funzione euristica** $h(n)$. Esempio classico: nell'8-puzzle si può usare il numero di tessere fuori posto come euristica.

Funzioni di valutazione e best-first search

Una strategia di ricerca informata ordina la frontiera in base a una **funzione di valutazione** $f(n)$ applicata a ciascun nodo. A seconda di come si definisce $f(n)$ si ottengono strategie diverse; questa famiglia di strategie è detta **best-first search**, perché espande per primo il nodo ritenuto "più promettente" secondo f . Vi appartengono la ricerca greedy, A^* e RBFS.

Spesso $f(n)$ include una componente $h(n)$, detta **euristica**, che stima il costo minimo del cammino residuo dallo stato corrispondente a n fino a uno stato obiettivo.

1. Ricerca greedy (avida)

$f(n) = h(n)$: si sceglie sempre il nodo che l'euristica stima più vicino all'obiettivo, ignorando completamente il costo già speso per arrivarci.

Esempio: se gli stati sono località di cui si conoscono le coordinate, si può usare la distanza in linea d'aria verso l'obiettivo come stima della "vicinanza" reale (su strada).

Problemi della ricerca greedy: - L'euristica può ingannare: la distanza in linea d'aria non riflette necessariamente la distanza reale su strada. Esempio: se in linea d'aria $\text{dist}(B,G) < \text{dist}(C,G)$ ma su strada $\text{dist}(B,G) > \text{dist}(C,G)$, la greedy sceglie B illudendosi che sia il ramo migliore, quando in realtà C conduce prima all'obiettivo. - Rischio di **vicoli ciechi** e loop: la greedy eredita i difetti della DFS, perché segue "avidamente" la direzione che sembra più promettente senza tener conto del cammino già percorso, rischiando di doversi lunghi tratti all'indietro se imbocca una strada senza uscita (es. Balme→Noasca: 18 km in linea d'aria ma 91 km su strada reale, con un vicolo cieco che costringe a tornare indietro). - **Non è né completa né ottima**

in generale (proprio come la DFS, di cui condivide la logica di avanzamento “in profondità” verso il nodo apparentemente migliore).

2. A* (A-star)

Idea: combinare i punti di forza della ricerca a costo uniforme e della ricerca greedy: - il **costo uniforme** guarda al **passato**: espande per primi i nodi il cui cammino dalla radice costa meno; - la **greedy** guarda al **futuro**: espande per primi i nodi che promettono di raggiungere l’obiettivo al costo minore.

$f(n) = g(n) + h(n)$, dove: - $g(n)$ = costo minimo (fra tutti i cammini finora visti) per raggiungere il nodo n dallo stato iniziale (guarda indietro, è un valore esatto sui cammini già esplorati); - $h(n)$ = stima del costo minimo del proseguimento del cammino da n a un obiettivo (guarda avanti, è una stima); - $f(n)$ = stima del costo minimo totale di un cammino risolutivo che passa per n .

Confronto con i valori “veri” (ideali, calcolabili solo con conoscenza totale, non disponibili all’algoritmo): - $g^*(n)$ = costo minimo reale per raggiungere n da s ; - $h^*(n)$ = costo minimo reale del proseguimento da n a un obiettivo; - $f^*(n) = g^*(n) + h^*(n) = C^*$ per i nodi sul cammino ottimo.

Nozione di obiettivo preferito

A* lavora su grafi con un insieme T di nodi obiettivo. Per un nodo n , un obiettivo $t \in T$ è detto **preferito** se il costo del cammino ottimo $h(n, t)$ è minore o uguale al costo per raggiungere qualunque altro nodo di T da n . Si pone $h(n) = \min_{t \in T} h(n, t)$: cioè $h(n)$ rappresenta il costo per raggiungere l’obiettivo più economico da n .

Pseudocodice di A*

Sia s il nodo iniziale, T l’insieme dei nodi obiettivo (target)

segna s come APERTO e calcola $f(s)$

ripeti:

```

    seleziona il nodo APERTO  $n$  con  $f(n)$  minima
      (i pari merito si risolvono a vantaggio di eventuali  $n \in T$ )
    se  $n \in T$ :
      marca  $n$  CHIUSO e termina (soluzione trovata)
    altrimenti:
      marca  $n$  CHIUSO
      applica la funzione successore a  $n$ :
        calcola  $f$  per tutti i successori  $n'$ 
        marca APERTI i successori  $n'$  non ancora CHIUSI
        per i successori  $n'$  già CHIUSI ma raggiunti ora con un  $f$ 
          più basso di quello calcolato in precedenza (cammino
            migliore trovato), rimarcali come APERTI
```

Struttura dati: coda a priorità ordinata su $f(n) = g(n) + h(n)$ (generalizza la coda a priorità della ricerca a costo uniforme, ma con priorità che include anche la stima euristica).

Euristiche ammissibili

Una funzione euristica h è **ammissibile** se, per ogni nodo n , $h(n) \leq h^*(n)$, dove $h^*(n)$ è il costo minimo reale per raggiungere un nodo obiettivo da n . Intuitivamente: **un’euristica ammissibile non sovrastima mai** — è sempre ottimistica (o al più esatta). Esempio: la distanza in linea d’aria è ammissibile rispetto alla distanza reale su strada, perché la linea retta è sempre il cammino più corto possibile fra due punti.

Ottimalità di A*

Teorema: se (1) l'euristica h è ammissibile e (2) tutti i passi hanno costo maggiore di una costante positiva piccola a piacere ($\epsilon > 0$), allora **A* termina e trova sempre una soluzione ottima**: A* è completo e ottimo.

Dimostrazione (caso albero di ricerca)

In un albero ogni nodo ha un solo cammino dalla radice (nessuna molteplicità di percorsi). Supponiamo per assurdo che A* scelga un obiettivo subottimo $G2$ al posto di un nodo n che si trova su un cammino ottimo verso il vero obiettivo G , con $f^*(G) = g^*(G) + h^*(G) = C^*$ (costo della soluzione ottima).

1. **$G2$ è un nodo obiettivo**, quindi $h(G2) = 0$ per definizione di h (l'euristica al goal vale 0, essendo la distanza residua nulla).
2. Quindi $f(G2) = g(G2) + h(G2) = g(G2)$.
3. Poiché $G2$ è per ipotesi subottimo, $f(G2) = g(G2) > C^*$.
4. Sia n un nodo qualunque sul cammino ottimo: in un albero, $g^*(n) = g(n)$ (unico percorso possibile fino a n , quindi il costo osservato coincide col costo reale minimo).
5. Poiché h è ammissibile, $h(n) \leq h^*(n)$.
6. Quindi $f(n) = g(n) + h(n) \leq g^*(n) + h^*(n) = f^*(n) = C^*$.
7. Combinando (3) e (6): $f(n) \leq C^* < f(G2)$.
8. Ma A* sceglie sempre il nodo APERTO con f minima: **fra n e $G2$, A* sceglierà sempre n** , mai $G2$ — contraddizione con l'assunzione iniziale. Quindi A* non può fermarsi su un obiettivo subottimo se esiste ancora un nodo n aperto su un cammino ottimo. ■

Caso grafo di ricerca: serve la consistenza (monotonicità)

Nei grafi esiste **molteplicità di cammini**: il primo cammino trovato verso uno stato non è necessariamente quello di costo minimo (per questo, nell'esempio della Romania, A* non si ferma la prima volta che incontra Bucharest come nodo generato, ma solo quando lo estrae come nodo da espandere con f minima). Questo invalida la dimostrazione precedente, che sfruttava l'unicità del cammino nell'albero.

Per i grafi occorre l'ipotesi più forte di **euristica consistente (o monotona)**: per ogni nodo n e ogni suo successore n' generato tramite un'azione a ,

$$h(n) \leq c(n, a, n') + h(n')$$

Questa è una **disuguaglianza triangolare**: la stima diretta da n al goal non può superare il costo di un passo verso n' più la stima residua da n' .

Dimostrazione che con h consistente i valori $f(n)$ sono non decrescenti lungo un cammino: 1. Per definizione, $g(n') = g(n) + c(n, a, n')$. 2. Per definizione, $f(n') = g(n') + h(n')$. 3. Sostituendo: $f(n') = g(n) + c(n, a, n') + h(n')$. 4. Per la disuguaglianza triangolare, $c(n, a, n') + h(n') \geq h(n)$, quindi $f(n') \geq g(n) + h(n) = f(n)$. 5. Dunque $f(n') \geq f(n)$. ■

Poiché A* espande i nodi in ordine non decrescente di f , se uno stato già CHIUSO viene re-incontrato lungo un altro cammino, il nuovo valore f sarà maggiore o uguale a quello con cui è stato chiuso la prima volta; di conseguenza **il primo incontro con un nodo obiettivo durante l'espansione (non la sola generazione) corrisponde già alla soluzione ottima**.

Monotonicità vs ammissibilità

La monotonicità (consistenza) è una proprietà **più forte** dell'ammissibilità: si dimostra che ogni euristica monotona è anche ammissibile, ma non vale il viceversa in generale (anche se in pratica capita spesso). Graficamente: l'insieme delle euristiche monotone è un sottoinsieme di quello delle euristiche ammissibili.

La distanza in linea d'aria nel problema della Romania è sia ammissibile sia monotona: per ogni città $luogo$ e un suo successore diretto $luogo1$, vale $h(luogo) < c(luogo, vai, luogo1) + h(luogo1)$ — la distanza in linea

d'aria da luogo a Bucharest è sempre minore della distanza su strada fino a luogo1 sommata alla distanza in linea d'aria residua.

Ammissibile non vuol dire informativo

$h(n) = 0$ è banalmente sempre ammissibile (non sovrastima mai, essendo il minimo possibile), ma è totalmente **priva di informazione utile**: degrada A^* a una ricerca "cieca" basata solo su $g(n)$. In particolare, se in aggiunta tutti i passi hanno costo uniforme pari a 1, **A^* con $h=0$ diventa equivalente alla ricerca in ampiezza**.

Ottimalità in senso forte di A^*

A^* è **ottimamente efficiente** per qualunque euristica data: non esiste alcun altro algoritmo ottimo (che usi la stessa euristica ed espanda nodi in modo coerente con essa) in grado di garantire l'espansione di un numero di nodi inferiore a quello espanso da A . *Il difetto pratico è che il numero di nodi espansi cresce esponenzialmente con la profondità della soluzione ottima, e A mantiene in memoria tutti i nodi generati* (comportandosi, sotto questo aspetto, come una ricerca in ampiezza): questo lo rende spesso impraticabile per problemi di grandi dimensioni per limiti di memoria prima ancora che di tempo.

Ridurre i requisiti di memoria: IDA* e RBFS

- **IDA*** (Iterative Deepening A): *unisce l'idea di A con quella dell'iterative deepening*, usando come limite di taglio ad ogni iterazione non la profondità ma il valore di $f(n)$ (non approfondito nel dettaglio nelle slide del corso).
- **RBFS** (Recursive Best-First Search): funziona come una DFS ricorsiva, ma mantiene un **upper bound dinamico** che rappresenta la stima di costo della migliore alternativa attualmente nota, permettendo di abbandonare un ramo non appena esso smette di essere il più promettente, invece di proseguire indefinitamente lungo lo stesso cammino.

RBFS in dettaglio (Korf, 1993)

Ogni chiamata ricorsiva ha tre argomenti: un nodo N , un valore $F(N)$ e un upper bound B . - $f(n)$: la funzione di valutazione statica ($f(n) = g(n) + h(n)$ come in A^* , non cambia nel tempo). - $F(n)$: valore **dinamico** associato al nodo, che dipende dai discendenti: $F(N) = f(N)$ se N è esplorato per la prima volta, altrimenti $F(N) = \min(F(\text{figli di } N))$ — cioè F "eredita" la miglior stima trovata nel sottoalbero. - B : upper bound, calcolato in base ai valori F dei nodi fratelli — memorizza il costo stimato della **migliore alternativa** rispetto al ramo attualmente in esplorazione (in pratica il secondo migliore fra i fratelli).

Chiamata iniziale: $\text{RBFS}(\text{radice}_r, f(r), \infty)$.

```
RBFS(N, F(N), B):
    se f(N) > B: ritorna f(N)                                // upper bound superato, cambia percorso
    se N è un goal: termina con successo
    se N non ha figli: ritorna infinito                        // vicolo cieco, cambia percorso
    per ogni figlio Ni di N:                                  // inizializza F dei successori
        se f(N) < F(N): F[i] := max(F(N), f(Ni))
        altrimenti: F[i] := f(Ni)
    ordina i figli Ni per F[i] crescente                      // F[1] = figlio più promettente
    se un solo figlio: F[2] := infinito
    finché (F[1] ≤ B e F[1] < infinito):                      // scendo solo se rispetto l'upper bound
        F[1] := RBFS(N1, F[1], min(B, F[2]))
        reinserisci N1 e F[1] in ordine
    ritorna F[1]
```


Intuizione: RBFS lavora come A* finché il ramo che sta costruendo rimane il migliore possibile secondo l'upper bound corrente. Non appena la stima di quel ramo peggiora oltre B, la ricerca lungo quel cammino viene **sospesa e "dimenticata"** (i nodi vengono cancellati dalla memoria), mantenendo solo, sul nodo radice del ramo abbandonato, il valore F aggiornato che riassume quanto costava proseguire di là. Se in seguito quel ramo torna ad essere il più conveniente, viene ri-esplorato da capo (con conseguente **ripetizione di lavoro**, difetto tipico di RBFS).

Valutazione RBFS

- **Ottimalità:** sì, se l'euristica è ammissibile.
- **Complessità spaziale: lineare, $O(b \cdot d)$** — enorme vantaggio su A*, perché mantiene in memoria solo i nodi del cammino corrente e i loro fratelli diretti (esattamente come la DFS/IDDFS), non tutti i nodi generati.
- **Complessità temporale:** difficile da definire in generale, perché dipende fortemente dall'accuratezza dell'euristica usata.
- **Difetti:** non tiene traccia dei cammini ripetuti (non riconosce se uno stato è già stato visitato per un'altra via) e, essendo vincolata a un consumo di memoria $O(b \cdot d)$, **non riesce a sfruttare memoria aggiuntiva disponibile** per migliorare l'efficienza, a differenza di A* che potrebbe beneficiarne mantenendo più informazione.

Funzioni euristiche: il caso di studio dell'8-puzzle

L'8-puzzle è uno dei primi problemi su cui si è sperimentata la ricerca informata. Generando casualmente lo stato iniziale, in media servono **22 mosse** per risolverlo, con un **branching factor medio pari a 3** (dipende dalla posizione della casella vuota: 4 mosse possibili se è al centro, 2 se è in un angolo, 3 se è sul bordo).

- L'**albero esaustivo** di ricerca conterrebbe circa 3^{22} nodi, oltre 30 miliardi di nodi.
- Il **grafo esaustivo** (evitando i duplicati, cioè non riesplorando stati già visitati) contiene "solo" circa **180.000 stati** (coerente con i $181.440 = 9!/2$ calcolati nella definizione formale del problema).
- Passando al **15-puzzle** (griglia 4×4), lo spazio esplode fino a circa **10^{13} stati**: la crescita è drammatica nonostante il problema sembri solo "un po' più grande".

Due euristiche ammissibili classiche

- **h_1 = numero di tessere fuori posto.** È ammissibile perché ogni tessera fuori posto dovrà essere spostata almeno una volta per arrivare alla configurazione obiettivo, quindi h_1 non può mai sovrastimare il numero di mosse residue.
- **h_2 = distanza di Manhattan (block distance).** Somma, su tutte le tessere, della distanza fra la posizione corrente e quella desiderata, contata come numero di celle attraversate in orizzontale più numero di celle attraversate in verticale. È ammissibile perché ogni mossa può avvicinare una tessera al proprio posto di **al più una posizione** per volta.

Esempio di calcolo: per un dato stato s , se tutte le 8 tessere sono fuori posto, $h_1(s) = 8$. Sommando le distanze di Manhattan individuali di ciascuna tessera (es. $3+1+2+\dots$) si può ottenere ad esempio $h_2(s) = 18$.

Confronto sperimentale fra euristiche

Per stabilire quale euristica sia "migliore" non basta guardare un singolo caso: istanze diverse dello stesso problema (stati iniziali/goal diversi), anche a parità di profondità della soluzione d , generano numeri di nodi espansi differenti. Il metodo corretto è **sperimentale**: 1. generare un numero significativo di istanze del problema; 2. applicare lo stesso algoritmo di ricerca (tipicamente A*) a ogni istanza, una volta per ciascuna euristica da confrontare; 3. raccogliere i dati (nodi generati, profondità della soluzione, ecc.); 4. calcolare le medie sui casi omogenei (es. stessa profondità di soluzione); 5. confrontare le prestazioni medie.

Effective branching factor (fattore di ramificazione effettivo) b^*

È la misura standard per quantificare la “qualità” di un’euristica a posteriori. Dato un problema risolto con A^* , siano: - N = numero di nodi generati a partire dal nodo iniziale; - d = profondità della soluzione trovata.

b^* è definito come il branching factor che avrebbe un **albero uniforme di profondità d** che contenga esattamente $N+1$ nodi:

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$

da cui si ricava (approssimativamente, per b^* grande) $N \approx (b^*)^{d+1}$, cioè $b^* \approx \sqrt[d+1]{N}$.

Interpretazione: quanto più b^* è vicino a 1, tanto migliore (più informativa) è l’euristica, perché significa che A^* con quell’euristica si comporta quasi come se seguisse un unico cammino diretto verso l’obiettivo, con pochissima esplorazione “sprecata” su rami alternativi. Poiché per una data euristica i valori di b^* misurati su istanze diverse tendono ad essere **abbastanza consistenti fra loro**, bastano poche misure su un piccolo campione di problemi per stimare affidabilmente la qualità di un’euristica.

Esperimento citato: 1200 problemi del 15-puzzle generati casualmente, con profondità di soluzione fra 2 e 24, risolti sia con iterative deepening sia con A^* usando prima h_1 poi h_2 ; per ciascuna profondità sono stati calcolati nodi generati e b^* medi. Il risultato tipico (coerente con la teoria sotto) è che **h_2 (Manhattan) produce sistematicamente meno nodi e un b^* più basso di h_1** (tessere fuori posto), risultando quindi euristica migliore.

Valutazione teorica: euristiche dominanti

Oltre alla valutazione sperimentale, esiste un criterio **a priori**: siano h_1 e h_2 due euristiche **ammissibili** tali che per ogni nodo n , $h_2(n) \geq h_1(n)$ (h_2 “approssima meglio” h^* di quanto non faccia h_1). Si dice che **h_2 domina h_1** (o che h_2 è più informata di h_1).

Perché la dominanza implica efficienza: si dimostra (teorema, non dimostrato nelle slide) che A^* espande tutti e soli i nodi con $f(n) < C^*$ (C^* = costo della soluzione ottima, costante che non dipende dall’euristica usata ma solo dal costo reale delle azioni). Poiché $f(n) = g(n) + h(n)$, ciò equivale a dire che vengono espansi tutti i nodi con $h(n) < C^* - g(n)$. Se $h_2(n) \geq h_1(n)$ per ogni n , allora l’insieme dei nodi che soddisfano la condizione con h_1 è un **soprainsieme** di quello con h_2 : usando h_1 , A^* espanderà sicuramente **almeno tutti** i nodi che espande usando h_2 (e in genere di più). Di qui: **un’euristica più informata (dominante) porta A^* a espandere un numero di nodi minore o uguale**, quindi è preferibile.

Nel caso dell’8/15-puzzle, sia h_1 sia h_2 sono ammissibili e per costruzione $h_2(n) \geq h_1(n)$ per ogni n (la distanza di Manhattan somma “quante celle” deve percorrere ciascuna tessera fuori posto, quindi è sempre almeno pari al numero di tessere fuori posto): questo conferma teoricamente perché h_2 è preferibile a h_1 .

Costruzione sistematica di euristiche ammissibili: problemi rilassati

Un **problema rilassato** si ottiene rimuovendo (parte dei) vincoli del problema originale. Il grafo degli stati del problema rilassato è un **supergrafo** di quello originario (meno vincoli $\Rightarrow$ più transizioni permesse $\Rightarrow$ più archi). Poiché il supergrafo contiene comunque tutte le soluzioni ottime del problema originale (oltre ad altre in più), **il costo della soluzione ottima nel problema rilassato è un’euristica ammissibile per il problema originale** — non può mai costare di più risolvere una versione del problema con meno vincoli.

Esempio — 8-puzzle: il vincolo originale è “una tessera può spostarsi da A a B se (1) A e B sono adiacenti e (2) B è vuota”. Tre possibili rilassamenti (rimozione di uno o più vincoli): 1. **Rimuovi solo il vincolo (2)**: una tessera può muoversi da A a B se A e B sono adiacenti (anche se B è occupata, “scambiando” le tessere) — la soluzione ottima di questo problema rilassato dà origine a un’euristica ammissibile, vicina a **h_2 (distanza di Manhattan)**. 2. **Rimuovi solo il vincolo (1)**: una tessera può muoversi ovunque, purché la destinazione B sia vuota (non serve adiacenza). 3. **Rimuovi entrambi i vincoli**: una tessera può spostarsi ovunque, sempre — questo dà origine a **h_1 (tessere fuori posto)**, perché nel problema completamente rilassato basta un solo movimento per ogni tessera fuori posto.

Il programma **Absolver II** (Prieditis, 1993) è un esempio storico di sistema in grado di generare automaticamente euristiche ammissibili per astrazione (rilassamento sistematico dei vincoli): ha scoperto la prima euristica ammissibile nota per il Cubo di Rubik e un'euristica per l'8-puzzle migliore di quelle proposte in precedenza. La ricerca su generazione automatica di euristiche continua tuttora, soprattutto nell'ambito del *planning* (costruzione automatica di piani).

Combinare euristiche non comparabili

Se si dispone di più euristiche ammissibili $h_1, \dots, h_k$ nessuna delle quali domina le altre (per alcuni nodi è migliore l'una, per altri nodi è migliore un'altra), si può costruire un'euristica composta:

$$h(n) = \max\{h_1(n), h_2(n), \dots, h_k(n)\}$$

Questa euristica composta è **ammissibile** (perché lo sono tutte le componenti: prendere il massimo fra valori tutti $\leq h^*$ dà ancora un valore $\leq h$) ed è **dominante per costruzione** su ciascuna delle euristiche che la compongono, perché per ogni nodo restituisce sempre la stima più accurata (più alta, quindi più vicina a h) fra quelle disponibili.

Alternativa: apprendere le euristiche induttivamente

Un approccio alternativo alla progettazione manuale è usare tecniche di **apprendimento automatico**: si arricchisce ogni stato con un insieme di *feature* (es. numero di celle fuori posto, numero di vicini errati...), si generano casualmente molti problemi, li si risolve raccogliendo i dati (stato, feature, costo reale per raggiungere la soluzione), e si applica un metodo di apprendimento induttivo per estrarre automaticamente una funzione euristica dai dati raccolti.

Riepilogo e punti chiave

- Un **problema di ricerca** è formalizzato da 4 elementi: **stato iniziale**, **funzione successore (azioni + modello di transizione)**, **test obiettivo**, **funzione di costo del cammino**. Lo **spazio degli stati** (l'insieme astratto di stati/transizioni definito dal problema) va tenuto concettualmente distinto dall'**albero/grafico di ricerca** (la struttura dati che l'algoritmo costruisce esplorando quello spazio, dove ogni nodo aggiunge informazione di struttura — genitore, costo accumulato — allo stato che rappresenta).
- Le strategie **non informate** usano solo la struttura del problema e si distinguono essenzialmente per **come gestiscono la frontiera**: FIFO (ampiezza), coda a priorità su $g(n)$ (costo uniforme), LIFO/stack (profondità), stack con limite (profondità limitata), stack ricostruito iterativamente (iterative deepening), doppia frontiera (bidirezionale).
- Nessuna strategia non informata è "sempre migliore delle altre": BFS e costo uniforme sono complete/ottime ma costano tantissima memoria ($O(b^{d+1})$); DFS è economica in spazio ma non completa né ottima in generale; **iterative deepening** è **generalmente la scelta di compromesso migliore** quando d non è nota, unendo completezza/ottimalità di BFS a un costo spaziale $O(b \cdot d)$ come la DFS.
- Le strategie **informate** sfruttano una funzione euristica $h(n)$, stima del costo residuo per raggiungere l'obiettivo. La **ricerca greedy** ($f=h$) è veloce ma non completa né ottima, perché ignora il costo già speso (eredita i difetti della DFS). **A*** ($f = g+h$) combina il "guardare indietro" del costo uniforme con il "guardare avanti" della greedy.
- **A* con euristica ammissibile ($h \leq h^*$) è ottimo su alberi di ricerca**; sui **grafi** serve la proprietà più forte di **consistenza/monotonicità** ($h(n) \leq c(n, a, n') + h(n')$), che garantisce f non decrescente lungo ogni cammino e quindi che il primo nodo obiettivo espanso sia già quello ottimo. Ogni euristica monotona è ammissibile, ma non vale il viceversa in generale.
- **A* è ottimamente efficiente**: nessun altro algoritmo ottimo espande meno nodi a parità di euristica; il prezzo è la memoria (mantiene tutti i nodi generati, come una BFS). **IDA*** e **RBFS** riducono la complessità spaziale a scapito, in genere, di lavoro ripetuto: RBFS usa un upper bound dinamico per abbandonare

rami non più promettenti, raggiungendo complessità spaziale lineare $O(b \cdot d)$ ma senza sfruttare memoria extra disponibile.

- La qualità di un'euristica si misura con l'**effective branching factor b^*** ($N+1 = 1+b^*+\dots+(b^*)^d$): più b^* è vicino a 1, più l'euristica è informativa. Un criterio teorico complementare è la **dominanza**: se $h_2(n) \geq h_1(n)$ per ogni n (entrambe ammissibili), h_2 domina h_1 e garantisce che A^* espanda un sottoinsieme dei nodi espansi con h_1 .
- Le euristiche ammissibili si costruiscono sistematicamente tramite **problemi rilassati** (rimozione di vincoli): il costo della soluzione ottima nel problema rilassato è sempre un'euristica ammissibile per il problema originale, perché il grafo rilassato è un supergrafo che contiene anche le soluzioni ottime originali. Nell'8-puzzle, h_1 (tessere fuori posto) e h_2 (distanza di Manhattan) nascono da due rilassamenti diversi dei vincoli di adiacenza/cella-vuota, e h_2 domina h_1 .
- Quando più euristiche ammissibili non sono confrontabili, $h(n) = \max(h_1(n), \dots, h_k(n))$ è sempre ammissibile ed è dominante su tutte le componenti.

MODULO 3 — Ricerca con avversario (giochi)

Introduzione: perché studiare i giochi in AI

Marvin Minsky (1968) osservava che *“i giochi non vengono scelti perché sono chiari e semplici, ma perché ci danno la massima complessità con le minime strutture iniziali”*. I giochi sono quindi un banco di prova ideale per l'AI: regole semplici e stato ben definito, ma spazio di ricerca enorme e presenza di un **avversario** che rende il problema strutturalmente diverso dalla ricerca “in solitaria” vista nei moduli precedenti.

Il tema dei giochi attraversa più discipline (pungolo scientifico): - **Matematica**: teoria dei grafi, complessità computazionale - **Computer Science**: AI, database, calcolo parallelo - **Economia**: teoria dei giochi, economia cognitiva/sperimentale - **Psicologia**: fiducia, percezione del rischio

Ambienti competitivi

Un **ambiente multi-agente** è competitivo quando gli obiettivi degli agenti sono **conflittuali**: il conseguimento dell'obiettivo di un agente impedisce (almeno in parte) il conseguimento di quello degli altri. In questo contesto, i problemi di ricerca con avversario sono detti **giochi**. Le azioni degli altri agenti non sono più cooperative come nella pianificazione classica, ma vanno anticipate come ostili.

Tassonomia dei giochi

I giochi si classificano lungo due assi indipendenti:

Condizioni di scelta (informazione) - **Informazione perfetta/completa**: lo stato del gioco è totalmente esplicito e visibile a tutti i giocatori (es. scacchi, dama, Otello, Forza4, tris) - **Informazione imperfetta**: lo stato è solo parzialmente noto ad ogni giocatore (es. Mastermind, Scarabeo, Bridge, Poker e in genere i giochi di carte, dove le carte avversarie sono nascoste)

Effetti della scelta (determinismo) - **Deterministici**: lo stato successivo è determinato unicamente dalle azioni degli agenti - **Stocastici**: lo stato successivo dipende anche da fattori esterni casuali (es. lancio di dadi in Backgammon e Monopoli, pescate a caso in Risiko)

	Informazione perfetta	Informazione imperfetta
Deterministici	Scacchi, Go, Dama, Otello, Forza4, Tris	Mastermind, Scarabeo, Bridge, Poker
Stocastici	Backgammon, Monopoli	Risiko

Il Modulo 3 si concentra sui **giochi ad informazione completa (perfetta)**, tipicamente deterministici e a due giocatori, a **somma zero**.

Gioco a somma zero

Un gioco a somma zero è un contesto di interazione multi-agente in cui la perdita (o il guadagno) di un agente è esattamente compensata dal guadagno (perdita) degli altri agenti.

Esempio intuitivo: la torta da dividere. Se le fette sono irregolari, la parte in più che riceve chi ha la fetta più grande è esattamente compensata dalla parte in meno ricevuta dagli altri. Nei giochi a due giocatori a somma zero, se l'utilità di uno stato terminale per il giocatore A è u , l'utilità dello stesso stato per B è $-u$: è sufficiente memorizzare un solo valore per conoscere automaticamente anche l'altro.

Perché il tris è un caso interessante da studiare

Osservando lo sviluppo di una partita a tris si notano fenomeni chiave: - Ogni giocatore, quando è il suo turno, sceglie tra più mosse possibili (branching) - **L'ambiente è multi-agente**: l'evoluzione dello stato è solo *parzialmente controllabile* dal singolo giocatore, perché l'altro giocatore muove alternandosi - Una mossa che apre a una vittoria "a tre passi" può essere più rischiosa di una vittoria immediata, perché nel frattempo l'avversario ha la possibilità di reagire e ribaltare la situazione - In una posizione di svantaggio, conviene comunque "prolungare il più possibile" la partita piuttosto che perdere subito, perché ciò lascia all'avversario più occasioni di commettere un errore

Questi esempi illustrano intuitivamente l'idea centrale della ricerca competitiva: bisogna ragionare **ipoteticamente sulle mosse dell'avversario**, non solo sulle proprie.

Differenze rispetto alla ricerca (in)formata "classica"

Ricerca classica	Ricerca con avversario
Obiettivo: trovare un cammino a costo minimo	Obiettivo: determinare una strategia (funzione che associa una mossa a ogni stato raggiungibile), non una singola sequenza fissa
Si usa $g(x)$, il costo del cammino	$g(x)$ non si usa: guadagno/perdita non dipendono dal costo delle azioni (assunto uniforme)
I nodi terminali hanno un costo/valore	I nodi terminali sono categorizzati come vittoria, sconfitta, parità
Il nodo successore è sempre scelto dall'agente	Anche l'avversario muove : la scelta del nodo successivo non è sempre sotto il controllo dell'agente

Perché serve una *strategia* e non un semplice piano: poiché non si sa in anticipo quale mossa farà l'avversario, occorre precalcolare la risposta migliore per **ogni possibile mossa avversaria**, non solo per un'unica sequenza ipotizzata.

Caratteristiche formali del gioco (a due giocatori)

- Due giocatori, convenzionalmente chiamati **MAX** e **MIN**; MAX muove per primo
- Ciascun giocatore non conosce in anticipo la mossa che farà l'altro, ma **conosce l'insieme delle mosse possibili** e può calcolare gli stati successivi che ne deriverebbero
- **Osservabilità**:
 - *totale* nei giochi a turni: ogni giocatore conosce l'esito delle mosse precedenti prima di scegliere la propria
 - *parziale* nei giochi ad azione simultanea: i giocatori non conoscono le mosse eseguite simultaneamente dall'avversario

- A partire dallo stato iniziale si costruisce un **albero di gioco**, applicando ricorsivamente le azioni disponibili e calcolando gli stati successivi
- Alcuni stati sono **terminali**: quando se ne raggiunge uno, la partita finisce
- I giocatori valutano gli stati tramite una funzione di **utilità**, che tiene necessariamente conto anche del punto di vista dell'avversario
- **Ipotesi di pessimismo**: si assume che l'avversario sia "infallibile" e giochi sempre la mossa che gli porta il massimo guadagno possibile (il peggior danno per noi) — è un'ipotesi conservativa che garantisce una strategia sicura anche contro l'avversario più forte possibile
- I giochi a turno più semplici sono descritti come **2-ply**: un *ply* è un mezzo turno, cioè la mossa di un solo giocatore (un turno completo = 2 ply, uno per MAX e uno per MIN)

Albero di gioco: definizione

L'**albero di gioco** è la struttura che rappresenta tutte le evoluzioni possibili della partita a partire dallo stato iniziale: - la **radice** è lo stato iniziale (turno di MAX) - ogni **nodo interno** rappresenta uno stato di gioco, etichettato come nodo MAX o nodo MIN a seconda di chi deve muovere - gli **archi** rappresentano le mosse legali - le **foglie** sono gli stati terminali, cui è associata un'**utilità** definita dalle regole del gioco (tipicamente: vittoria/sconfitta/pareggio per il gioco del tris, un valore numerico continuo per giochi con punteggio)

Algoritmo Minimax

Idea e strategia ottima

La **strategia ottima** per un agente è la sequenza di mosse (in realtà, la funzione che mappa ogni stato raggiungibile nella mossa migliore) che, assumendo un avversario perfetto, massimizza l'utilità garantita. Poiché l'agente non sa come muoverà l'altro, può solo "*immedesimarsi*" nell'avversario e ragionare ipoteticamente: costruisce mentalmente l'intero albero di gioco, assumendo che: - nei nodi in cui muove **MAX**, verrà scelta la mossa che **massimizza** il valore per MAX - nei nodi in cui muove **MIN** (l'avversario, infallibile), verrà scelta la mossa che **minimizza** il valore per MAX (cioè quella più svantaggiosa possibile per MAX)

Definizione ricorsiva: valore-minimax

$$\text{Minimax}(n) = \begin{cases} \text{Utilità}(n) & \text{se } n \text{ è terminale} \\ \max_{\{s \in \text{Succ}(n)\}} \text{Minimax}(s) & \text{se } n \text{ è un nodo MAX} \\ \min_{\{s \in \text{Succ}(n)\}} \text{Minimax}(s) & \text{se } n \text{ è un nodo MIN} \end{cases}$$

Il valore minimax è quindi calcolato **ricorsivamente per ogni nodo** dell'albero, dal basso (foglie) verso l'alto (radice), e permette all'agente di scegliere la mossa da eseguire alla radice (si sceglie il figlio con il valore minimax più alto, se si è in un nodo MAX).

Da qui il nome "minimax": **ogni giocatore, minimizzando il guadagno massimo dell'altro, minimizza automaticamente la propria perdita** — grazie alla proprietà di somma zero (utilità di uno = meno utilità dell'altro), tenere traccia di un solo valore per nodo è sufficiente.

Pseudocodice

```
funzione MINIMAX-DECISION(stato) restituisce una mossa
    return arg max_{a ∈ Azioni(stato)} MIN-VALUE(Risultato(stato, a))
```

```
funzione MAX-VALUE(stato) restituisce un valore di utilità
    if TEST-TERMINALE(stato) then return Utilità(stato)
    v ← -∞
    for each a in Azioni(stato) do
        v ← MAX(v, MIN-VALUE(Risultato(stato, a)))
```

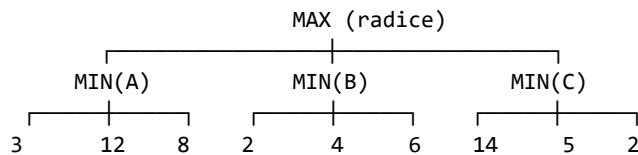
```
return v
```

```
funzione MIN-VALUE(stato) restituisce un valore di utilità
  if TEST-TERMINALE(stato) then return Utilità(stato)
  v ← +∞
  for each a in Azioni(stato) do
    v ← MIN(v, MAX-VALUE(Risultato(stato, a)))
  return v
```

L'algoritmo effettua una **visita ricorsiva in profondità** (DFS) dell'intero albero, con chiamate alternate tra MAX-VALUE e MIN-VALUE. Ogni nodo assume il proprio valore solo alla **chiusura** della relativa chiamata ricorsiva (cioè dopo che tutti i suoi figli sono stati valutati): i nodi terminali ricevono come valore la loro utilità diretta; i nodi interni ricevono il max o il min dei valori dei figli, a seconda del tipo.

Esempio passo-passo

Consideriamo un piccolo albero di gioco a 4 livelli (2 turni completi, 4 ply: MAX-MIN-MAX-MIN), con branching factor 3 e 9 foglie, valori di utilità arbitrari (schema analogo a quello usato nelle slide):



Valutazione bottom-up: 1. **Foglie:** valore = utilità diretta (3, 12, 8, 2, 4, 6, 14, 5, 2) 2. **Nodo MIN(A)**, figlio della radice: MIN sceglie la mossa peggiore per MAX tra {3, 12, 8} → $\min(3, 12, 8) = 3$ 3. **Nodo MIN(B)**: $\min(2, 4, 6) = 2$ 4. **Nodo MIN(C)**: $\min(14, 5, 2) = 2$ 5. **Radice (MAX)**: MAX sceglie la mossa che massimizza il proprio guadagno tra i valori calcolati per i figli, {3, 2, 2} → $\max(3, 2, 2) = 3$ → **MAX sceglierà il ramo verso MIN(A)**

Il valore minimax della radice è **3**: è il massimo risultato che MAX può *garantirsi* contro un avversario che gioca in modo ottimale (non è detto sia il massimo assoluto raggiungibile nell'albero — qui 14 — perché per raggiungerlo MAX dovrebbe "sperare" in un errore di MIN, cosa che l'ipotesi pessimistica esclude).

Questo è esattamente lo schema mostrato nelle slide (radice MAX → 3 nodi MIN → 9 foglie): a ogni nodo MIN si applica $v \leftarrow \min(\dots)$ scegliendo la mossa meno vantaggiosa per MAX, mentre alla radice MAX si applica $v \leftarrow \max(\dots)$ scegliendo la mossa più vantaggiosa per sé.

Complessità

- **Temporale:** $O(b^m)$, dove b = fattore di ramificazione (branching factor) e m = profondità massima dell'albero — è una visita esaustiva in profondità dell'intero albero
- **Spaziale:** $O(bm)$ se i successori vengono generati uno alla volta (non tutti contemporaneamente); $O(b \cdot m)$ è comunque **lineare** nella profondità (tipico vantaggio della DFS), a differenza della complessità temporale che resta **esponenziale**

Proprietà

- **Completo:** sì, su alberi/grafi finiti
- **Ottimale:** sì, ma solo a condizione che **sia MAX sia MIN giochino in modo ottimale** (se l'avversario reale non gioca ottimamente, la strategia minimax resta comunque *sicura*, garantendo almeno il valore calcolato)

Estensione a più di due giocatori

Minimax si estende a giochi con più di due giocatori sostituendo il valore scalare di ogni nodo con un **vettore di utilità** $\langle u_1, \dots, u_k \rangle$, dove u_i è l'utilità del nodo per il giocatore i . Un fenomeno tipico che emerge in questo scenario è la possibile **formazione di alleanze** tra sottoinsiemi di giocatori.

? Domanda d'esame: perché in un gioco a due giocatori a somma zero basta un solo valore per nodo, mentre con più giocatori serve un vettore? Nella somma zero a due giocatori l'utilità di un giocatore è sempre l'opposto di quella dell'altro ($u_2 = -u_1$), quindi un solo numero descrive completamente la situazione di entrambi. Con tre o più giocatori questa relazione di opposizione non vale più (il guadagno di uno non implica necessariamente e proporzionalmente la perdita degli altri, soprattutto se si formano alleanze), quindi occorre tracciare separatamente l'utilità di ciascun giocatore in ogni nodo.

Teoria delle decisioni: maximax, maximin, minimax regret

Prima di tornare ai giochi con avversario, le slide introducono tre approcci generali di **teoria delle decisioni** per scegliere tra alternative quando l'esito dipende da fattori **non controllabili** (nei giochi, il ruolo di "fattore non controllabile" sarà giocato proprio dall'avversario).

Scenario di esempio: tabella dei payoff

Si deve scegliere un investimento tra tre alternative, sapendo che il mercato può salire, restare stabile o scendere (andamento non controllabile, dipende da dinamiche esterne):

alternative	sale	stabile	scende
fondo1	40	45	5
fondo2	70	30	-20
azioni	55	40	-10

Le **scelte** (righe) sono sotto il controllo del decisore, se ne può fare **una sola**; i **payoff** (che possono rappresentare guadagni, perdite, o altre misure come tempo o risorse, a seconda del problema specifico) dipendono anche dall'andamento del mercato, non controllabile.

Approccio maximax (ottimistico)

Si guarda, per ciascuna scelta, il **payoff più alto ottenibile**, e si sceglie l'alternativa che promette il massimo assoluto: il "massimo dei massimi".

Nell'esempio: fondo1→45, fondo2→**70**, azioni→55 ⇒ si sceglie **fondo2**.

È un approccio **ottimistico**: non offre alcuna garanzia che le dinamiche esterne (il mercato) evolvano effettivamente nella direzione più favorevole per la scelta fatta.

Approccio maximin (pessimistico/conservativo)

Si guarda, per ciascuna scelta, il **payoff peggiore possibile** (la perdita maggiore), e si sceglie l'alternativa che **minimizza la perdita massima**: il "massimo dei minimi".

Nell'esempio: fondo1→5, fondo2→-20, azioni→-10 ⇒ si sceglie **fondo1** (unica opzione che non va mai in perdita).

È l'approccio **conservativo/pessimistico**: si presuppone che accada sempre lo scenario peggiore per la scelta fatta — è esattamente la stessa logica prudente su cui si basa **minimax** nei giochi con avversario (si veda oltre).

Approccio minimax regret

Introduce il concetto di **rimpianto (regret)**: quanto si "perde" rispetto alla scelta che, col senno di poi, sarebbe stata ottimale in quello scenario.

Regret = Best payoff (nello scenario) - Real payoff (della scelta fatta)

Passo 1 — calcolo dei regret per ciascuno scenario (colonna): - Scenario "sale": best payoff = 70 (fondo2). Regret: fondo1 = 70-40 = 30; fondo2 = 70-70 = 0; azioni = 70-55 = 15 - Analogamente si calcolano i regret per "stabile" e "scende"

Passo 2 — **regret table** risultante:

alternative	sale	stabile	scende
fondo1	30	0	0
fondo2	0	15	25
azioni	15	5	15

Passo 3 — per ciascuna alternativa si prende il **regret massimo** (il caso peggiore di rimpianto): fondo1→30, fondo2→25, azioni→15. Si sceglie l'alternativa con il **minimo tra i regret massimi**: **azioni**, con rimpianto massimo pari a 15.

Quando si usa ciascun approccio

- **Maximax**: quando il decisore è disposto a rischiare pur di non perdere l'occasione migliore (visione ottimistica del futuro/dell'avversario)
- **Maximin**: quando occorre una garanzia di sicurezza, si vuole evitare lo scenario peggiore ad ogni costo (visione pessimistica/prudente) — è il criterio "difensivo" per eccellenza
- **Minimax regret**: quando si vuole evitare grandi rimpianti *a posteriori*, bilanciando le altre due visioni: non è né il più ottimista né il più prudente in assoluto, ma minimizza il massimo "errore di scelta"

Collegamento con i giochi con avversario

Nei giochi, la "dinamica esterna non controllabile" è costituita proprio dalle **mosse dell'avversario**: non sappiamo quale sceglierà, dobbiamo basarci solo sullo stato corrente e sulle mosse disponibili. I giocatori razionali sono **pessimisti/conservativi** per definizione (ipotesi di avversario infallibile): per questo l'algoritmo Minimax è concettualmente lo stesso approccio **maximin** applicato ricorsivamente lungo l'albero di gioco, turno dopo turno.

Potatura alfa-beta (Alpha-Beta Pruning)

Motivazione

Minimax richiede una visita **esaustiva** dell'albero, con complessità temporale esponenziale $O(b^m)$: in giochi reali (scacchi, Go) è del tutto improponibile. L'idea della potatura alfa-beta è che **non serve esplorare tutti i rami**: se durante la visita si scopre che un ramo non potrà mai influenzare la decisione finale alla radice (perché è già "battuto" da un'alternativa migliore trovata altrove), si può **evitare di espanderlo**, senza perdere nulla in termini di correttezza del risultato alla radice.

Esempio intuitivo dalle slide: se in un nodo MIN si è già trovato un successore di valore 2, e gli altri fratelli non ancora espansi sono foglie di un sottoalbero il cui valore minimo sarebbe comunque più alto di 2, è inutile espanderli: darebbero comunque un vantaggio a MAX che MIN (razionale) non permetterebbe mai di realizzare.

Definizione di α e β

Durante l'esplorazione si mantengono due valori, aggiornati e propagati lungo la ricorsione:

- **α (alfa)** = il **massimo lower bound** trovato finora per MAX lungo il cammino dalla radice al nodo corrente = il valore della miglior scelta per MAX trovata finora in un qualsiasi punto di scelta lungo il cammino. Inizialmente $\alpha = -\infty$. È aggiornato dai nodi **MAX**.
- **β (beta)** = il **minimo upper bound** trovato finora per MIN lungo il cammino = il valore della miglior scelta per MIN trovata finora lungo il cammino. Inizialmente $\beta = +\infty$. È aggiornato dai nodi **MIN**.

Ha senso continuare a esplorare un nodo **se e solo se** il suo valore stimato è compreso nell'intervallo $[\alpha, \beta]$. Idealmente, $[\alpha, \beta]$ è un intervallo che si **restringe progressivamente** man mano che la ricerca procede.

Quando si pota un ramo

Se in un certo nodo i due estremi si "invertono" (cioè si verifica $\beta \leq \alpha$), nessun valore del sottoalbero rimanente potrà mai essere contemporaneamente minore di β e maggiore di α : è quindi inutile continuare l'esplorazione di quel sottoalbero, e si può interrompere immediatamente (**potatura/pruning**).

- **Beta-pruning**: avviene in un nodo MIN quando si trova un successore con valore $v \leq \alpha$ (il nodo MIN, che sta minimizzando, non potrà mai restituire alla radice/al padre MAX un valore migliore di α , quindi il ramo padre-MAX ignorerà comunque questo nodo)
- **Alfa-pruning**: avviene simmetricamente in un nodo MAX quando si trova un successore con valore $v \geq \beta$

Nota tecnica dalle slide: il pruning avviene correttamente solo se il test usato è `if (v ≤ α) return value` (disuguaglianza **non stretta**) e non `if (v < α)`; questo garantisce la potatura anche nei casi limite in cui il valore coincide esattamente con il bound.

Pseudocodice

```
funzione ALFA-BETA-DECISION(stato) restituisce una mossa
  return arg max_{a ∈ Azioni(stato)} MIN-VALUE(Risultato(stato,a), -∞, +∞)
```

```
funzione MAX-VALUE(stato, α, β) restituisce un valore di utilità
  if TEST-TERMINALE(stato) then return Utilità(stato)
  v ← -∞
  for each a in Azioni(stato) do
    v ← MAX(v, MIN-VALUE(Risultato(stato,a), α, β))
    if v ≥ β then return v          # beta cutoff (poda: MIN non lascerà mai arrivare qui)
    α ← MAX(α, v)                  # MAX-VALUE aggiorna il lower bound
  return v
```

```
funzione MIN-VALUE(stato, α, β) restituisce un valore di utilità
  if TEST-TERMINALE(stato) then return Utilità(stato)
  v ← +∞
  for each a in Azioni(stato) do
    v ← MIN(v, MAX-VALUE(Risultato(stato,a), α, β))
    if v ≤ α then return v          # alfa cutoff (poda: MAX non sceglierà mai questo ramo)
```

```

     $\beta \leftarrow \min(\beta, v)$ 
    return v
# MIN-VALUE aggiorna l'upper bound

```

Come riassunto nelle slide: **MAX-VALUE** modifica il **lower bound** (α), **MIN-VALUE** modifica l'**upper bound** (β).

Esempio passo-passo (tracciamento sull'albero delle slide)

Riprendendo lo stesso albero di gioco usato per Minimax (radice MAX, 3 figli MIN, ciascuno con 3 foglie: gruppi {3,17,2}, {12,15,25}, {0,2,14} secondo l'ordine delle slide):

1. **Radice:** $\alpha = -\infty$, $\beta = +\infty$. Si scende nel primo figlio MIN, propagando gli stessi bound (scendendo la ricorsione i bound non cambiano fino al primo terminale)
2. Si raggiunge la prima foglia del primo nodo MIN: valore **3**. Il nodo MIN aggiorna il proprio upper bound locale: $\beta = 3$ (non può garantire a MAX più di 3, avendo già questa opzione)
3. Si esamina il secondo figlio di quel nodo MIN: valore **17**. Poiché $17 > 3$, per MIN (che vuole minimizzare) questo non migliora la sua scelta: β resta 3
4. Terminato il nodo MIN (valore finale 3, esaminati tutti i figli, es. anche 2 se presente non cambia il min), si retropropaga: il nodo MAX radice aggiorna $\alpha = 3$ (MAX sa di potersi garantire almeno 3 tramite questo ramo)
5. Si passa al secondo figlio MIN della radice, con $\alpha = 3$, $\beta = +\infty$ ereditati. Il suo primo figlio è terminale con valore **2**: il nodo MIN aggiorna $\beta = 2$. Ma ora $\beta(2) \leq \alpha(3)$: gli estremi si sono "incrociati" → **beta-pruning**: il nodo MIN non può avere un valore superiore a 2, che è già peggiore (per MAX) del 3 già garantito altrove, quindi si interrompe l'esplorazione dei restanti fratelli di questo nodo MIN (non serviranno mai a MAX). Il nodo assume il valore stimato 2 (una **stima**, perché la ricerca è stata interrotta prima di esplorare tutto)
6. Retropropagando: 2 non migliora α alla radice ($2 < 3$), quindi α resta 3 alla radice
7. Si passa al terzo figlio MIN della radice, ereditando $\alpha = 3$, $\beta = +\infty$. Si scende fino a un nodo MAX intermedio, il quale trova un figlio di valore **15**: aggiorna il proprio $\alpha = 15$ (MAX qui può garantirsi almeno 15). Ma il nodo MAX ha come vincolo dall'alto $\beta = 3$ ereditato dal genitore MIN: poiché ora $\alpha(15) \geq \beta(3)$, si verifica **alfa-pruning**: l'esplorazione si interrompe, il nodo assume il valore stimato 15
8. Il nodo MIN padre (il terzo figlio della radice) ha ora visto i valori 3 e 15 tra i suoi successori: essendo MIN, sceglie il minimo, cioè **3** (β resta 3, non modificato dal valore peggiore 15)
9. Retropropagando fino alla radice: MAX ha visto i tre rami con valori 3, 2, 3 → **max = 3**. Il valore finale della radice è **3**, identico a quello ottenuto con Minimax puro

Nell'esempio completo delle slide (20 nodi totali), alfa-beta esamina solo **11 nodi su 20**, restituendo comunque il valore corretto (3) e la stessa mossa ottima alla radice.

Equivalenza con Minimax e guadagno in complessità

Alpha-beta pruning e Minimax sono **equivalenti** nel senso che: - trovano entrambi la **stessa mossa ottima** per il nodo radice - attribuiscono al nodo radice la **stessa valutazione**

Alfa-beta raggiunge lo stesso risultato **espandendo molti meno nodi**. Nel caso migliore, la complessità temporale passa da $O(b^m)$ a $O(b^{m/2})$. Esempio intuitivo dalle slide: con $b=2$, $m=8$, si passa da $2^8=256$ nodi a $2^4=16$ nodi espansi — un risparmio enorme che, applicato ricorsivamente, permette di raddoppiare la profondità di ricerca esplorabile nello stesso tempo rispetto a Minimax puro.

Importanza dell'ordinamento delle mosse

L'efficacia della potatura **dipende fortemente dall'ordine** in cui i successori di ciascun nodo vengono considerati:

- Se il successore più promettente (**killer move**) viene esaminato **per ultimo**, non è possibile evitare di esplorare i sottoalberi dei suoi fratelli, e il guadagno di alfa-beta si riduce drasticamente
- La complessità $O(b^{m/2})$ si ottiene **solo** quando si riesce a espandere per primi i figli più promettenti (ordinamento ottimale dei successori): in questo caso il branching factor "effettivo" diventa circa la **radice quadrata** del branching factor originale (esempio dagli scacchi: $b=35$ diventa ≈ 6 con buon ordinamento e alfa-beta)
- Quando l'ordinamento dei successori **non è possibile** (caso medio, ordine casuale), la complessità è $O(b^{(3m/4)})$ — una via di mezzo tra il caso migliore e quello peggiore

? **Domanda d'esame:** perché l'ordinamento delle mosse è così cruciale per alfa-beta? Perché la potatura si verifica solo quando i bound α e β si "incrociano" abbastanza presto durante l'esplorazione dei fratelli di un nodo. Se il valore migliore (killer move) viene trovato subito, i bound si stringono rapidamente e i fratelli successivi, meno promettenti, vengono scartati senza doverli espandere. Se invece la mossa migliore viene trovata per ultima, i bound restano larghi per tutta l'esplorazione dei fratelli precedenti, e nessuna potatura può avvenire: nel caso peggiore alfa-beta degenera esattamente nella stessa complessità di Minimax, $O(b^m)$.

Come trovare le killer moves

Le slide propongono tre tecniche per stimare in anticipo quali mosse esplorare per prime: 1. **Apprendimento:** il sistema "ricorda" le esperienze passate e le usa per stimare la mossa più promettente (efficace, ma richiede tempo per imparare) 2. **Combinazione con iterative deepening:** le informazioni ottenute alle iterazioni più superficiali (profondità minori) vengono usate per ordinare le mosse alle iterazioni più profonde successive 3. **Tabelle di trasposizione:** spesso usate insieme ad alfa-beta + iterative deepening

Trasposizioni

In molti giochi, sequenze di mosse **diverse nell'ordine** possono condurre allo **stesso stato finale** (es. M1-M2-M3-M4 e M2-M3-M1-M4 producono lo stesso stato). Questi ordinamenti alternativi sono detti **trasposizioni**.

Quando lo spazio degli stati è grande, riconoscere le trasposizioni è importante per **evitare di rivisitare (ed esplorare) più volte gli stessi stati**. Si mantiene una **hash table** (tabella delle trasposizioni): ogni volta che si genera un nuovo stato, si verifica se corrisponde a uno stato già raggiunto tramite una trasposizione precedente; in caso affermativo, non lo si esplora di nuovo (si riusa il valore già calcolato). Quando lo spazio eccede le capacità di memoria disponibili, si mantengono in tabella solo le voci usate più **di frequente** o più **di recente** (politiche tipo LRU).

Ricerca con profondità limitata e funzioni di valutazione euristica

Il problema del real-time

Alfa-beta riduce lo spazio esplorato, ma **deve comunque arrivare fino agli stati terminali** per calcolare valori esatti. In giochi con alberi molto profondi (es. scacchi) questo può risultare troppo lento quando esiste un **vincolo sui tempi di risposta** (contesto *real-time*). Serve quindi introdurre dei **test di cutoff** che permettano di fermare la ricerca e produrre una decisione **prima** di raggiungere i nodi terminali.

Funzione di valutazione euristica

Idea generale: in molti problemi è possibile **categorizzare gli stati non terminali** in base a caratteristiche osservabili, stimando la probabilità che, partendo da quello stato, si arrivi alla vittoria (rispetto a pareggio/sconfitta), sulla base di statistiche sulle classi di stati simili.

In pratica, però, il numero di classi necessarie per una stima significativa è **troppo alto** da gestire esplicitamente. La soluzione standard è usare una **funzione di valutazione lineare** che combina un insieme di caratteristiche (*features*) pesate:

$$\text{eval}(s) = \sum_{i=1..k} w_i \cdot f_i(s)$$

dove f_i sono le feature dello stato (es. negli scacchi: numero di pedoni, alfieri, torri ecc. ancora posseduti da ciascun giocatore) e w_i i pesi associati, che ne indicano l'importanza relativa. Questa combinazione **lineare** è la forma più semplice possibile di aggregazione delle feature (le slide notano esplicitamente che questo concetto sarà ripreso parlando di reti neurali, dove le combinazioni possono diventare non lineari).

Alfa-beta con cutoff (versione con profondità limitata)

Si modifica l'algoritmo sostituendo il test di terminalità con un **test di taglio** generico:

```
if [TEST-TAGLIO(stato, profondità)] then return eval(stato)
```

al posto di `if TEST-TERMINALE(stato) then return Utilità(stato).`

Possibili strategie per decidere **quando tagliare**: 1. **Profondità massima predefinita**: si taglia sempre a una profondità fissata a priori 2. **Iterative deepening**: quando è il proprio turno, si usa tutto il tempo disponibile per cercare la mossa migliore con approfondimenti crescenti (si esegue la ricerca a profondità 1, poi 2, poi 3...); allo scadere del tempo disponibile si restituisce la miglior mossa trovata fino a quel momento

Problema dell'orizzonte

I tagli artificiali introducono un rischio noto come **problema dell'orizzonte**: l'algoritmo "non vede oltre" il punto di taglio, ma in certe fasi del gioco la situazione può ribaltarsi molto rapidamente subito dopo — ovvero la funzione di valutazione risulta **instabile** in quel punto. Tagliare la ricerca in questi punti è **prematurato e rischioso**, perché l'avversario potrebbe forzare successivamente un forte cambiamento della valutazione che l'algoritmo non ha potuto "vedere".

Quiescenza

Concetto proposto da Berliner (1973) per mitigare il problema dell'orizzonte: la decisione di terminare o continuare la ricerca in un nodo deve dipendere dalla **quiescenza** della funzione di valutazione in quel nodo, cioè dalla stabilità/permanenza nel tempo del segno (positivo o negativo) della valutazione.

- Nodi la cui valutazione è **quiescente** (stabile) possono essere tagliati in sicurezza
- Nodi **non quiescenti** (valutazione instabile, suscettibile di grandi variazioni imminenti) richiedono **ulteriore esplorazione** dei sottoalberi che li hanno come radice, prima di potersi fidare della stima

(Berliner realizzò anche il primo programma capace di battere un maestro umano in un gioco — nel suo caso il backgammon.)

Giochi stocastici: cenni (expectiminimax)

Le slide lette non trattano esplicitamente l'algoritmo *expectiminimax* in dettaglio (non presente come sezione dedicata in questi tre file), ma introducono la distinzione tra giochi **deterministici** e **stocastici** nella tassonomia

iniziale (es. Backgammon, Monopoli, Risiko), evidenziando che in questi giochi lo stato successivo dipende **anche da fattori esterni** non controllabili da nessuno dei due giocatori (tipicamente il lancio di dadi). Concettualmente, questo richiederebbe di estendere l'albero di gioco con **nodi di caso** (chance nodes), il cui valore si calcola come **valore atteso** (media pesata sulle probabilità) dei valori dei successori, anziché come max o min — da qui il nome *expectiminimax*. Non essendo questo argomento sviluppato nelle slide di riferimento del modulo, va eventualmente approfondito sul libro di testo (Russell & Norvig) se richiesto dal programma d'esame.

Programmi storici che giocano

Le slide riportano una rassegna di sistemi che hanno raggiunto risultati storicamente rilevanti applicando (spesso in combinazione) i concetti visti:

- **Checkers (dama)**: Samuel, Chinook
- **Othello**: Logistello
- **Backgammon**: TD-gammon
- **Go**: AlphaGo
- **Bridge**: Bridge Baron, GIB
- **Scacchi**: DeepBlue

DeepBlue

- Primo calcolatore a battere un Campione del Mondo in carica (Garry Kasparov) a scacchi, con cadenza di tempo da torneo (10 febbraio 1996)
- Hardware: sistema a parallelismo massivo con 30 nodi basati su RS/6000, supportato da 480 processori VLSI dedicati al gioco degli scacchi; sistema operativo AIX; algoritmo implementato in C
- Capacità: **200 milioni di posizioni al secondo**
- Algoritmo: **iterative deepening + alpha-beta search con tabella delle trasposizioni**
- Elemento chiave del successo: generare **estensioni oltre il limite di profondità** per le posizioni ritenute particolarmente interessanti (idea affine alla quiescenza)
- Di routine raggiungeva profondità 14, in certi casi fino a 40
- Funzione di valutazione: oltre **8000 feature**, inizializzate manualmente e poi raffinate automaticamente; database con 700.000 partite di gran maestri e ampio database di finali di partita (tutte le posizioni con 5 pezzi rimanenti, molte con 6)
- Curiosità: Kasparov sospettò (a torto o a ragione) che alcune mosse fossero state suggerite scorrettamente da un umano

AlphaGo

- Sviluppato da Google, primo programma a battere un maestro umano a Go **senza handicap**, su goban di dimensione standard (2015)
- Nelle 500 partite disputate contro altri programmi: le vinse quasi tutte su singolo computer (tranne una), tutte quando eseguito su un cluster (1202 CPU + 176 GPU, circa 25 volte più hardware); la versione cluster batté quella single-computer nel 77% delle partite
- Approccio: combinazione di **deep learning (reti neurali)** e **ricerca su alberi**; le reti furono addestrate su un dataset di 30.000.000 di mosse umane e poi ulteriormente raffinate tramite **self-play** (partite contro se stesso)

Un parallelo con la memoria umana

Le slide chiudono il modulo con un confronto tra la memoria “di lavoro” usata dagli algoritmi di ricerca (il numero di nodi che occorre mantenere in memoria durante la visita) e la capacità della memoria di lavoro (a breve termine) umana:

- **Legge di Miller (1956):** la capacità della memoria a breve termine umana è stimata in 7 ± 2 elementi
- **Cowan (2001):** revisione più recente, stima 4 ± 1 elementi (“The magical number 4 in short-term memory: A reconsideration of mental storage capacity”)

Il parallelo è un modo per collegare, in chiave interdisciplinare, i limiti computazionali/di memoria degli algoritmi di ricerca con i limiti cognitivi umani nell'affrontare compiti analoghi (da cui l'uso di euristiche anche nel gioco umano).

Riepilogo e punti chiave

- I **giochi** sono problemi di ricerca **multi-agente competitiva**: gli obiettivi degli agenti sono in conflitto, e le mosse dell'avversario non sono controllabili. Si distinguono per informazione (completa/incompleta) ed effetti delle scelte (deterministico/stocastico); questo modulo tratta soprattutto giochi **deterministici a informazione completa, a due giocatori, a somma zero**.
- In un **gioco a somma zero**, il guadagno di un giocatore è l'esatto opposto della perdita dell'altro: basta un solo valore di utilità per nodo per descrivere entrambi i giocatori.
- L'**albero di gioco** rappresenta tutte le evoluzioni possibili; i nodi si alternano tra turno di **MAX** (massimizza) e turno di **MIN** (minimizza, rappresenta l'avversario, assunto infallibile/ottimale).
- **Minimax** calcola ricorsivamente, dal basso verso l'alto, il valore di ogni nodo (max nei nodi MAX, min nei nodi MIN, utilità diretta nelle foglie), garantendo ad ogni giocatore il miglior risultato ottenibile contro un avversario ottimale. Complessità: $O(b^m)$ in tempo, $O(bm)$ in spazio; completo e ottimale (se entrambi giocano in modo ottimale).
- Gli approcci di **teoria delle decisioni** (maximax, maximin, minimax regret) generalizzano il ragionamento a scenari con esiti incerti non controllabili: **maximax** è ottimistico (massimizza il miglior caso), **maximin** è pessimistico/conservativo (massimizza il caso peggiore — è l'approccio concettualmente identico a Minimax), **minimax regret** minimizza il massimo rimpianto rispetto alla scelta ottimale a posteriori.
- La **potatura alfa-beta** rende Minimax praticabile potando i rami che non possono influenzare la decisione alla radice, mantenendo α (miglior garanzia per MAX) e β (miglior garanzia per MIN) lungo la ricorsione: si pota quando $\beta \leq \alpha$. Trova sempre lo **stesso risultato** di Minimax, ma con complessità che nel caso migliore scende a $O(b^{m/2})$ — dimezzando l'esponente, quindi permettendo di raddoppiare la profondità esplorabile a parità di tempo.
- L'efficacia della potatura dipende **criticamente dall'ordinamento delle mosse**: esplorare prima le mosse più promettenti (killer move) è ciò che permette di avvicinarsi al caso migliore; senza un buon ordinamento la complessità peggiora fino a $O(b^{3m/4})$.
- Nei giochi reali, dove non si può visitare l'intero albero in tempo utile, si introducono **cutoff a profondità limitata** con **funzioni di valutazione euristica** (tipicamente combinazioni lineari pesate di feature), incorrendo però nel **problema dell'orizzonte**; la nozione di **quiescenza** aiuta a decidere in modo più robusto dove tagliare la ricerca.
- Programmi storici come **DeepBlue** (scacchi, iterative deepening + alfa-beta + tabelle di trasposizione) e **AlphaGo** (Go, reti neurali + ricerca ad albero + self-play) sono applicazioni celebri di questi principi, ciascuno esteso con tecniche aggiuntive (funzioni di valutazione enormi, apprendimento automatico) per gestire la complessità dei rispettivi giochi.

Modulo 4: Constraint Satisfaction Problems (CSP)

I problemi visti nei moduli precedenti (ricerca cieca ed euristica) trattano gli stati come “scatole nere”: l'unica cosa che l'algoritmo di ricerca può fare è generare successori, testare l'obiettivo, eventualmente stimare una distanza tramite euristica. Nei CSP si apre la scatola: **lo stato ha una struttura interna esplicita** (un insieme di variabili con valori), e i vincoli fra le variabili sono rappresentati in modo dichiarativo. Questo permette di costruire algoritmi generali di risoluzione (non specifici per un singolo problema) e tecniche di potatura molto più efficaci della ricerca cieca.

Esempi di ambiti in cui i vincoli sono centrali: design di oggetti (vincoli funzionali/fisici), allocazione di risorse (quantità, priorità, tempi), progettazione di circuiti, pianificazione di percorsi robotici (es. manovre di parcheggio).

Definizione formale di CSP

Un **constraint satisfaction problem (CSP)** è definito da:

- un insieme di **variabili** $X_1, \dots, X_n$;
- un insieme di **domini** $D_1, \dots, D_n$ (uno per variabile, con i valori ammissibili);
- un insieme di **vincoli** $C_1, \dots, C_m$ che restringono le combinazioni di valori ammissibili;
- opzionalmente, una **funzione obiettivo** da massimizzare/minimizzare (nel caso più semplice di CSP “puro” non è richiesta: basta trovare un assegnamento che soddisfi tutti i vincoli).

Stati e assegnamenti

- Gli **stati** di un CSP corrispondono a tutti i possibili **assegnamenti** di valori alle variabili.
- Un **assegnamento** $\{X_1 = v_1, X_2 = v_2, \dots\}$ attribuisce valori a un sottoinsieme (anche vuoto o totale) delle variabili.
- Un assegnamento è detto:
 - **completo** se assegna un valore a tutte le variabili;
 - **consistente** se non viola alcun vincolo;
 - **soluzione** se è completo e consistente.
- Quando esiste una soluzione si dice anche che esiste un “mondo possibile” che soddisfa i vincoli.

Esempio guida: colorazione della mappa dell'Australia

Colorare i 7 territori dell'Australia (WA, NT, SA, Q, NSW, V, T) con i colori {R, G, B} in modo che due territori confinanti non abbiano lo stesso colore.

- **Variabili:** WA, NT, SA, Q, NSW, V, T (una per territorio).

- **Domini:** uguali per tutte, $D = \{R, G, B\}$.
- **Vincoli** (binari, uno per ogni coppia di territori confinanti): $WA \neq NT, WA \neq SA, NT \neq SA, NT \neq Q, SA \neq Q, SA \neq NSW, SA \neq V, Q \neq NSW, NSW \neq V$. (la Tasmania T è un'isola: non confina con nessuno, quindi non ha vincoli binari con le altre variabili).

Un possibile assegnamento parziale, es. $\{WA=R, NT=R, SA=G, \dots\}$, non è consistente (WA e NT confinano e hanno lo stesso colore). Un assegnamento consistente è $\{WA=R, NT=G, SA=B, \dots\}$. Una soluzione completa è $\{WA=R, NT=G, SA=B, Q=R, NSW=G, V=R, T=B\}$: da notare che **qualunque permutazione dei tre colori applicata a una soluzione è ancora una soluzione** (simmetria del problema).

I vincoli binari si rappresentano naturalmente come archi di un **grafo di vincoli**, i cui nodi sono le variabili: questa rappresentazione è centrale per gli algoritmi di propagazione (arc consistency, AC-3) che vedremo più avanti.

Tipi di domini e di variabili

- **Domini finiti:** è possibile enumerare esplicitamente i vincoli mettendo in relazione i valori (es. Australia). Il caso particolare in cui il dominio è $\{\text{vero}, \text{falso}\}$ dà luogo a CSP **booleani**, la cui decisione è NP-completa (es. 3-SAT).
- **Domini infiniti (discreti):** non si possono enumerare i vincoli; si usano linguaggi di specifica. Esempio: per dire che "Lavoro2 deve iniziare almeno 5 giorni dopo Lavoro1" si scrive $\text{Lavoro1} + 5 < \text{Lavoro2}$.
- **Domini continui:** tipici dei problemi di scheduling con tempi di inizio/fine reali. Se i vincoli sono disuguaglianze lineari che definiscono una regione convessa, il CSP si risolve con la **programmazione lineare** in tempo polinomiale.

Arità dei vincoli

- **Vincoli unari:** coinvolgono una sola variabile (e un valore). Esempio: $T \neq G$ (la Tasmania non può essere verde).
- **Vincoli binari:** coinvolgono due variabili; sono gli archi del grafo di vincoli. Esempio: $WA \neq NT$.
- **Vincoli a tre o più variabili (n-ari):** coinvolgono un numero qualsiasi di variabili; si rappresentano con **ipergrafi** (archi che connettono più di due nodi). Esempio: $\text{diverse}(WA, NT, SA)$. Spesso possono essere riscritti come insieme equivalente di vincoli binari.

Esempio classico di vincolo n-ario: criptoaritmetica

```

SEND
+ MORE
-----
MONEY

```

Ogni lettera rappresenta una cifra diversa (S, E, N, D, M, O, R, Y). Dominio di S e M: $\{1, \dots, 9\}$ (non possono essere zero, essendo cifre iniziali); dominio delle altre lettere: $\{0, \dots, 9\}$. Vincoli: "a lettere diverse corrispondono valori diversi" (alldifferent) più il vincolo aritmetico globale, che coinvolge **tutte** le variabili e non è scomponibile in vincoli binari:

$$(1000 \cdot S + 100 \cdot E + 10 \cdot N + D + 1000 \cdot M + 100 \cdot O + 10 \cdot R + E) = (10000 \cdot M + 1000 \cdot O + 100 \cdot N + 10 \cdot E + Y)$$

Vincoli vs. criteri di preferenza

I vincoli possono essere più o meno rigidi:

- una **soluzione** deve soddisfare **tutti** i vincoli veri e propri;
- una soluzione **può violare** uno o più **criteri di preferenza** (soft constraints);

- il soddisfacimento dei criteri di preferenza permette di **ordinare** le soluzioni ammissibili, distinguendo quelle preferibili da quelle meno preferibili.

Esempio: nella costruzione di un orario un docente preferisce le lezioni del mattino; questo “vincolo” è in realtà un criterio di preferenza da comporre con quelli di altri docenti. Il risultato finale potrebbe non soddisfare la preferenza (il docente finisce sempre al pomeriggio), ma può comunque essere la soluzione migliore fra quelle disponibili — non ottimale rispetto alla preferenza, ma ottima rispetto ai vincoli rigidi più il criterio complessivo.

CSP come problema di ricerca nello spazio degli stati

I CSP si possono formulare come problemi di ricerca:

- **stato iniziale** = assegnamento vuoto { };
- **funzione successore** = assegna un valore a una delle variabili non ancora assegnate, scartando le scelte che generano conflitti immediati;
- **test obiettivo** = l'assegnamento corrente è completo (tutte le variabili hanno un valore);
- **costo di cammino** = costante per ogni passo (es. 1), perché conta solo raggiungere un assegnamento completo consistente, non “quanto costa” arrivarci.

Poiché ogni passo valorizza esattamente una variabile precedentemente non assegnata, **la profondità massima dell'albero di ricerca è n** (il numero di variabili).

Perché l'ordine di assegnamento non conta: commutatività

Un aspetto fondamentale, specifico dei CSP rispetto alla ricerca generica: **uno stato è un assegnamento di valori** $\{X_1=v_1, \dots, X_n=v_n\}$, e l'ordine (il cammino) con cui i valori sono stati assegnati alle variabili è **irrilevante ai fini del risultato**. Assegnare prima $X_1=v_1$ e poi $X_2=v_2$, oppure prima $X_2=v_2$ e poi $X_1=v_1$, porta allo stesso stato finale $\{X_1=v_1, X_2=v_2\}$. Si dice che i **CSP sono commutativi**.

Questa proprietà è cruciale dal punto di vista pratico: **gli algoritmi possono sempre scegliere prima una variabile e poi un valore per essa**, senza bisogno di considerare tutti gli ordini possibili delle variabili come rami distinti dell'albero di ricerca. Questo riduce drasticamente il fattore di ramificazione effettivo, come mostrato di seguito.

Rappresentare gli stati: generate-and-test vs. ricerca con backtracking

Approccio a forza bruta: generate-and-test

Usa **solo informazioni di stato**, ignorando i vincoli durante la generazione:

finché non hai una soluzione e ci sono alternative:

- 1) genera un assegnamento completo
- 2) controlla se è consistente
- 3) se sì: è una soluzione, esci dal ciclo
- 4) se no: torna al passo 1

se hai una soluzione: restituiscila

altrimenti: fallimento

Può richiedere di esplorare **l'intero spazio degli assegnamenti completi**, la maggior parte dei quali inconsistente: è enormemente dispendioso.

Esempio, 8 regine: posizionare 8 regine su una scacchiera 8×8 senza che nessuna sia sotto attacco (esistono 92 soluzioni, 12 a meno di simmetrie).

- *Rappresentazione 1*: variabili Q1..Q8 = posizione (1..64) della regina i-esima $\rightarrow d^n = 64^8 \approx 2,81 \times 10^{14}$ assegnamenti possibili.
- *Rappresentazione 2* (migliore, sfrutta la conoscenza che ogni regina occupa una colonna diversa): variabili Q1..Q8 = numero di riga della regina nella colonna i, dominio {1..8} $\rightarrow 8^8 = 16.777.216$ assegnamenti, molti meno ma ancora troppi.
- Con 16 regine si arriverebbe a $16^{16} \approx 1,8 \times 10^{19}$ configurazioni, stimate in ~1200 anni di computazione con generate-and-test.

La lezione è che **come rappresentiamo il problema conta moltissimo**, e che introdurre i vincoli esplicitamente (invece di generare e poi testare) è la strada per rendere trattabile la ricerca.

Ricerca in profondità con backtracking

Si esplora lo spazio degli assegnamenti con una **depth-first search**, ma si usano i vincoli per **potare** (pruning) i cammini appena si genera un conflitto, invece di aspettare l'assegnamento completo. È una ricerca non informata + vincoli. La limitatezza della profondità dei cammini ($= n$) rende ragionevole la profondità, anche se il fattore di ramificazione può essere alto.

Analisi del branching factor: con n variabili e d valori medi per variabile, se **non** si sfrutta la commutatività, al primo livello il fattore di ramificazione è $n \cdot d$ (si può scegliere una qualsiasi fra n variabili e uno qualsiasi fra d valori), al secondo livello $(n-1) \cdot d$, ecc. L'albero avrebbe **$n! \cdot d^n$ foglie** (nell'esempio Australia: 7 variabili, 3 valori $\rightarrow 7! \cdot 3^7 = 11.022.480$ foglie).

Sfruttando la commutatività (si fissa **prima la variabile**, poi si sceglie il valore), il fattore di ramificazione si riduce a **d per livello**, e le foglie scendono a **d^n** (nell'esempio Australia: $3^7 = 2.187$, un risparmio enorme rispetto a 11.022.480).

Pseudocodice del backtracking

funzione BACKTRACKING-SEARCH(csp) restituisce soluzione o fallimento
 restituisci BACKTRACK({}, csp)

```

funzione BACKTRACK(assegnamento, csp) restituisce soluzione o fallimento
  se assegnamento è completo allora restituisci assegnamento
  var ← SELECT-UNASSIGNED-VARIABLE(csp)
  per ogni valore in ORDER-DOMAIN-VALUES(var, assegnamento, csp) fai
    se valore è consistente con assegnamento (rispetto ai vincoli) allora
      aggiungi {var = valore} ad assegnamento
      inferenze ← INFERENCE(csp, var, valore)
      se inferenze ≠ fallimento allora
        aggiungi inferenze ad assegnamento
        risultato ← BACKTRACK(assegnamento, csp)
        se risultato ≠ fallimento allora restituisci risultato
      rimuovi {var = valore} e le inferenze da assegnamento
  restituisci fallimento
  
```

Punti chiave: si fissa **una variabile per volta** (SELECT-UNASSIGNED-VARIABLE), si cerca un **valore consistente** con l'assegnamento parziale corrente secondo i vincoli, e si procede ricorsivamente; quando **tutti** i valori per la variabile corrente sono stati provati senza successo, si arriva a "fallimento" e si fa **backtracking** verso la chiamata precedente per esplorare un'altra alternativa.

Limiti del backtracking semplice: il thrashing

Il backtracking di base è una ricerca in profondità **non informata**: non è particolarmente efficiente, ed è soggetta a **thrashing**: lo stesso errore (lo stesso assegnamento incompatibile fra due variabili) può ripetersi identico in più punti diversi dell'albero di ricerca, perché l'algoritmo non "ricorda" perché ha fallito.

Esempio: se il valore v_i assegnato a X_i rende impossibile ogni valore per X_j (variabile scelta successivamente), il sottoalbero fallisce. Se in un altro ramo dell'albero si arriva di nuovo a scegliere prima $X_i=v_i$ e poi X_j , si ripete esattamente lo stesso fallimento, sprecando lavoro.

Per migliorare servono risposte a tre domande:

1. L'assegnamento della variabile corrente ha un impatto sulle variabili non ancora assegnate? → **inferenza/propagazione dei vincoli** (forward checking, arc consistency...).
2. L'ordinamento dei valori da provare influisce sul risultato? → **euristica del valore** (least constraining value).
3. Quando un cammino fallisce, è possibile fare tesoro dell'esperienza per evitare fallimenti simili? → **backjumping**, conflict set, apprendimento di NOGOOD.

Euristiche per la scelta della variabile

A ogni passo del backtracking va scelta una variabile non assegnata su cui procedere. Farlo a caso è sub-ottimale.

Minimum Remaining Values (MRV, o euristica "fail-first")

Sceglie la variabile con il **minor numero di valori legali rimasti** (consistenti con l'assegnamento corrente). L'idea è "fallire il prima possibile", così da scoprire subito i vicoli ciechi invece di scoprirli tardi dopo aver investito lavoro su altre variabili.

Esempio (Australia): assegnati $WA=R$ e $NT=B$, restano 5 variabili. SA ha un solo valore possibile (deve essere diversa da rosso e da blu → verde): è la scelta più vincolata, quindi MRV sceglie SA.

Degree heuristic (euristica di grado)

MRV non aiuta sempre: ad esempio nella scelta della **prima** variabile da assegnare (nessun vincolo è ancora stato attivato, tutte le variabili hanno lo stesso numero di valori residui) o in caso di **pareggio** fra più variabili con lo stesso MRV. In questi casi si usa l'**euristica di grado**: si sceglie la variabile **coinvolta nel maggior numero di vincoli** con le altre variabili non ancora assegnate.

Esempio: se l'unico assegnamento fatto finora è $NT=B$, WA, SA e Q sono in parimerito per MRV (ciascuna con 2 valori consistenti); l'euristica di grado sceglie **SA**, che è coinvolta in 5 vincoli (confina con WA, NT, Q, NSW, V), più di ogni altra variabile.

Le due euristiche si usano in combinazione: MRV come criterio principale, degree heuristic come tie-breaker.

Euristica per la scelta del valore: Least Constraining Value (LCV)

Una volta scelta la variabile, in che ordine provare i suoi valori possibili? L'euristica del **valore meno vincolante** predilige il valore che **lascia più libertà** (più opzioni residue) alle variabili adiacenti nel grafo dei vincoli.

Esempio: assegnati $WA=R$ e $NT=G$, si deve assegnare Q, che può valere B o R. I vicini di Q sono SA (dominio residuo {B}) e NSW (dominio residuo {R,G,B}):

- se $Q=B$: SA non ha più valori possibili (dominio vuoto!) e NSW si riduce a 2 valori;
- se $Q=R$: SA non viene per nulla ristretto, e NSW resterà con {G,B}.

Quindi **$Q=R$ è la scelta meno vincolante**: una scelta più restrittiva fatta in anticipo rischia di tagliare fuori soluzioni raggiungibili, causando fallimenti evitabili più avanti nella ricerca.

Da notare la logica opposta rispetto a MRV: per la **variabile** si vuole la più vincolata (fail-first, per potere presto rami sbagliati), per il **valore** si vuole il meno vincolante (per non eliminare a priori soluzioni buone). MRV serve a decidere *cosa* assegnare, LCV serve a decidere *come* provare ad assegnarlo.

? Domanda d'esame: perché per la scelta della variabile si usa "la più vincolata" (MRV) mentre per la scelta del valore si usa "il meno vincolante" (LCV)? Non è contraddittorio? Non è contraddittorio perché i due criteri rispondono a obiettivi diversi. Scegliere la variabile più vincolata (con meno valori residui) serve a **individuare il prima possibile i vicoli ciechi**: se una variabile ha pochissime opzioni è più probabile che porti presto a un fallimento, ed è meglio scoprirlo subito piuttosto che dopo aver costruito un assegnamento parziale grande e costoso da disfare (principio del "fail-first"). Scegliere invece, per quella variabile, il valore meno vincolante per le altre serve a **massimizzare la probabilità di successo** del ramo che si sta esplorando, lasciando il maggior numero possibile di opzioni aperte alle variabili adiacenti non ancora assegnate. In sintesi: si sceglie "dove rischiare di fallire subito" (variabile) e poi, una volta lì, "come minimizzare il rischio di fallire" (valore).

Rendere informata la ricerca: propagazione dei vincoli (inferenza)

Le euristiche di ordinamento migliorano *come* si esplora l'albero, ma non riducono di per sé lo spazio di ricerca. Il passo successivo è alternare fasi di **esplorazione** (assegnare una variabile) a fasi di **inferenza**: dopo ogni assegnamento si propagano le conseguenze sui domini delle altre variabili, eliminando valori che non potranno mai portare a una soluzione consistente con la scelta appena fatta. Questo è il principio della **consistenza locale**: ci si concentra su una proprietà di consistenza relativa a una parte del CSP (un sottoproblema) e si propaga l'informazione a tutto il grafo, cercando di costruire (o scartare) la soluzione.

Le proprietà/tecniche di consistenza locale principali, in ordine crescente di forza (e di costo):

1. **Forward checking** (più un approccio pratico che una proprietà formale)
2. **Node consistency** (riguarda singole variabili / vincoli unari)
3. **Arc consistency** (riguarda coppie di variabili / vincoli binari)
4. **Path consistency** (riguarda triplette di variabili)

Forward checking

Idea: quando si assegna un valore a una variabile, si **guardano subito in avanti** i vicini diretti sul grafo dei vincoli e si eliminano dai loro domini i valori che sarebbero ormai incompatibili, invece di scoprirlo solo quando si arriverà ad assegnare quei vicini.

Esempio: se $WA=R$, i vincoli impongono che i vicini diretti NT e SA non potranno più essere R: si rimuove R dai loro domini residui **subito**, prima ancora di doverli assegnare. Il forward checking si accompagna naturalmente a MRV, perché aggiorna in tempo reale il numero di valori residui di ogni variabile (l'informazione di cui MRV ha bisogno).

Esempio numerico (Australia): la tabella seguente mostra come si riducono i domini man mano che si effettuano gli assegnamenti $WA=R$, poi $Q=G$, poi $V=B$:

	WA	NT	Q	NSW	V	SA	T
INIZIO	RGB	RGB	RGB	RGB	RGB	RGB	RGB
$WA=R \rightarrow$	R	GB	RGB	RGB	RGB	GB	RGB
$Q=G \rightarrow$	R	B	G	RB	RGB	B	RGB
$V=B \rightarrow$	R	B	G	R	B	$\emptyset$	RGB

Quando SA arriva a dominio vuoto (perché confina con WA, NT, Q e V, tutti già fissati su colori diversi), si ha fallimento immediato e si fa backtracking, senza dover neppure provare ad assegnare SA esplicitamente.

Limiti del forward checking

Il forward checking **non cattura tutte le inconsistenze**, perché propaga solo dal nodo appena assegnato verso i suoi vicini diretti, senza controllare le conseguenze fra coppie di vicini fra loro. Esempio: dopo $WA=R$ e $Q=G$, i domini residui di NT e SA restano entrambi $\{B\}$ (entrambi ridotti a un solo valore, lo stesso): questo forza un'inconsistenza, perché NT e SA sono confinanti e sarebbero costretti ad avere **lo stesso** colore B, violando il vincolo $NT \neq SA$. Il forward checking, però, non se ne accorge, perché non confronta i domini residui di NT e SA fra loro: se ne accorgerà solo più avanti nella ricerca, quando tenterà di assegnarli esplicitamente.

Node consistency

Un grafo di vincoli è **node consistent** quando lo sono tutte le sue variabili; una variabile è node consistent quando **rispetta tutti i vincoli unari** che la riguardano. Esempio: il vincolo "età compresa fra 18 e 35 anni" si esprime con due vincoli unari, $età > 18$ e $età < 35$; l'algoritmo di node consistency filtra dal dominio della variabile "età" tutti i valori che non li rispettano. È la forma più semplice di consistenza locale, e si applica una tantum come preprocessing sui domini iniziali (rende i vincoli unari "impliciti" nel dominio, così gli algoritmi successivi possono ignorarli).

Arc consistency

Un grafo di vincoli è **arc consistent** quando rispetta tutti i vincoli binari. È una proprietà importante perché **ogni vincolo a tre o più variabili può sempre essere trasformato in un insieme equivalente di vincoli binari**, dunque tecniche pensate per l'arc consistency hanno applicabilità generale.

Nota di confronto con forward checking: il forward checking, di fatto, realizza un controllo di arc consistency limitato ai soli archi che collegano la variabile appena assegnata con i suoi vicini diretti; **non propaga** però il ragionamento ai vicini dei vicini. L'arc consistency, applicata sistematicamente (con un algoritmo come AC-3), fa esattamente questo: propaga a catena su tutto il grafo.

Definizione formale: dato un grafo di vincoli, un **arco** è un lato orientato (l'arco $WA \rightarrow NSW$ è diverso, e va trattato separatamente, dall'arco $NSW \rightarrow WA$). Un arco $X \rightarrow Y$ è **consistente** quando: per ogni valore possibile di X (vertice sorgente) esiste **almeno un** valore di Y (vertice destinazione) ad esso consistente rispetto al vincolo.

Esempio: se il dominio di WA è $\{R, G\}$ e il dominio di NSW è $\{R\}$: l'arco $WA \rightarrow NSW$ **non** è consistente, perché se $WA=R$ allora NSW non ha alcun valore assegnabile (essendo $NSW=\{R\}$ e dovendo $NSW \neq WA$); l'arco $NSW \rightarrow WA$, invece, è consistente (per l'unico valore di NSW, R, esiste un valore consistente in WA, cioè G).

Quando un arco non è consistente, si possono **eliminare valori dal dominio della variabile sorgente** finché l'arco non diventa consistente.

Algoritmo AC-3

Sviluppato nel 1977 da Alan Mackworth (Arc Consistency algorithm #3). Viene insegnato perché è più efficiente dei precedenti (AC-1, AC-2) e più semplice dei successivi (AC-4, ecc.). Si può usare come **preprocessing** prima della ricerca, oppure **intrecciato con il backtracking** per propagare via via le scelte fatte: quest'ultimo uso prende il nome di algoritmo **MAC** (Maintaining Arc Consistency).

Idea generale: si mantiene una coda (queue) di archi da verificare. Si estrae un arco ($X_i \rightarrow X_j$), si controlla se è consistente; se si eliminano valori dal dominio di X_i per renderlo consistente, **tutti gli archi che puntano a X_i** (del tipo $X_k \rightarrow X_i$, per ogni vicino X_k di X_i diverso da X_j) devono essere rimessi in coda, perché la riduzione del dominio di X_i potrebbe aver reso quegli archi non più consistenti.

funzione AC-3(csp) restituisce false se un dominio è svuotato (CSP inconsistente), true

```

↪ altrimenti
    queue ← insieme di tutti gli archi ( $X_i, X_j$ ) del csp
    finché queue non è vuota:
        rimuovi un arco ( $X_i, X_j$ ) da queue

```

```

se REVISE(csp, Xi, Xj) allora
  se Di è vuoto allora restituisci false
  per ogni Xk vicino di Xi,  $Xk \neq Xj$ :
    aggiungi (Xk, Xi) a queue
  restituisci true

```

funzione REVISE(csp, Xi, Xj) restituisce true se il dominio di Xi è stato modificato

```

revised ← false
per ogni x in Di:
  se non esiste alcun valore y in Dj tale che (x,y) soddisfa il vincolo tra Xi e Xj:
    elimina x da Di
    revised ← true
restituisci revised

```

Esempio guidato (Australia, con WA=R, Q=G già assegnati): si inizializza la coda con tutti gli archi orientati (WA→NT, NT→WA, NT→Q, Q→NT, WA→SA, SA→WA, SA→Q, Q→SA, ...). Si estraggono e verificano man mano:

- WA→NT: consistente (R è compatibile con G e B, i valori residui di NT) → si rimuove semplicemente dalla coda.
- NT→WA: **non** consistente (R di NT è incompatibile con R di WA) → si rimuove R dal dominio di NT; si rimette in coda ogni arco $Xk \rightarrow NT$ (WA→NT, SA→NT, Q→NT).
- NT→Q: **non** consistente (G di NT incompatibile con G di Q) → si rimuove G da NT; si rimettono in coda gli archi verso NT (già presenti).
- Q→NT, WA→SA: consistenti.
- SA→WA: **non** consistente (R di SA incompatibile con R di WA) → si rimuove R da SA; si rimette in coda WA→SA.
- NT→WA, WA→NT: consistenti (di nuovo, coi domini aggiornati).
- SA→Q: **non** consistente (G di SA incompatibile con G di Q) → si rimuove G da SA; si rimettono in coda gli archi verso SA.
- NT→WA, WA→NT: consistenti.
- Q→SA: consistente.
- NT→SA: **non** consistente (l'unico valore rimasto in NT, B, è incompatibile con l'unico valore rimasto in SA, B) → si rimuove B da NT.
- A questo punto **NT ha dominio vuoto**: l'algoritmo segnala che l'assegnamento WA=R, Q=G **non è consistente** (nessuna estensione è possibile), senza dover esplorare esplicitamente tutte le variabili rimanenti.

Questo esempio mostra bene il valore pratico di AC-3: scopre l'inconsistenza **per propagazione**, risparmiando la ricerca esplicita.

Costo computazionale di AC-3

- Un CSP con n variabili e vincoli binari ha al più n^2 archi.
- Sia d il numero massimo di valori nel dominio di una variabile.
- Il tempo nel caso peggiore è $O(n^2 d^3)$.
- È più costoso del forward checking, ma anche più efficace (elimina più inconsistenze).
- **AC-3 è incompleto**: esistono assegnamenti globalmente inconsistenti che l'arc consistency da sola non rileva (vedi sotto).

Incompletezza di AC-3 (arc-consistent ma senza soluzione)

Esempio: un grafo a 3 nodi (1, 2, 3) tutti collegati a due a due (triangolo), con dominio {bianco, nero} per ciascuno e vincolo "colori diversi" fra ogni coppia collegata. Il grafo è **arc consistent** (per ogni valore di ogni nodo esiste un valore compatibile nel vicino), **eppure non esiste soluzione**, perché con solo 2 colori non si può colorare un triangolo (3-clique) in modo che tutte le coppie abbiano colori diversi. Questo

mostra che l'arc consistency, da sola, **non è sufficiente** a garantire (né a rilevare l'assenza di) una soluzione: serve una proprietà più forte, la **path consistency**, oppure comunque va completata con una fase di ricerca (backtracking) sopra i domini ridotti.

Path consistency

Proprietà **più forte** dell'arc consistency: identifica vincoli impliciti dedotti da **triplette** di variabili. Una coppia di variabili $\{X_1, X_2\}$ è path consistent rispetto a una terza variabile X_3 quando: per ogni assegnamento $\{X_1=a, X_2=b\}$ consistente con i vincoli esistenti fra X_1 e X_2 , esiste un valore di X_3 che soddisfa contemporaneamente i vincoli su $\{X_1, X_3\}$ e $\{X_2, X_3\}$.

Esempio (Australia semplificata a 2 colori R/B): consideriamo la coppia $\{WA, SA\}$ e la variabile NT . I casi consistenti per $\{WA, SA\}$ sono $\{WA=R, SA=B\}$ o $\{WA=B, SA=R\}$. In entrambi i casi NT non ha alcun valore compatibile con entrambi WA e SA contemporaneamente (NT confina con entrambi e con solo 2 colori disponibili non può differire da entrambi): quindi il CSP **non ha soluzione**, cosa che l'arc consistency da sola non avrebbe rilevato.

L'algoritmo **PC-2** (sempre di Mackworth) propaga la path consistency in modo analogo a come AC-3 propaga l'arc consistency, ma con complessità maggiore.

Generalizzazione: k-consistency

Un CSP è **k-consistent** quando: per ogni sottoinsieme di $k-1$ variabili e per ogni loro assegnamento consistente, è sempre possibile trovare un assegnamento consistente per una qualunque k -esima variabile.

- 1-consistency = node consistency
- 2-consistency = arc consistency
- 3-consistency = path consistency

Un CSP è **fortemente k-consistent** quando è k -consistent, $(k-1)$ -consistent, ..., fino a 1-consistent (cioè tutti i livelli di consistenza fino a k sono soddisfatti simultaneamente). Ad esempio, un CSP fortemente 3-consistent ha: tutte le triplette di variabili legate da vincoli che soddisfano la path consistency, tutte le coppie legate da vincolo che soddisfano l'arc consistency, e tutte le variabili che soddisfano i vincoli unari.

Risultato teorico: un CSP fortemente k -consistent (con k pari al numero di variabili n) può essere risolto **senza mai fare backtracking**, in tempo $O(n \cdot d)$: intuitivamente, essendo 1-consistent si può iniziare da un valore consistente per X_1 ; essendo 2-consistent si trova un valore consistente per le variabili direttamente legate a X_1 ; essendo 3-consistent si trova un valore consistente per le variabili legate a coppie delle precedenti; e così via. Il problema pratico è che **decidere se un CSP è fortemente k-consistent ha esso stesso costo esponenziale**, quindi in pratica questo risultato è più di interesse teorico che applicativo diretto — motiva però l'uso di tecniche di consistenza parziale (arc consistency) come compromesso costo/beneficio.

Vincoli speciali (globali)

Alcuni tipi di vincoli ricorrono così spesso da essere stati studiati con algoritmi dedicati più efficienti del trattamento generico:

Alldifferent($X_1, \dots, X_n$)

Impone che tutte le variabili elencate assumano valori tutti diversi fra loro (generalizzazione del vincolo binario "diverso da" a n variabili). Se le n variabili possono assumere complessivamente **meno di n valori distinti** ($m < n$), il vincolo non può mai essere soddisfatto (pigeonhole principle) e si rileva subito il fallimento. Altrimenti, si assegnano per prime le variabili che hanno un solo valore possibile, si verifica che il vincolo resti soddisfatto, e si propaga la scelta riducendo via via i domini delle altre variabili coinvolte (simile in spirito al forward checking, ma specializzato per questo tipo di vincolo).

Atmost(N, A1, ..., Ak)

Vincolo di risorse: le attività $A_1, \dots, A_k$ possono complessivamente impegnare **al più N** risorse. Supponendo i domini di ogni A_i ordinati crescentemente, si sommano i valori minimi $v_{11} + \dots + v_{k1}$: se la somma supera N, il vincolo è insoddisfacibile; altrimenti si possono eliminare i valori massimi che, sommati ai minimi altrui, violerebbero il limite N. Per domini molto grandi (es. logistica, dove non è pratico enumerare esplicitamente ogni valore) i domini si rappresentano come **intervalli** $[v_{i1}, v_{im}]$ e la propagazione agisce sulla ridefinizione degli estremi degli intervalli, non su enumerazioni esplicite.

Oltre il backtracking cronologico: backjumping

Il limite del backtracking cronologico

Quando il backtracking standard raggiunge un vicolo cieco, torna indietro **sempre e solo** alla variabile assegnata al passo immediatamente precedente (backtracking **cronologico**), indipendentemente dal fatto che quella variabile abbia o meno causato il conflitto. Questo non sfrutta l'informazione sui vincoli ed eredita i limiti delle strategie di ricerca cieche.

Esempio: assegnamento corrente $\{Q=R, NSW=G, V=B, T=R\}$. Si sceglie SA: tutti i valori possibili vanno in conflitto, perché Q, NSW e V (tutti confinanti con SA) coprono già l'intero spettro dei colori. La variabile scelta alla chiamata ricorsiva precedente era **T**, quindi il backtracking cronologico tornerebbe indietro su T — ma **T non ha nessun ruolo nel conflitto** (T non confina con SA)! Si sprecherebbe lavoro riprovando altri valori di T inutilmente.

Backjumping: backtracking non cronologico

Strategia più intelligente: tornare indietro direttamente a **una variabile che potrebbe risolvere il problema** (nell'esempio: V). Per farlo, ogni variabile mantiene un **conflict set**: l'insieme degli assegnamenti (di altre variabili) che hanno contribuito a creare il conflitto. Nell'esempio, il conflict set di SA è $\{Q=R, NSW=G, V=B\}$.

Definizione formale di conflict set: sia A un assegnamento parziale consistente, sia X una variabile non ancora assegnata. Se $A \cup \{X=v_i\}$ risulta inconsistente per **ogni** valore v_i del dominio di X, allora A è un conflict set di X.

Un conflict set è **minimo** quando togliendo uno qualsiasi degli assegnamenti che lo compongono esso cessa di essere un conflict set (cioè ogni elemento del conflict set è davvero necessario a causare il conflitto). Quando si verifica un fallimento su una variabile, il **backjumping** usa il conflict set per decidere direttamente a quale variabile precedente saltare (nell'esempio: V), senza passare per le variabili intermedie estranee al conflitto (T).

Relazione fra backjumping e forward checking

Il forward checking (FC) può essere arricchito per costruire automaticamente i conflict set: quando una variabile viene assegnata e FC propaga la scelta eliminando valori dai domini di altre variabili (es. $V=B$ elimina B dal dominio di SA), basta registrare la relazione "chi ha causato l'eliminazione": $\text{conflict}(SA) \leftarrow \text{conflict}(SA) \cup \{V=B\}$.

Nota importante: il forward checking, se arricchito così, rileva **esattamente gli stessi conflitti** che rilevarebbe il backjumping puro. Quindi **usare backjumping insieme a forward checking è ridondante** — le due tecniche, combinate ingenuamente, non si sommano in efficacia perché coprono lo stesso tipo di informazione.

Assegnamenti NOGOOD e conflict-directed backjumping

NOGOOD

Analogia con i sistemi operativi: nel controllo del deadlock esistono stati “sicuri”, “insicuri” e di “blocco” — uno stato insicuro non è ancora un blocco, ma vi conduce necessariamente. Analogamente, un **assegnamento NOGOOD** in un CSP è un **assegnamento parziale** che, pur non essendo ancora un vicolo cieco immediato, **non può mai essere esteso a una soluzione** — lo si scopre solo esplorando ulteriormente.

Esempio (Australia): $\{WA=R, NSW=R\}$ è NOGOOD. Non è un vicolo cieco immediato (la ricerca può proseguire, ad esempio con $T=R$, senza problemi apparenti), e i conflict set di V , Q e NT conterranno separatamente o l’assegnamento di WA o quello di NSW , ma raramente entrambi assieme finché non si prova a chiudere il cerchio. Solo quando si tenta di assegnare NT , SA , Q o V (che devono essere B o G , ma essendo a loro volta confinanti fra loro esauriscono le combinazioni possibili con soli 2 colori residui) si scopre il fallimento — ma a quel punto non è ovvio “risalire” alla vera causa (la scelta iniziale $WA=R, NSW=R$).

Conflict-directed backjumping

Il backjumping “semplice” basato sui soli conflict set locali non risolve completamente il problema NOGOOD, perché il conflitto dipende sia dagli assegnamenti già fatti sia dalle variabili non ancora assegnate (e dalle loro reciproche relazioni). Il **conflict-directed backjumping** aggiorna dinamicamente i conflict set mano a mano che si esplora, propagando le informazioni scoperte “a ritroso” fra variabili, così da saltare direttamente ai veri responsabili del conflitto, anche se non direttamente confinanti con la variabile fallita.

Regola di aggiornamento: sia X_j la variabile corrente su cui tutti i valori falliscono. Si fa backjump alla variabile X_i aggiunta più di recente a $\text{conf}(X_j)$, aggiornando: $\text{conf}(X_i) \leftarrow \text{conf}(X_i) \cup \text{conf}(X_j) - \{X_i\}$.

Esempio numerico completo (Australia): assegnando nell’ordine $WA=R, NSW=R, T=B, NT=B, Q=G$, si arriva a dover assegnare SA , ma nessun valore funziona. I conflict set costruiti via FC durante il percorso sono:

```
conf(WA) = {}
conf(NSW) = {}
conf(T) = {}
conf(NT) = {WA}
conf(Q) = {NSW, NT}
conf(SA) = {WA, NSW, NT, Q}
```

- **Backjump a Q** (variabile aggiunta più di recente a $\text{conf}(SA)$): $\text{conf}(Q) \leftarrow \{NSW, NT\} \cup \{WA, NSW, NT, Q\} - \{Q\} = \{WA, NSW, NT\}$. Da notare: **WA non confina con Q**, ma viene comunque registrato nel suo conflict set perché ha contribuito indirettamente al conflitto — si scopre così un “vincolo implicito”. Si prova $Q=B, Q=R$: falliscono entrambi (arc consistency lo esclude), dominio di Q vuoto.
- **Backjump a NT:** $\text{conf}(NT) \leftarrow \{WA\} \cup \{WA, NSW, NT\} - \{NT\} = \{WA, NSW\}$. Si prova $NT=G$ (fallisce) e $NT=B$ (fallisce): dominio di NT vuoto.
- **Backjump a NSW:** $\text{conf}(NSW) \leftarrow \{\} \cup \{WA, NSW\} - \{NSW\} = \{WA\}$. Il metodo ha così **scoperto la relazione fra NSW e WA** (che in effetti sono confinanti). Si prova $NSW=G$: questa volta si riesce a costruire una soluzione completa.

Osservazione conclusiva: la ricerca era partita da uno stato NOGOOD, e l’ha scoperto solo esplorando; ma i vincoli hanno guidato la scoperta di relazioni implicite fra variabili. Registrare gli stati NOGOOD minimali permette all’algoritmo di evitare di ripetere in futuro gli stessi assegnamenti falliti: si tratta, a tutti gli effetti, di una **forma di apprendimento** (nearly identica nello spirito al clause learning dei moderni SAT solver).

Applicazioni ed esempi aggiuntivi

Sudoku come CSP

- **Variabili:** una per ogni casella x_{ij} (i = riga, j = colonna).

- **Dominio:** {1, ..., 9}; alcune variabili hanno un valore prefissato dalla griglia iniziale (i valori prefissati sono **vincoli unari**).
- **Vincoli** (tutti espressi con il vincolo globale alldifferent):
 - per ogni riga i: alldifferent(x_{i1} , ..., x_{i9})
 - per ogni colonna j: alldifferent(x_{1j} , ..., x_{9j})
 - per ogni riquadro 3×3: alldifferent delle 9 celle del riquadro.

N-regine su larga scala

Grazie a tecniche avanzate (in particolare il local search, cfr. min-conflicts, e formulazioni CSP efficienti), alcuni algoritmi moderni sono riusciti a risolvere il problema delle N regine con **N = 6.000.000**: un salto enorme rispetto ai pochi milioni di configurazioni testabili con generate-and-test già discusso per N=16.

Applicazioni reali di tipo ricerca operativa

- **Location of facilities:** dato un elenco di magazzini e richieste dei clienti, trovare un percorso che soddisfi le richieste minimizzando i costi.
- **Job scheduling:** sequenziare operazioni di macchinari per produrre diversi prodotti minimizzando il tempo totale di produzione.
- **Car sequencing:** nell'industria automobilistica, ordinare la produzione di auto con optional diversi senza eccedere la capacità delle aree di lavoro dedicate al montaggio degli optional.
- **Cutting stock problem:** tagliare materiale in pezzi più piccoli minimizzando lo spreco.
- **Vehicle routing:** trovare il percorso di costo minimo per rifornire n clienti da un unico deposito.
- **Timetabling:** costruzione automatica di orari (didattici, tenendo conto di aule/laboratori e altri vincoli).
- **Rostering / crew scheduling:** definizione di turni ed equipaggi (es. compagnie aeree). Nel 1998 CSP realizzati in ECLiPSe sono stati usati per definire gli equipaggi dei treni delle Ferrovie dello Stato italiane.

Software e strumenti per CSP

- **Linguaggi di programmazione a vincoli:** dichiarativi, spesso sviluppati come moduli di Prolog.
- **ECLiPSe** (eclipseclp.org): ambiente per programmazione a vincoli; usato ad esempio da Opel (premio VDA Logistics Award 2015, ottimizzazione della catena di distribuzione con Flexis AG).
- **CHR** (Constraint Handling Rules, Università di Leuven): disponibile per vari Prolog, Java, Haskell, C; applicato ad esempio alla navigazione robotica.
- **SAT solver:** risolvono CSP a variabili booleane; molti sono basati sull'algoritmo **DPLL** arricchito con risoluzione di conflitti, backjumping, propagazione, apprendimento di clausole. MiniSat, vincitore della competizione SAT 2005, è open source.
- **GEODE** (C++, con interfaccia grafica) e **GEODE/R** (binding Ruby): framework generali per la modellazione e risoluzione di CSP.
- **Answer Set Programming (ASP):** non nasce specificamente per i CSP, ma è uno strumento (concettuale e pratico) con cui si possono programmare e risolvere CSP (es. il problema della zebra), approfondito nel corso "Intelligenza Artificiale e Laboratorio" della magistrale.

? Domanda d'esame: qual è la differenza pratica fra forward checking, arc consistency (AC-3) e path consistency, e quando conviene usare l'una o l'altra? Le tre tecniche formano una gerarchia crescente di potenza (e di costo): il **forward checking** propaga solo dalla variabile appena assegnata ai suoi vicini diretti, aggiornandone i domini residui; è economico e si integra naturalmente con MRV, ma non rileva inconsistenze fra due vicini non ancora assegnati (es. NT e SA entrambi ridotti allo stesso unico valore, pur essendo confinanti). L'**arc consistency** (realizzata dall'algoritmo AC-3) propaga sistematicamente su tutti gli archi del grafo, non solo quelli toccati dall'ultimo assegnamento: quando riduce un dominio, rimette in coda tutti gli archi entranti nel nodo modificato, così l'informazione si propaga "a catena" su tutto il grafo. Costa $O(n^2d^3)$ contro il costo minore del forward checking, ma è più efficace; resta

comunque **incompleta** (un grafo può essere arc consistent e non avere comunque soluzione, come nell'esempio del triangolo a 2 colori). La **path consistency** è più forte ancora, perché ragiona su triplette di variabili e può scoprire l'assenza di soluzione anche quando l'arc consistency non ce la fa, ma il suo algoritmo (PC-2) ha complessità maggiore. In pratica: il forward checking si usa sempre "in linea" durante il backtracking per il suo basso costo; l'arc consistency (via AC-3 o l'algoritmo MAC che la integra nel backtracking) si usa quando serve una potatura più efficace e ci si può permettere il costo aggiuntivo; la path consistency (e la k-consistency in generale) restano soprattutto di interesse teorico, perché il loro costo cresce rapidamente e decidere il grado di consistenza di un CSP è esso stesso costoso.

Riepilogo e punti chiave

- Un **CSP** è definito da variabili, domini e vincoli; una soluzione è un assegnamento **completo e consistente**. I vincoli possono essere unari, binari (rappresentabili come grafo di vincoli) o n-ari (rappresentabili come ipergrafo, spesso riscrivibili in vincoli binari equivalenti); si distinguono inoltre vincoli rigidi da criteri di preferenza (soft).
- I CSP si formulano naturalmente come problema di ricerca nello spazio degli assegnamenti (stato iniziale = assegnamento vuoto, successori = estensione di una variabile, obiettivo = assegnamento completo). Grazie alla **commutatività** (l'ordine di assegnamento non influisce sul risultato) si può sempre fissare prima la variabile e poi il valore, riducendo il fattore di ramificazione da $n \cdot d$ a d per livello.
- Il **generate-and-test** è impraticabile su problemi anche piccoli (esplosione combinatoria: d^n). Il **backtracking** (depth-first + vincoli usati per potare) è già molto più efficiente, ma soffre di **thrashing** se non guidato da euristiche e inferenza.
- **Euristiche di scelta della variabile**: MRV (Minimum Remaining Values / fail-first) sceglie la variabile più vincolata, per scoprire prima i fallimenti; la **degree heuristic** (grado, numero di vincoli con variabili non assegnate) serve come tie-breaker, specie all'inizio della ricerca. **Euristica di scelta del valore**: Least Constraining Value (LCV) sceglie il valore che lascia più opzioni aperte ai vicini, per massimizzare la probabilità di successo del ramo.
- **Tecniche di inferenza/propagazione** (dalla più economica alla più potente): forward checking (propaga solo ai vicini diretti della variabile appena assegnata) → node consistency (vincoli unari) → arc consistency/AC-3 (vincoli binari, propagazione a catena su tutto il grafo, $O(n^2 d^3)$, ma incompleta) → path consistency/PC-2 (triplezze di variabili, più forte ma più costosa). Generalizzazione: **k-consistency** e **forte k-consistency** (un CSP fortemente n-consistent si risolve senza backtracking, ma verificarlo è esponenziale).
- **Vincoli globali** come Alldifferent e Atmost hanno algoritmi di propagazione dedicati, più efficienti del trattamento generico, e sono la base di molte modellazioni pratiche (es. Sudoku è interamente esprimibile con vincoli Alldifferent).
- Il **backtracking cronologico** torna sempre indietro alla variabile immediatamente precedente, anche se non è la causa del conflitto. Il **backjumping** usa i **conflict set** per saltare direttamente alla variabile realmente responsabile; se combinato con forward checking arricchito con il tracciamento dei conflitti, il backjumping diventa ridondante (FC da solo rileva già gli stessi conflitti). Il **conflict-directed backjumping** affina ulteriormente il meccanismo per gestire gli **assegnamenti NOGOOD** (assegnamenti parziali non estendibili a soluzione, scoperti solo esplorando), aggiornando dinamicamente i conflict set e scoprendo relazioni implicite fra variabili non direttamente connesse: è, di fatto, una forma di apprendimento.
- I CSP hanno applicazioni pratiche pervasive (scheduling, routing, timetabling, car sequencing, crew scheduling...) e un ecosistema di software dedicato (ECLiPSe, CHR, SAT solver come MiniSat, GECODE, ASP), a conferma che si tratta di un paradigma di modellazione, non solo di un esercizio accademico.

Modulo 5: Rappresentazione della conoscenza e Logica proposizionale

Perché rappresentare la conoscenza

Rappresentare la conoscenza significa dotarsi di un **linguaggio formale** in cui esprimere ciò che si sa del mondo, in modo tale da poter applicare a quella conoscenza dei **processi di ragionamento automatici (inferenze)**. Lo scopo dell'inferenza è duplice:

- derivare **nuova conoscenza**, cioè informazioni che erano implicite e diventano esplicite;
- **prendere decisioni**, in particolare decidere quale azione eseguire (nel caso di un agente).

Cosa vuol dire “ragionare”

Ragionare significa **rendere esplicita conoscenza che era implicita**. Esempio classico:

- So che *tutti i cetacei vivono in mare* e che *tutte le balene sono cetacei*.
- Posso concludere che *tutte le balene vivono in mare*, **senza aver verificato personalmente** ogni singola balena: mi basta **assumere vere** le due premesse e applicare uno schema di ragionamento valido.

Il punto cruciale è che questo tipo di ragionamento **lavora sulla forma delle affermazioni**, trattate come schemi astratti (Tutti gli A hanno la proprietà P; Tutti i B sono A $\implies$ Tutti i B hanno P), indipendentemente dal contenuto specifico. Questo è ciò che rende il ragionamento **automatizzabile**: basta

1. un **linguaggio** per strutturare le affermazioni in modo standard;
2. delle **regole di ragionamento** codificate (quando un'affermazione si può dire vera, quando una conclusione segue da altre);
3. un **algoritmo** che sappia applicare le regole alla conoscenza rappresentata.

Agenti basati sulla conoscenza

Un agente basato sulla conoscenza è caratterizzato da:

- **Knowledge Base (KB)**: insieme di formule espresse nel linguaggio di rappresentazione, possedute dall'agente. Può cambiare nel tempo. La conoscenza posseduta all'inizio è la **background knowledge**.
- **Tell (assert)**: meccanismo per aggiungere nuove formule alla KB.
- **Ask (query)**: meccanismo per interrogare la KB.

Sia ask che tell possono attivare processi di inferenza. Vincolo fondamentale: **ogni risposta a una ask deve essere conseguenza logica** delle tell fatte e della background knowledge. Esempio: se la KB contiene “quando piove la strada è bagnata” e faccio tell(piove), allora ask(strada bagnata?) deve rispondere Yes.

Schema generale dell'agente (KB-Agent):

Agente ha: KB

Tempo = 0

```
function KB-Agent(percezione) returns azione:
  1. tell(KB, costruisci-formulaP(percezione, tempo))
  2. azione ← ask(KB, costruisci-interrogazioneA(tempo))
  3. tell(KB, costruisci-formulaA(azione, tempo))
  4. tempo ← tempo + 1
  5. return azione
```

L'agente parte da una KB iniziale; ad ogni ciclo la percezione corrente viene tradotta in formula e aggiunta alla KB (tell), si interroga la KB per decidere l'azione (ask), si registra l'azione eseguita (tell), e si avanza il tempo (necessario per mantenere una "storia").

Una **versione 2** sostituisce la prima tell con modify (che può sia aggiungere sia **rimuovere** elementi, non solo accumulare fatti) e usa add per la seconda operazione. Esempio: se la KB contiene "il bicchiere è vuoto" e l'azione è "riempi il bicchiere", modify cancella quel fatto e aggiunge "bicchiere pieno" — necessario perché il mondo cambia, non solo si accumula.

La KB quindi evolve nel tempo: background knowledge + percezione(0) + azione(0) + percezione(1) + azione(1) + ...

Dato, informazione, conoscenza

Distinzione concettuale importante:

- **Dato:** il risultato grezzo della percezione sensoriale, privo di significato (es. un segno grafico).
- **Informazione:** ciò che il dato rappresenta (es. quel segno è la lettera "u" di un certo alfabeto) — in parte legata allo scopo per cui si percepisce.
- **Conoscenza:** cattura le **relazioni** fra le informazioni (es. le regole per comporre lettere in parole e frasi).

Esempio: bug algorithm

Un agente con conoscenza solo locale dell'ambiente, capace di procedere in linea retta e, incontrato un ostacolo, di applicare **wall following** (segue il muro tenendosi parallelo, dopo aver scelto una direzione di rotazione). Percezioni ∈ {ostacolo, libero, allineato, arrivato}, azioni ∈ {avanza, ruota}. Mostra concretamente come la KB si accresce ciclo dopo ciclo con i tell di percezioni e azioni.

Programmazione dichiarativa

Programmare un agente basato sulla conoscenza significa **specificare la KB** (che include anche la specifica delle azioni disponibili, precondizioni ed effetti) **in forma dichiarativa**: si dice *cosa* è vero/cosa fare, non *come* calcolarlo proceduralmente. L'agente è dotato di meccanismi generali di ask/tell validi per qualsiasi KB — è un paradigma radicalmente diverso da quello procedurale, dove tutto è codificato da procedure specifiche del dominio.

Esempi di programmazione dichiarativa: XML (descrive la struttura di un documento, non come renderizzarlo), una query SQL (dice quali dati estrarre, non come attraversare le tabelle), un'espressione regolare (descrive una forma, non l'algoritmo di ricerca).

Esempio: il mondo dei blocchi

Blocchi su un tavolo, impilabili uno sull'altro, un blocco alla volta. Predicati: holding(x), handempty, clear(x), ontable(x), on(x,y). Le azioni sono descritte dichiarativamente con **precondizioni** ed **effetti** (add/delete), ad esempio l'azione pick(x,y):


```

pick(x, y)
Preconditions: on(x,y)  $\wedge$  clear(x)  $\wedge$  handempty
Effects:
  add(clear(y))
  add(holding(x))
  delete(on(x,y))
  delete(clear(x))
  delete(handempty)

```

Qui **add** corrisponde a **tell**, **delete** rimuove elementi dalla descrizione dello stato (è l'operazione "opposta" al **tell**). La **background knowledge** è la descrizione delle azioni, la **percezione** è lo stato iniziale, e il **metodo** è l'applicazione di un criterio generale (non specifico del dominio) per determinare l'azione successiva — questo è esattamente lo spirito della programmazione dichiarativa applicata alla pianificazione.

Rappresentare la conoscenza tramite la logica

Un **linguaggio di rappresentazione** è lo strumento che permette di codificare la conoscenza in una forma su cui si può applicare un ragionamento automatico. Una formula che rispetta le regole grammaticali del linguaggio è **ben formata**. La **semantica** del linguaggio definisce la verità delle formule rispetto a un mondo possibile (es. $\text{on}(E,A) \wedge \text{clear}(B)$ può essere FALSA in un certo stato, mentre $\text{clear}(E) \vee \text{clear}(C)$ VERA).

I concetti chiave della logica come strumento di rappresentazione sono: **modello**, **conseguenza**, **inferenza**, **algoritmo di inferenza**, **grounding**.

Modello

Un **modello** è un "mondo possibile": fissa il valore di verità di tutte le formule assegnando valori agli elementi da cui quei valori dipendono. Esempio: per la formula $x + y = 4$, è vera nei modelli $x=1,y=3$ o $x=2,y=2$ ecc., falsa in $x=0,y=0$ ecc. Quando il dominio è la realtà fisica, il modello è un'**astrazione matematica/simbolica** significativa di quella realtà.

Definizione formale: dati un modello m e una formula α , **m è un modello di α se α è vera in m** . Si indica con **$M(\alpha)$** l'insieme di tutti i modelli di α .

Conseguenza logica (entailment, $\models$)

$A \models B$ ("A implica logicamente B", "B è conseguenza logica di A") significa: **in tutti i modelli in cui A è vera, è vera anche B**. Esempio: $(x+y=4) \models (x+y<5)$.

Attenzione alla **direzionalità**: $A \not\models B$ (B non è conseguenza di A) significa solo che *esiste almeno un modello* in cui A è vera e B è falsa — non che B sia sempre falsa quando A è vera. Esempio: $(x+y=4) \not\models (x<3)$, perché nel modello $x=4,y=0$ la prima è vera e la seconda falsa (basta un controesempio, anche se in altri modelli entrambe potrebbero essere vere).

Visione insiemistica: $A \models B$ corrisponde all'inclusione insiemistica **$M(A) \subseteq M(B)$** — l'insieme dei modelli che rendono vera A è un sottoinsieme di quelli che rendono vera B.

Equivalenza

$A \equiv B$ se e solo se $A \models B$ e $B \models A$, cioè le due formule sono vere esattamente negli stessi modelli: **$M(A) = M(B)$** .

Validità, insoddisfacibilità, soddisfacibilità

- **Validità (tautologia):** una formula P è **valida** se è vera in **tutti** i modelli ($P \equiv \text{True}$). Esempio: $Q \vee \neg Q$ è una tautologia.
- **Insoddisfacibilità (contraddizione):** una formula P è **insoddisfacibile** se è **falsa** in tutti i modelli ($P \equiv \text{False}$). Esempio: $Q \wedge \neg Q$.
- **Soddisfacibilità:** una formula P è **soddisfacibile** se esiste **almeno un** modello in cui è vera. Se m soddisfa P si dice che *m è un modello di P* . (Applicazione pratica: nei CSP si cerca proprio un modello, cioè un assegnamento che soddisfi tutti i vincoli.)

Relazione importante: **P è valida se e solo se $\neg P$ è insoddisfacibile** (e viceversa, per dualità).

Inferenza

Inferenza (dal latino *in ferre*, “portare dentro”): processo con cui, da una proposizione accettata come vera, si passa a una seconda proposizione la cui verità è derivata dalla prima. Punto essenziale: **l’inferenza è sintattica**, lavora sulla struttura delle formule secondo le regole del linguaggio, non sul loro “significato”.

Esempio: da “tutti gli uomini sono mortali” e “Socrate è un uomo” si inferisce “Socrate è mortale”, applicando lo schema generale $\text{uomo}(X) \Rightarrow \text{mortale}(X)$, $\text{uomo}(X) \vdash \text{mortale}(X)$.

Regole di inferenza principali:

- **Modus Ponens:** da $A \Rightarrow B$ e A si deriva B . È il fondamento del ragionamento deduttivo (“Se piove, la strada è bagnata. Piove. $\Rightarrow$ La strada è bagnata”).
- **Eliminazione della congiunzione:** da $A \wedge B$ si deriva B (o A).

Formalizzazione: sia i un algoritmo di inferenza. $KB \vdash_i A$ significa che A è derivabile da KB usando l’algoritmo i , cioè esiste una sequenza di passi che da KB porta ad A . Si dice anche: “ A segue da KB ”, “ A può essere inferito da KB ”, “esiste una dimostrazione di A a partire da KB ”.

Proprietà desiderate di un algoritmo di inferenza:

- **Correttezza (soundness):** l’algoritmo deriva *solo* conseguenze logiche vere — se $KB \vdash_i A$ allora $KB \models A$. Non produce mai conclusioni sbagliate.
- **Completezza (completeness):** l’algoritmo è in grado di derivare *tutte* le conseguenze logiche — se $KB \models A$ allora $KB \vdash_i A$. Non “perde” conclusioni valide.

Le logiche devono **sempre** garantire la correttezza (altrimenti le inferenze non corrispondono a reali conseguenze nel mondo); non sempre garantiscono la completezza.

Piano dell’astrazione vs piano della realtà: le formule sono astrazioni usate per ragionare sul mondo; la semantica collega le formule al mondo reale dando loro significato; i modelli catturano il mondo reale insieme alla relazione di conseguenza.

Grounding

Il **grounding** cattura il legame fra la rappresentazione simbolica/formale e l’ambiente reale che essa rappresenta — possiamo pensarlo come derivante dalla percezione. Esempio: so che “quando piove le strade sono bagnate”; vedo (percepisco) che piove, e quindi concludo che nel mondo reale le strade sono bagnate. Il grounding è ciò che rende la logica utile per ragionare sul mondo reale e non solo su simboli astratti.

Logica proposizionale

È uno dei tipi di logica più semplici: **le formule non contengono variabili**. Si articola in: sintassi, semantica, inferenza, e i concetti di equivalenza/validità/soddisfacibilità già visti in generale.

Sintassi

Formule atomiche: - i **simboli proposizionali** rappresentano ciascuno una formula elementare che può essere vera o falsa (per convenzione hanno un nome che inizia con la maiuscola, es. Piove); - True e False sono formule atomiche speciali.

Formule complesse, costruite componendo formule tramite operatori (connettivi): - **Negazione** ($\neg$): un **letterale** è una formula atomica, eventualmente negata; - **Congiunzione** ($\wedge$): le formule combinate sono dette **congiunti**; - **Disgiunzione** ($\vee$): le formule combinate sono dette **disgiunti**; - **Implicazione** ($\Rightarrow$): collega una **premessa/antecedente** a una **conclusione/consequente**; - **Biimplicazione/equivalenza** ($\Leftrightarrow$).

Grammatica formale:

```
formula      → formulaAtomica | formulaComplessa
formulaAtomica → True | False | simbolo
simbolo      → P | Q | R | ...
formulaComplessa → ¬ formula                (negazione)
                  | (formula ∧ formula)      (congiunzione)
                  | (formula ∨ formula)      (disgiunzione)
                  | (formula ⇒ formula)      (implicazione)
                  | (formula ⇔ formula)      (equivalenza)
```

Precedenza degli operatori (dal più forte/legante al più debole): $\neg$, $\wedge$, $\vee$, $\Rightarrow$, $\Leftrightarrow$. Esempio: $\neg Q \vee P$ equivale a $((\neg Q) \vee P)$.

? Domanda d'esame: "Vero o falso? Si consideri la formula $\text{Piove} \wedge \text{Vento}$: è vera o falsa?" **Risposta ragionata:** la domanda è mal posta finché non si fissa un **modello**. Il valore di verità di una formula proposizionale non è assoluto: dipende dall'assegnamento di verità ai simboli atomici che la compongono. $\text{Piove} \wedge \text{Vento}$ è vera **solo** nel modello in cui sia Piove sia Vento sono vere (T,T); è falsa in tutti gli altri tre casi (T,F), (F,T), (F,F), perché la congiunzione richiede che *entrambi* i congiunti siano veri. Questo è il punto pedagogico della slide: senza specificare il modello di riferimento, non si può rispondere "vera" o "falsa" in assoluto.

Semantica

La semantica definisce le regole per calcolare il valore di verità di ogni formula. Un **modello**, in logica proposizionale, è un **assegnamento di un valore di verità a ciascun simbolo proposizionale**. Con **N simboli proposizionali** ci sono **2^N modelli possibili**. La semantica è definita **ricorsivamente**:

Formule atomiche: - True è sempre vera, False è sempre falsa in ogni modello; - il valore di verità di ogni altro simbolo va specificato esplicitamente dal modello.

Formule complesse (P, Q formule qualsiasi): - $\neg Q$ è vera **se e solo se** Q è falsa; - $Q \wedge P$ è vera **se e solo se** sia P sia Q sono vere; - $Q \vee P$ è vera **se e solo se** almeno una fra P e Q è vera; - $Q \Rightarrow P$ è sempre vera, **tranne** quando Q è vera e P è falsa; - $Q \Leftrightarrow P$ è vera **se e solo se** P e Q hanno lo **stesso** valore di verità.

Tabella di verità dei connettivi:

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
F	F	T	F	F	T	T
F	T	T	F	T	T	F

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
T	F	F	F	T	F	F
T	T	F	T	T	T	T

L'implicazione: come interpretarla

$P \Rightarrow Q$ è **equivalente a** $\neg P \vee Q$. Va letta così:

- se P è **falsa**, non ci si preoccupa del valore di Q: l'implicazione è comunque **vera** (il conseguente non deve necessariamente essere vero, l'affermazione è "vacuamente" soddisfatta);
- se P è **vera**, allora Q deve **necessariamente** essere vera perché l'implicazione sia vera.

In sintesi: $P \Rightarrow Q$ serve a catturare le situazioni in cui **ogni volta che è vero l'antecedente, è vero anche il conseguente**; quando l'antecedente è falso, l'implicazione non dice nulla e viene considerata vera per convenzione.

Attenzione: l'implicazione logica **non è una relazione causale**. È vera o falsa solo in base ai valori di verità di P e Q, indipendentemente dal legame di significato/causa fra le due proposizioni. Esistono altri tipi di implicazione legati al significato delle parole (non trattati come implicazione logica): - "Fido è un cane $\Rightarrow$ Fido è un mammifero" (ragionamento **ontologico**); - "John ha vinto la partita $\Rightarrow$ John ha giocato la partita" (ragionamento **temporale**); - "John è stato condannato per furto $\Rightarrow$ il furto è un crimine" (ragionamento **causale**).

? Domanda d'esame: "È vera l'implicazione Torino è in Lombardia $\Rightarrow$ Giulio Cesare governò Roma?"

Risposta ragionata: sì, è **vera**. Sembra assurda perché intuitivamente attribuiamo all'implicazione un legame causale/di significato, ma l'implicazione logica dipende **solo** dai valori di verità: Torino è in Lombardia è **falsa** (Torino è in Piemonte), e quando l'antecedente è falso $P \Rightarrow Q$ è vera **indipendentemente** dal valore di verità di Q. Questo è l'esempio didattico standard per mostrare che l'implicazione logica non coincide con la nozione intuitiva di causa-effetto.

Biimplicazione

$P \Leftrightarrow Q$ si legge "P se e solo se Q" ed è vera esattamente quando P e Q hanno **lo stesso** valore di verità (entrambe vere o entrambe false).

Verificare l'entailment: $KB \models P$?

Esistono due approcci:

1. **Model checking:** enumerare tutti i possibili modelli, selezionare quelli in cui KB è vera, e verificare che in *tutti* questi P sia vera. È **costoso**: con N simboli proposizionali ci sono 2^N modelli — crescita esponenziale.
2. **Theorem proving:** usare regole di inferenza per cercare una derivazione **senza costruire esplicitamente i modelli**. Più efficiente perché ignora le proposizioni irrilevanti alla dimostrazione (che possono essere numerose).

Il theorem proving si basa su due risultati fondamentali:

Teorema di deduzione: date due formule R e Q, **($R \models Q$) se e solo se ($R \Rightarrow Q$) è valida** (cioè è una tautologia). Quindi per verificare $KB \models P$ si può: - dimostrare che $KB \Rightarrow P$ è vera in ogni modello (enumerazione, costoso), oppure - dimostrare **per inferenza sintattica** (manipolando la forma delle formule) che $(KB \Rightarrow P) \equiv \text{True}$.

Dimostrazione per refutazione: sfruttando che validità e soddisfacibilità sono legate dalla negazione (A è valida sse $\neg A$ è insoddisfacibile), e che $(R \Rightarrow Q) \equiv (\neg R \vee Q)$, negando si ottiene $\neg(\neg R \vee Q) \equiv (R \wedge \neg Q)$ (De Morgan). Si arriva quindi al risultato: **date R e Q, ($R \models Q$) se e solo se ($R \wedge \neg Q$) è insoddisfacibile**.

Questo corrisponde esattamente a una **dimostrazione per assurdo/contraddizione**: per verificare $KB \models P$, 1. si assume **per assurdo** $\neg P$; 2. si dimostra che $KB \wedge \neg P$ è insoddisfacibile, cioè che partendo da $KB \wedge \neg P$ si deriva False (una contraddizione); 3. la ricerca della dimostrazione è formalmente analoga a una ricerca in uno spazio degli stati (stato iniziale = background knowledge, azioni = regole di inferenza, goal = stato che contiene la formula da dimostrare — in questo caso False).

Il ragionamento si applica a **logiche monotone**: se $KB \models P$ allora anche $KB \wedge Q \models P$ — aggiungere informazione non invalida mai le conclusioni già derivate, l'insieme delle conseguenze può solo crescere.

Regole di inferenza (equivalenze logiche)

Oltre a modus ponens ed eliminazione della congiunzione, altre regole derivano da equivalenze logiche standard:

Equivalenza	Nome
$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$	Commutatività $\vee$
$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$	Commutatività $\wedge$
$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$	Associatività $\wedge$
$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$	Associatività $\vee$
$\neg(\neg\alpha) \equiv \alpha$	Eliminazione doppia negazione
$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha)$	Contrapposizione
$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$	Eliminazione implicazione
$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$	De Morgan
$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$	De Morgan
$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$	Distributività
$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$	Distributività
$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$	Eliminazione bicondizionale

Correttezza e completezza: la dimostrazione avviene combinando (1) un algoritmo di inferenza e (2) un insieme di regole di inferenza. L'insieme completo delle regole viste è **corretto** (deriva solo conseguenze vere) e **completo** (deriva tutte le conseguenze logiche). Se si usa solo un **sottoinsieme** delle regole, si rischia di perdere completezza: ad esempio, senza la regola del doppio negato non si può derivare $P \vdash \neg\neg P$.

La regola di risoluzione (Resolution)

La **risoluzione** è una singola regola di inferenza che, combinata con un algoritmo di ricerca completo, produce un algoritmo di inferenza **corretto e completo**: se $KB \vdash P$ allora $KB \models P$ (correttezza) e se $KB \models P$ allora $KB \vdash P$ (completezza). Permette di realizzare **dimostrazioni per refutazione** sia in logica proposizionale sia in logica del prim'ordine.

Si applica a **clausole**, cioè disgiunzioni di letterali. Date due clausole che contengono un **letterale complementare** P_i e Q_j (uno negazione dell'altro):

$$(P_1 \vee \dots \vee P_i \vee \dots \vee P_n) \quad (Q_1 \vee \dots \vee Q_j \vee \dots \vee Q_m)$$

$$(P_1 \vee \dots \vee P_{i-1} \vee P_{i+1} \vee \dots \vee P_n \vee Q_1 \vee \dots \vee Q_{j-1} \vee Q_{j+1} \vee \dots \vee Q_m)$$

La clausola risultante (**resolvent**) contiene tutti i letterali delle due clausole originali **tranne** la coppia complementare; se un letterale compare più volte viene **fattorizzato** (mantenuto una sola volta): es. $A \vee \neg A$ diventa A .

Esempio: da $A \vee B \vee \neg C$ e $C \vee A$ (letterali complementari $\neg C$ e C), si ottiene $A \vee B \vee A$, che fattorizzato diventa $A \vee B$.

Relazione con il modus ponens: il modus ponens è un **caso particolare** di risoluzione. Si evidenzia eliminando l'implicazione: da C e $C \Rightarrow A$ (cioè $\neg C \vee A$), risolvendo su $C/\neg C$ si ottiene A — esattamente il risultato del modus ponens.

Dall'agente alla necessità della forma clausale

Nello schema dell'agente basato sulla conoscenza, la *ask* attiva un processo di inferenza in cui **la query, negata**, viene aggiunta alla KB e, applicando iterativamente la risoluzione, si cerca di derivare la clausola vuota (contraddizione), confermando così l'entailment.

Prerequisito: la risoluzione si applica a clausole, quindi la KB proposizionale va prima **tradotta in CNF** (Conjunctive Normal Form).

Forma Normale Congiuntiva (CNF)

CNF (Conjunctive Normal Form): data una qualunque formula proposizionale, esiste sempre una **coniunzione di clausole** ad essa equivalente.

Grammatica delle clausole:

```
CNFsentence → Clause ∧ ... ∧ Clause
Clause      → Literal ∨ ... ∨ Literal
Literal     → Symbol | ¬Symbol
Symbol      → P | Q | ...
```

Algoritmo di traduzione in CNF (4 passi):

1. **Eliminare la biimplicazione:** $(\alpha \Leftrightarrow \beta)$ diventa $((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$.
2. **Eliminare l'implicazione:** $(\alpha \Rightarrow \beta)$ diventa $(\neg \alpha \vee \beta)$.
3. **Portare la negazione all'interno** applicando De Morgan ed eliminando le doppie negazioni: $\neg(\alpha \wedge \beta) \rightarrow (\neg \alpha \vee \neg \beta)$; $\neg(\alpha \vee \beta) \rightarrow (\neg \alpha \wedge \neg \beta)$; $\neg \neg \alpha \rightarrow \alpha$.
4. **Distribuire l'∨ sull'∧** dove necessario: $(\alpha \vee (\beta \wedge \gamma)) \rightarrow ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$.

Il risultato finale è una congiunzione di clausole, ciascuna disgiunzione di letterali — la forma richiesta dalla risoluzione.

Esempio completo: dalle formule alle clausole

Partendo da una base di conoscenza proposizionale su pioggia/atmosfera/strada, tradotta in clausole:

```
C1) ¬Piove ∨ Atmosfera_umida
C2) ¬Notte ∨ Vento ∨ Atmosfera_umida
C3a) ¬Atmosfera_umida ∨ Prato_bagnato
C3b) ¬Atmosfera_umida ∨ Strada_Bagnata
C4) ¬Innaffiatore_on ∨ Prato_bagnato
C5) ¬Piove ∨ Ombrello_aperto
C6) ¬Sole ∨ ¬Vento ∨ Innaffiatore_on
C7) ¬Sole ∨ ¬Vento ∨ Atmosfera_asciutta
C8) ¬Sole ∨ ¬Notte
C10) ¬Atmosfera_asciutta ∨ ¬Atmosfera_umida
```

(Osservazione: due regole diverse della KB originale possono tradursi nella *stessa* clausola, come accade per C8.)

Algoritmo di risoluzione proposizionale

```
function CP-RISOLUZIONE(KB, A) returns true | false
```

Input: KB (base di conoscenza), A (query, formula proposizionale)

```

clausole ← clausole della CNF di  $(KB \wedge \neg A)$ 
new ← { }

loop do:
  for each coppia  $C_i, C_j$  in clausole do:
    resolvents ← IR-RISOLUZIONE( $C_i, C_j$ )
    if resolvents contiene la clausola vuota:
      return true
    new ← new  $\cup$  resolvents
  if new  $\subseteq$  clausole:
    return false
  clausole ← clausole  $\cup$  new

```

Il funzionamento è quello di una **dimostrazione per refutazione**: si nega la query, la si aggiunge alla KB in CNF, e si cerca — risolvendo iterativamente tutte le coppie di clausole — di generare la **clausola vuota** (che rappresenta False, cioè una contraddizione). Se la si trova, $KB \models A$ è confermato (return true). Se non si generano più nuove clausole (new è già incluso in clausole, punto fisso raggiunto) senza aver trovato la clausola vuota, allora $KB \not\models A$ (return false).

Teorema di completezza della risoluzione (non dimostrato a lezione): se un insieme di clausole è insoddisfacibile, la **chiusura della risoluzione** (l'insieme di tutte le clausole derivabili) contiene la clausola vuota.

Esempio applicativo: pioggia, atmosfera, strada

Data la KB precedente, si aggiungono i fatti percepiti F1) Sole e F2) vento. Si vuole verificare:

$KB \models \neg \text{Piove}$?

Si usa la refutazione: si nega il goal ottenendo la clausola **GN) Piove**, la si aggiunge alla KB in CNF, e si cerca di derivare la clausola vuota:

1. **GN) Piove** risolve con **C1) $\neg \text{Piove} \vee \text{Atmosfera_umida} \rightarrow \text{C21) } \text{Atmosfera_umida}$**
2. **C21)** risolve con **C10) $\neg \text{Atmosfera_asciutta} \vee \neg \text{Atmosfera_umida} \rightarrow \text{C22) } \neg \text{Atmosfera_asciutta}$**
3. **C22)** risolve con **C7) $\neg \text{Sole} \vee \neg \text{Vento} \vee \text{Atmosfera_asciutta} \rightarrow \text{C23) } \neg \text{Sole} \vee \neg \text{Vento}$**
4. **C23)** risolve con **F1) Sole $\rightarrow \text{C24) } \neg \text{Vento}$**
5. **C24)** risolve con **F2) Vento $\rightarrow$ clausola vuota**

Si è generata la clausola vuota: la KB (con Piove aggiunto) è insoddisfacibile, quindi per refutazione **$KB \models \neg \text{Piove}$** è confermato: la risposta è **true**.

Clausole di Horn, Forward Chaining, Backward Chaining

In molti contesti pratici la conoscenza si presenta in una forma molto specifica — le **clausole di Horn** — per cui sono stati sviluppati meccanismi di inferenza dedicati, molto più efficienti della risoluzione generale.

Clausole di Horn

Definizione: una clausola di Horn è una disgiunzione di letterali in cui **al più uno** è positivo (non negato).

- Se la clausola contiene **esattamente un** letterale positivo, si dice **clausola definita**.

- Esempi di clausole di Horn: $\neg B \vee C$, $\neg A \vee \neg B \vee C$ (queste due sono anche clausole definite, avendo esattamente un positivo), $\neg A \vee \neg B$ (nessun positivo, non è definita ma è comunque di Horn).

Le clausole di Horn **catturano implicazioni** in cui l'antecedente è una congiunzione di letterali positivi e il conseguente è un singolo letterale positivo: $\neg B \vee C$ equivale a $B \Rightarrow C$; $\neg A \vee \neg B \vee C$ equivale a $A \wedge B \Rightarrow C$. Sono la **base della programmazione logica** (es. Prolog).

Perché sono importanti: su clausole di Horn si possono applicare meccanismi di inferenza **molto naturali** per il ragionamento umano, e soprattutto permettono di verificare la conseguenza logica in un **tempo lineare** rispetto alla dimensione della KB — mentre la risoluzione generale non ha questa garanzia di efficienza. Questo rende l'inferenza proposizionale su clausole di Horn **computazionalmente economica**.

Forward Chaining (concatenazione in avanti)

Permette di derivare una query costituita da un **singolo simbolo proposizionale**, a partire da una KB di sole clausole di Horn. È un procedimento **iterativo, guidato dai dati (data-driven)**:

- Si parte dai **fatti noti** (letterali già veri).
- Si applica **modus ponens**: da $F \Rightarrow Q$ e F (con F verificata) si deriva Q .
- Ogni volta che **tutte le premesse** di un'implicazione risultano vere, si **aggiunge** il letterale conseguente all'insieme dei fatti noti.
- Terminazione**: se si deriva la query cercata $\rightarrow$ return true; se ad un certo punto non si possono fare altre inferenze senza aver derivato la query $\rightarrow$ return false.

Ha **complessità lineare** nella dimensione della KB.

Esempio (rappresentazione a grafo AND-OR)

KB:

$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$

Fatti: A, B. Obiettivo: dimostrare Q.

Nel grafo AND-OR gli archi uniti da un "archetto" rappresentano letterali in AND (tutte le premesse di una stessa regola), mentre frecce distinte verso lo stesso nodo rappresentano OR (regole diverse con la stessa conclusione — un arco OR si attiva se almeno un disgiunto/regola è vero). Partendo dai fatti A e B, l'attivazione si propaga: $A \wedge B$ attivano L (tramite $A \wedge B \Rightarrow L$); con L noto e B noto si attiva M (tramite $B \wedge L \Rightarrow M$); con L e M noti si attiva P (tramite $L \wedge M \Rightarrow P$); infine con P noto si attiva Q (tramite $P \Rightarrow Q$). Tutti questi letterali risultano quindi veri.

Osservazioni sul forward chaining:

- Complessità lineare.**
- Completo:** deriva tutte le formule atomiche dimostrabili a partire dalla KB.
- "Inconscio" (data-driven):** è guidato solo dai dati disponibili, senza usare l'informazione sul goal che si vuole dimostrare — non sa dove sta andando.
- Adatto a problemi come il **riconoscimento di oggetti** (si parte da osservazioni per arrivare a conclusioni).
- Svantaggio: può attivare **molte inferenze inutili**, irrilevanti ai fini della query specifica.

Backward Chaining (concatenazione all'indietro)

Parte dalla formula da dimostrare (il **goal**):

- se il goal è **già vero** (è un fatto noto), termina restituendo `true`;
- altrimenti cerca clausole di Horn di cui il goal è la **conclusione**, e cerca ricorsivamente di dimostrarne le **premesse**, usando anche i fatti noti come base.

È quindi un ragionamento **goal-driven** (guidato dagli obiettivi), opposto in direzione al forward chaining.

Esempio 1

- R1) $A \wedge C \Rightarrow F1$
 R2) $B \wedge F1 \Rightarrow F2$

Si vuole dimostrare $F2$, dati i fatti A, B, C . $F2$ non è un fatto noto, ma esiste R2 la cui conclusione è $F2$: bisogna dimostrare B (fatto noto, verificato subito) e $F1$. $F1$ non è un fatto noto, ma esiste R1 la cui conclusione è $F1$: bisogna dimostrare A e C , entrambi fatti noti. Quindi $F1$ è vera, e di conseguenza $F2$ è vera. I **valori di verità si propagano dal basso verso l'alto** (come valori di ritorno di una ricorsione).

Esempio 2 (stessa KB del forward chaining)

Con la stessa KB ($P \Rightarrow Q, L \wedge M \Rightarrow P, B \wedge L \Rightarrow M, A \wedge P \Rightarrow L, A \wedge B \Rightarrow L$) e fatti A, B , si vuole dimostrare Q :

- Q è vera se P è vera (regola $P \Rightarrow Q$);
- P è vera se L e M sono vere (regola $L \wedge M \Rightarrow P$);
- M è vera se L e B sono vere: B è un fatto noto, L va dimostrata;
- L è vera se A e B sono vere (regola $A \wedge B \Rightarrow L$): entrambi sono fatti noti! Oppure, alternativamente, se A e P sono vere (regola $A \wedge P \Rightarrow L$), ma di P non si conosce ancora il valore — non importa, **basta che una delle due alternative (regole con conclusione L) sia vera** per concludere che L è vera;
- una volta stabilito che L è vera, anche M risulta vera (avendo B e L); quindi P risulta vera (avendo L e M); infine Q risulta vera.

Nella ricerca si presta attenzione a **evitare loop** (nell'esempio, il tentativo di dimostrare L passando per P che a sua volta dipende da L viene "ignorato" una volta che L è già stata dimostrata per un'altra via) e a **non ridimostrare** un sottogol già dimostrato in precedenza.

Osservazioni sul backward chaining:

- Realizza un ragionamento **guidato dagli obiettivi (goal-driven)**.
- È usato nel **theorem proving** e nella **programmazione logica** (es. Prolog) come meccanismo di inferenza principale.
- È spesso **più efficiente** del forward chaining perché l'uso esplicito del goal **focalizza** la ricerca solo sulle regole rilevanti, evitando di derivare fatti inutili.
- La complessità temporale è **meno che lineare** (nella pratica, grazie al focus sul goal).

Forward vs Backward chaining: confronto

Aspetto	Forward Chaining	Backward Chaining
Direzione Guida	dai fatti verso le conclusioni <i>data-driven</i> (guidato dai dati)	dal goal verso le premesse <i>goal-driven</i> (guidato dagli obiettivi)
Completezza	completo (deriva tutto ciò che è dimostrabile)	dimostra solo ciò che serve al goal
Efficienza	lineare, ma può fare inferenze inutili	spesso più efficiente, focalizzato
Uso tipico	riconoscimento di pattern/oggetti	theorem proving, programmazione logica (Prolog)

Riepilogo e punti chiave

- **Rappresentare la conoscenza** significa dotarsi di un linguaggio formale su cui applicare **inferenza automatica**: da conoscenza esplicita (KB) si derivano conclusioni implicite. Un agente basato sulla conoscenza è definito da una **KB**, dalle operazioni **tell** (aggiungere formule) e **ask** (interrogare), con il vincolo che ogni risposta a una ask sia conseguenza logica di quanto asserito.
- I concetti fondamentali della logica come strumento sono: **modello** (mondo possibile che assegna valori di verità), **conseguenza logica** $\models$ ($A \models B$: B vera in tutti i modelli in cui A è vera), **equivalenza** $\equiv$, **validità/tautologia** (vera in ogni modello), **insoddisfacibilità/contraddizione** (falsa in ogni modello), **soddisfacibilità** (vera in almeno un modello), e **inferenza** (processo sintattico che deriva nuove formule).
- Un buon algoritmo di inferenza deve essere **corretto** (soundness: deriva solo conseguenze vere) e idealmente **completo** (completeness: deriva tutte le conseguenze vere).
- La **logica proposizionale** ha sintassi basata su simboli atomici e connettivi ($\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$), e semantica definita ricorsivamente tramite tabelle di verità; con N simboli si hanno 2^N modelli. L'**implicazione** $P \Rightarrow Q$ equivale a $\neg P \vee Q$ ed è vera sempre tranne quando P è vera e Q falsa — **non è una relazione causale**.
- Per verificare $KB \models P$ si può fare model checking (costoso, esponenziale) oppure **theorem proving**, basato sul teorema di deduzione o, più praticamente, sulla **dimostrazione per refutazione**: si nega il goal, lo si aggiunge alla KB, e si dimostra che l'insieme è insoddisfacibile.
- La **CNF** (forma normale congiuntiva: congiunzione di clausole, ciascuna disgiunzione di letterali) si ottiene eliminando biimplicazione e implicazione, portando le negazioni all'interno (De Morgan), e distribuendo $\vee$ su $\wedge$. È il prerequisito per applicare la **risoluzione**.
- La **regola di risoluzione** combina due clausole con letterali complementari producendo un resolvent; combinata con un algoritmo di ricerca completo è **corretta e completa**. Il **modus ponens è un caso particolare** di risoluzione. L'algoritmo CP-RISOLUZIONE cerca di derivare la **clausola vuota** da $(KB \wedge \neg \text{query})$ in CNF: se ci riesce, $KB \models \text{query}$.
- Le **clausole di Horn** (al più un letterale positivo) permettono inferenza in **tempo lineare**, molto più efficiente della risoluzione generale, e sono alla base della programmazione logica.
- **Forward chaining**: parte dai fatti, applica modus ponens iterativamente, è **data-driven**, completo, lineare, ma può fare inferenze irrilevanti.
- **Backward chaining**: parte dal goal, cerca le regole che lo concludono e ne dimostra ricorsivamente le premesse, è **goal-driven**, generalmente più efficiente perché focalizzato, usato in Prolog e nel theorem proving.

Modulo 6: Logica del primo ordine (FOL)

“Rappresentiamo mondi complessi in cui gli oggetti sono in relazione gli uni con gli altri e studiamo come ragionare su queste rappresentazioni più espressive.” (C. Baroglio)

Perché non basta la logica proposizionale

Pregi della logica proposizionale (da mantenere)

La logica proposizionale è un buon punto di partenza perché ha tre proprietà preziose che **vogliamo conservare** anche nel passaggio a una logica più potente:

- **È dichiarativa:** separa nettamente la conoscenza (le formule) dall’inferenza (l’algoritmo che le manipola); consente di derivare fatti da fatti sfruttando una semantica basata su una relazione di verità fra formule e mondi possibili.
- **È composizionale:** il valore di verità di una formula si ottiene componendo i valori di verità delle sue parti (es. il valore di $A \wedge B$ dipende solo dai valori di A e di B).
- **Non è ambigua:** ogni formula ha un significato preciso, indipendente dal contesto.

I limiti espressivi

Il problema della logica proposizionale è la **mancanza di espressività**:

1. **Non permette rappresentazioni compatte.** Per dire che “una persona quando è felice ascolta musica” dobbiamo scrivere una proposizione *per ogni singola persona*:

```
ascoltaMusica(mia)  $\Leftarrow$  felice(mia)
ascoltaMusica(jody)  $\Leftarrow$  felice(jody)
ascoltaMusica(yolanda)  $\Leftarrow$  felice(yolanda)
...
```

Non esiste un modo per scrivere l’affermazione generale una sola volta.

2. **Non permette di esprimere relazioni fra elementi** (es. $\text{padre}(x, y)$): in proposizionale ogni fatto è un simbolo atomico indivisibile, non si può parlare di *oggetti* che stanno in *relazione* fra loro.

In sintesi: la logica proposizionale tratta il mondo come un insieme di fatti vero/falso “piatti”, senza struttura interna. Serve una logica che parli di **oggetti** e delle **relazioni** che li legano.

Nota di contesto (cenno da slide): esistono molte altre logiche specializzate — temporale (ragiona sul tempo, es. “A non è vero finché B non diventa vero”), epistemica (conoscenza: “l’agente i sa A”), deontica (obblighi/permessi/proibizioni), fuzzy (gradi di verità in $[0,1]$, utile per predicati vaghi come $\text{vecchio}(x)$). FOL è la strada scelta dal corso perché aggiunge espressività mantenendo semantica dichiarativa e

composizionale, senza ambiguità.

Dalla logica proposizionale a FOL

Logica proposizionale: ogni fatto è vero o falso, punto.

Logica del prim'ordine (FOL, First-Order Logic): il mondo è fatto di **oggetti** in **relazione** fra loro; una relazione può essere verificata oppure no per una data tupla di oggetti.

	Modello proposizionale	Modello FOL
Cosa contiene	Attribuzione di valori di verità a simboli proposizionali	Un dominio D (insieme di oggetti del mondo) più delle relazioni fra tali oggetti

Elementi sintattici di FOL

- **Simboli di costante:** Richard, John, 2, Gamba, Corona — nominano oggetti specifici.
- **Simboli di predicato:** Fratello, <, >, SullaTesta — esprimono relazioni/proprietà.
- **Simboli di funzione:** +, Antenato — restituiscono un oggetto a partire da altri oggetti.
- **Simboli di variabile:** x, y, z.
- **Connettivi:** $\Rightarrow$ $\Leftarrow$ $\wedge$ $\vee$ $\neg$ (stessi della proposizionale).
- **Uguaglianza:** =.
- **Quantificatori:** $\forall$ $\exists$ (il vero elemento nuovo).
- **Punteggiatura:** () , .

Per convenzione (nel libro di testo) i simboli di costante, predicato e funzione iniziano con la maiuscola. I simboli di predicato e di funzione hanno un'**arietà** fissa (numero di parametri) e ogni simbolo ha un'**interpretazione**.

Grammatica di FOL

```

formula      → formulaAtomica | (formula connettivo formula)
               | quantificatore variabile,... formula | ¬ formula
formulaAtomica → predicato(termine, ...) | termine = termine
termine      → funzione(termine, ...) | costante | variabile
connettivo   → ⇒ | ⇐ | ∧ | ∨
quantificatore → ∀ | ∃
  
```

Predicati vs funzioni

Entrambi operano su tuple di oggetti del dominio, ma restituiscono cose diverse:

- **Predicati:** dato un insieme di oggetti, catturano una proprietà e restituiscono **vero/falso**. Esempio: Uomo(Andrea), Accanto(X, a).
- **Funzioni:** dato un insieme di oggetti, restituiscono **un oggetto** del dominio. Esempio: più(3,5), parent(X).

Una relazione, in generale, è un insieme di tuple di oggetti del dominio: ad es. { <GiovanniSenzaTerra, RiccardoCuorDiLeone>, <RiccardoCuorDiLeone, GiovanniSenzaTerra> } rappresenta "chi è fratello di chi".

Modelli e interpretazione

Un **modello** in FOL è una coppia $M = (D, I)$:

- D (dominio del discorso): insieme di ≥ 1 oggetti e delle loro relazioni.
- I (interpretazione): associazione fra i simboli del linguaggio e gli elementi/relazioni del dominio:
 - simboli costanti $\rightarrow$ elementi del dominio;
 - simboli di predicato $\rightarrow$ relazioni fra elementi del dominio;
 - simboli di funzione $\rightarrow$ relazioni funzionali fra oggetti del dominio.

Come nella proposizionale, M è un modello di α se α è vera in M .

Un punto sottile ma importante: il valore di verità di una formula dipende interamente dall'interpretazione scelta.

- Se **cambio coerentemente i nomi dei simboli** (es. $\text{John} \rightarrow \text{JohnUsurpatore}$, $\text{Richard} \rightarrow \text{RichardRe}$) ma **mantengo la stessa struttura di interpretazione**, il valore di verità **non cambia**.
- Se invece **cambio l'interpretazione** (es. faccio corrispondere John alla "Corona" invece che a Giovanni senza terra), il valore di verità **può cambiare**: $\text{Fratello}(\text{John}, \text{Richard})$ può diventare falsa anche se sintatticamente è la stessa formula di prima.
- Anche cambiare l'interpretazione dei **predicati** (es. Fratello come "fratellanza fra monaci" invece che "fratellanza di sangue") cambia il valore di verità.
- Se cambio il **dominio del discorso** stesso, devo necessariamente ridefinire l'interpretazione, e la verità delle formule può cambiare di conseguenza.

In breve: **una formula FOL non ha un valore di verità assoluto**, lo ha solo relativamente a un modello (D, I) fissato. Questo è coerente con l'idea (già vista in proposizionale) che la verità è definita rispetto a un'interpretazione.

Soddisfacibilità, validità, insoddisfacibilità (richiamo)

- **Soddisfacibile**: esiste almeno un modello che la rende vera.
- **Valida**: vera in tutti i modelli.
- **Insoddisfacibile**: mai vera in nessun modello.

Quanti modelli ha una KB FOL?

In proposizionale, con N simboli proposizionali, ci sono 2^N modelli possibili (enumerabili). In FOL la situazione è molto peggiore: bisognerebbe considerare, per ogni possibile cardinalità n del dominio (da 1 a ∞), ogni possibile interpretazione di ciascun predicato k -ario su n oggetti, ogni possibile riferimento di ciascuna costante, ecc. — un'enumerazione a più livelli annidati, potenzialmente **infinita**.

? Domanda d'esame: Perché in FOL non si può verificare la conseguenza logica per enumerazione dei modelli, come si faceva in proposizionale? **Risposta:** perché se il dominio D è illimitato e una formula contiene quantificatori, per stabilirne il valore di verità servirebbe calcolare il valore di verità di infinite istanze (una per ogni possibile elemento del dominio sostituito alla variabile quantificata). I modelli possibili in FOL possono quindi essere infiniti, e l'enumerazione — strumento che funzionava in proposizionale — smette di essere praticabile. Occorrono altre tecniche di inferenza (regole di inferenza "liftate", risoluzione, ecc., vedi sotto).

Termini e formule atomiche

Un **termine** è un'espressione che si riferisce a un oggetto del dominio:

termine $\rightarrow$ funzione(termine, ...) | costante | variabile

- Le **costanti** danno un nome a oggetti specifici e noti.
- Le **funzioni** permettono di riferirsi a oggetti che *non hanno un nome proprio*. Importante: una funzione **non costruisce** l'oggetto restituito, si limita a **riferirlo**. Esempio: `GambaSinistra(John)` è un modo per indicare l'oggetto "gamba sinistra di John" già esistente nel dominio — non serve sapere come sia fatta una gamba per usare questo termine.

Termine ground: un termine che non contiene variabili (es. `GambaSinistra(John)`, `Richard`, `Corona`).

Interpretazione di un termine (processo ricorsivo): - se è una costante, l'identificazione con l'oggetto del dominio è immediata; - se è $f(t_1, \dots, t_k)$, prima si interpretano ricorsivamente $t_1, \dots, t_k$ ottenendo gli oggetti $o_1, \dots, o_k$, poi si applica la funzione F (l'interpretazione di f) a questi oggetti, ottenendo l'oggetto risultato o .

Da notare: l'oggetto o (es. "la gamba sinistra di Giovanni") non compare mai esplicitamente nelle formule con un proprio nome, ma esiste comunque nel dominio e viene "raggiunto" passando attraverso il termine funzionale.

Formula atomica:

formulaAtomica $\rightarrow$ predicato(termine, ...) | termine = termine

È vera quando, dato un modello con una certa interpretazione, la relazione denotata dal predicato è verificata dagli oggetti denotati dai termini. Esempi:

- `sposato(madre(Richard), padre(John))`
- `padre(Richard) = padre(John)`

Formule complesse si ottengono componendo formule atomiche con i connettivi, come in proposizionale:

- $\text{Re}(\text{John}) \Rightarrow \neg \text{Re}(\text{Richard})$
- $\text{Persona}(X) \wedge \text{SullaTesta}(\text{Corona}, X) \Rightarrow \text{Re}(X)$

Quantificatori

I quantificatori permettono di esprimere proprietà su **collezioni** di oggetti, facendo riferimento a oggetti generici tramite variabili (es. "tutti gli uomini" diventa "tutti gli X che sono uomini").

- $\forall$: quantificatore **universale** ("per ogni")
- $\exists$: quantificatore **esistenziale** ("esiste")

Quantificatore universale $\forall$

Semantica: dati una formula F e un modello $M = (D, I)$, l'espressione $\forall x F$ è vera in M se e solo se F è vera per **qualsiasi** interpretazione di x in M (cioè per ogni oggetto del dominio sostituito a x , senza filtri di "tipo").

Forma generale: $\forall \langle \text{variabili} \rangle \langle \text{formula} \rangle$

Esempio: "Al corso di sistemi intelligenti tutti sono intelligenti":

$\forall x \text{ Partecipa}(x, \text{SISINT}) \Rightarrow \text{Intelligente}(x)$

Concettualmente questa formula si può "espandere" (se il dominio fosse finito) in una **congiunzione**:

$(\text{Partecipa}(\text{Miriam}, \text{SISINT}) \Rightarrow \text{Intelligente}(\text{Miriam})) \wedge$
 $(\text{Partecipa}(\text{Matteo}, \text{SISINT}) \Rightarrow \text{Intelligente}(\text{Matteo})) \wedge \dots$

per ogni oggetto del dominio, non solo per gli studenti.

Quantificatore esistenziale $\exists$

Semantica: $\exists x$ F è vera in M se e solo se F è vera per **almeno una** (qualche) interpretazione di x in M.

Forma generale: $\exists$ (variabili) (formula)

Esempio: “Al corso di sistemi intelligenti qualcuno è intelligente”:

$\exists x$ Partecipa(x, SISINT) $\wedge$ Intelligente(x)

Si espande in una **disgiunzione**:

(Partecipa(Miriam, SISINT) $\wedge$ Intelligente(Miriam)) $\vee$
(Partecipa(Matteo, SISINT) $\wedge$ Intelligente(Matteo)) $\vee$...

Regola pratica: $\forall$ va con $\Rightarrow$, $\exists$ va con $\wedge$

Questo è uno dei punti in cui si commettono più errori. Le slide lo mostrano esplicitamente con un confronto:

? Domanda d'esame: Che differenza c'è fra $\forall x$ Partecipa(x, SISINT) $\Rightarrow$ Intelligente(x) e $\forall x$ Partecipa(x, SISINT) $\wedge$ Intelligente(x)? Sono equivalenti? **Risposta: NO.** 1. $\forall x$ Partecipa(x, SISINT) $\Rightarrow$ Intelligente(x) significa “tutti quelli che partecipano a SISINT sono intelligenti” — l'implicazione filtra correttamente solo i partecipanti, gli altri oggetti del dominio rendono vera l'implicazione per vacuità (antecedente falso). 2. $\forall x$ Partecipa(x, SISINT) $\wedge$ Intelligente(x) significa “tutti (ogni oggetto del dominio, comprese sedie e numeri) partecipano a SISINT E sono intelligenti” — affermazione quasi certamente falsa e comunque non quella voluta.

Motivo strutturale: con $\forall$ la formula è valutata per *ogni* oggetto del dominio; usare $\wedge$ costringe *ogni* oggetto a soddisfare entrambe le condizioni, mentre di solito si vuole restringere il discorso a una sottoclasse (i partecipanti) — e questo si ottiene con l'implicazione, che è vera automaticamente quando l'oggetto non è nella sottoclasse.

? Domanda d'esame: E per $\exists x$ Partecipa(x, SISINT) $\Rightarrow$ Intelligente(x) vs $\exists x$ Partecipa(x, SISINT) $\wedge$ Intelligente(x)? Sono equivalenti? **Risposta: NO, e qui l'errore è ancora più insidioso.** 1. $\exists x$ Partecipa(x, SISINT) $\Rightarrow$ Intelligente(x) è equivalente (per la legge $A \Rightarrow B \equiv \neg A \vee B$) a $\exists x \neg$ Partecipa(x, SISINT) $\vee$ Intelligente(x): basta che esista **un solo oggetto qualsiasi** che *non* partecipa al corso (es. una sedia) per rendere vera la formula, indipendentemente da chi sia intelligente! Questa formula quindi **non cattura l'intento** di “esiste qualcuno intelligente fra i partecipanti”. 2. $\exists x$ Partecipa(x, SISINT) $\wedge$ Intelligente(x) invece dice correttamente “esiste (almeno) un partecipante a SISINT che è intelligente” — questa è la traduzione corretta dell'intento.

Conclusione pratica: quando si traduce dal linguaggio naturale, usare $\forall \dots \Rightarrow \dots$ per affermazioni universali su una sottoclasse, e $\exists \dots \wedge \dots$ per affermazioni esistenziali su una sottoclasse. Usare $\forall$ con $\wedge$ o $\exists$ con $\Rightarrow$ produce quasi sempre formule sbagliate o triviali.

Quantificatori annidati: l'ordine conta

Regole di equivalenza per quantificatori dello stesso tipo (commutano liberamente fra loro):

1. $\exists x \exists y$ F $\equiv \exists y \exists x$ F (si scrive anche $\exists x, y$ F)
2. $\forall x \forall y$ F $\equiv \forall y \forall x$ F (si scrive anche $\forall x, y$ F)

Ma **quantificatori di tipo diverso NON commutano**:

3. $\forall x \exists y$ F: “per ogni x esiste un y (che può dipendere da x)”. Esempio: $\forall x \exists y$ Ama(x, y) = “tutti amano qualcuno (potenzialmente un qualcuno diverso per ciascuno)”.
4. $\exists y \forall x$ F: “esiste un y (fisso) tale che vale per tutti gli x”. Esempio: $\exists y \forall x$ Ama(x, y) = “esiste qualcuno (uno specifico) che è amato da tutti”.

? Domanda d'esame: Perché $\forall x \exists y \text{ Ama}(x,y)$ e $\exists y \forall x \text{ Ama}(x,y)$ non sono equivalenti? **Risposta:** nel primo caso ogni x può scegliere un y diverso (ognuno ama qualcuno, magari persone diverse); nel secondo caso esiste un **singolo** oggetto y , fissato una volta per tutte, che soddisfa la relazione con *ogni* x (una sola persona amata da tutti). La seconda formula è molto più forte e implica la prima, ma non viceversa. L'ordine di annidamento dei quantificatori determina quindi la "portata" (scope) delle dipendenze fra variabili: la variabile quantificata più esternamente è fissata prima, quella più interna può dipendere dalla precedente. Questo concetto tornerà cruciale nella **skolemizzazione**.

Quantificatori e negazione (leggi di De Morgan generalizzate)

1. $\forall x \neg F \equiv \neg \exists x F$ — "a nessuno piace il cavolfiore" $\equiv$ "non esiste nessuno a cui piaccia il cavolfiore"
2. $\exists x \neg F \equiv \neg \forall x F$ — "c'è qualcuno a cui non piace il cavolfiore" $\equiv$ "non è vero che piace a tutti"
3. $\forall x F \equiv \neg \exists x \neg F$ — "per tutti vale F " $\equiv$ "non esiste nessuno per cui non vale F "
4. $\exists x F \equiv \neg \forall x \neg F$ — "c'è qualcuno per cui vale F " $\equiv$ "non è vero che per tutti non vale F "

In pratica, $\forall$ e $\exists$ sono duali fra loro esattamente come $\wedge$ e $\vee$ lo sono rispetto alla negazione in proposizionale.

Uguaglianza e quantificatori

L'**uguaglianza** ($=$) riguarda esclusivamente **termini** (oggetti), non formule.

Problema tipico: come esprimere "John ha almeno due fratelli"?

- **Tentativo sbagliato:** $\exists y, z \text{ Fratello}(\text{John}, y) \wedge \text{Fratello}(\text{John}, z)$ — **non basta**, perché è soddisfatta anche se y e z vengono unificati con lo stesso oggetto (es. entrambi con Richard).
- **Correzione:** bisogna imporre esplicitamente che y e z si riferiscano a oggetti diversi:

$$\exists y, z \text{ Fratello}(\text{John}, y) \wedge \text{Fratello}(\text{John}, z) \wedge \neg(y=z)$$

Il problema dell'unicità dei nomi

Consideriamo l'asserzione $\text{Fratello}(\text{John}, \text{Richard}) \wedge \text{Fratello}(\text{John}, \text{Ramon})$. Intuitivamente sembra dire che John ha due fratelli, ma **nella semantica standard di FOL** è soddisfatta anche se Richard e Ramon sono due nomi diversi per la **stessa persona**! Per escluderlo servirebbe aggiungere $\neg(\text{Richard}=\text{Ramon})$. E se volessimo dire che John ha *esattamente* due fratelli (non di più), servirebbe anche:

$$\forall x (\text{Fratello}(\text{John}, x) \Rightarrow (x=\text{Richard} \vee x=\text{Ramon}))$$

Il risultato è corretto ma **le formule diventano rapidamente complicatissime e poco intuitive**.

Database semantics (semantica alternativa più intuitiva)

Per evitare queste complicazioni, la programmazione logica adotta spesso la **database semantics**, basata su tre assunzioni:

1. **Unicità dei nomi (Unique Names Assumption):** costanti diverse denotano sempre oggetti diversi.
2. **Closed-world assumption:** le formule atomiche di verità sconosciuta sono considerate false.
3. **Domain closure:** il modello non contiene più oggetti di quelli nominati dalle costanti presenti.

Con questa semantica, semplicemente $\text{Fratello}(\text{John}, \text{Richard}) \wedge \text{Fratello}(\text{John}, \text{Ramon})$ rappresenta correttamente "John ha (esattamente) due fratelli, Richard e Ramon" — molto più intuitivo. Riduce anche il numero di modelli possibili, rendendoli tipicamente finiti.

Nota importante per l'esame: **il libro di testo (Russell & Norvig) usa la semantica standard di FOL** anche dopo aver introdotto la database semantics — quest'ultima è tipica di Prolog/programmazione logica, va tenuta distinta e usata solo quando siamo certi dell'identità di tutti gli elementi del dominio.

Inferenza in FOL: due approcci

1. **Proposizionalizzazione** della KB e uso di un algoritmo per la logica proposizionale.
2. **Lifting** delle regole di inferenza al prim'ordine (con unificazione) — l'approccio più efficiente ed usato in pratica.

Interrogare una KB FOL

Due tipi di query:

- ask(KB, Re(John)): formula **ground** (senza variabili) — risposta true/false.
- ask(KB, Re(x)): formula con variabile libera — risposta false se non esiste alcun valore che renda vera la formula, altrimenti un **termine ground** che, sostituito a x, la rende vera.

Sostituzione

Una **sostituzione** θ è un insieme $\{x_1/g_1, x_2/g_2, \dots, x_n/g_n\}$ dove le x_i sono variabili e le g_i sono termini ground. La notazione F/θ (o $F\theta$) indica la formula ottenuta applicando θ a F .

Esempio: $F = \text{Fratello}(x, y)$, $\theta = \{x/\text{John}\} \implies F\theta = \text{Fratello}(\text{John}, y)$.

Proposizionalizzazione: UI ed EI

Regola di istanziazione universale (UI):

$$\frac{\forall x \alpha}{\text{SUBST}(\{x/g\}, \alpha)}$$

Da una formula universalmente quantificata si può inferire qualunque istanza ottenuta sostituendo alla variabile un **qualsiasi termine ground** del vocabolario. Esempio: da $\forall x (\text{Partecipa}(x, \text{SISINT}) \implies \text{Intelligente}(x))$, con $\theta_1 = \{x/\text{Rufus}\}$ si inferisce $\text{Partecipa}(\text{Rufus}, \text{SISINT}) \implies \text{Intelligente}(\text{Rufus})$.

Regola di istanziazione esistenziale (EI):

$$\frac{\exists x \alpha}{\text{SUBST}(\{x/k\}, \alpha)}$$

dove k deve essere una **costante nuova** (non ancora usata nella KB). Esempio: da $\exists x \text{Corona}(x) \wedge \text{SullaTesta}(x, \text{John})$ si inferisce $\text{Corona}(C1) \wedge \text{SullaTesta}(C1, \text{John})$, con $C1$ costante nuova generata appositamente — questo processo di generare un nome nuovo si chiama **skolemizzazione** (le costanti così introdotte sono dette **costanti di Skolem**).

Differenza fondamentale fra UI ed EI: - **UI:** produce tutte le istanze possibili; la nuova KB è **logicamente equivalente** a quella originaria. - **EI:** si applica **una sola volta** per ciascuna formula esistenziale e poi la formula quantificata viene scartata; la nuova KB **non** è logicamente equivalente all'originaria, ma è **soddisfacibile se e solo se** lo era la KB originaria (equivalenza *inferenziale*, non logica).

Il problema delle funzioni e il teorema di Herbrand

Se il vocabolario contiene funzioni, esse possono essere annidate ricorsivamente ($f(f(f(\dots x \dots)))$), quindi l'insieme dei possibili termini ground — e delle sostituzioni possibili per UI — diventa **potenzialmente infinito**.

Teorema di Herbrand: se una formula è conseguenza logica della KB FOL originaria, allora esiste una dimostrazione **finita** della sua verità a partire dalla KB proposizionalizzata. I termini vengono costruiti "in ampiezza" (breadth-first): prima le sole costanti, poi i termini con una sola applicazione di funzione, poi quelli con due applicazioni, ecc.

Conseguenza — semidecidibilità di FOL: - **Completezza:** se una formula consegue dalla KB, la si troverà (dimostrazione finita, per Herbrand). - Ma se la conseguenza **non** vale, la ricerca (a causa delle funzioni ricorsive) può proseguire **all'infinito** senza mai terminare. - Quindi: non esiste un algoritmo generale in grado di dimostrare che una conseguenza *non* vale in FOL. FOL è **semidecidibile** (si può confermare il "sì" ma non sempre il "no").

Inefficienza della proposizionalizzazione

Esempio:

$\forall x \text{ Re}(x) \wedge \text{Avido}(x) \Rightarrow \text{Malvagio}(x)$
 $\text{Re}(\text{John})$
 $\text{Avido}(\text{John})$
 $\text{Fratello}(\text{Richard}, \text{John})$

La proposizionalizzazione genera un'istanza dell'implicazione **per ogni costante del vocabolario**, incluso $\text{Re}(\text{Richard}) \wedge \text{Avido}(\text{Richard}) \Rightarrow \text{Malvagio}(\text{Richard})$, anche se è ovvio a colpo d'occhio che solo John può risultare malvagio. Si "spreca tempo" a creare istanze inutili — tanto più gravoso quanto più il vocabolario è ampio (e peggio ancora se sono presenti variabili universali multiple, come $\forall y \text{ Avido}(y)$, che moltiplicano ulteriormente le istanze inutili).

Questo motiva il passaggio a regole di inferenza "**liftate**" direttamente al prim'ordine, che evitano l'esplosione combinatoria della proposizionalizzazione.

Modus Ponens Generalizzato (MPG)

Formula

$$\frac{p'1, p'2, \dots, p'n, \quad (p1 \wedge p2 \wedge \dots \wedge pn \Rightarrow q)}{q\theta}$$

dove $p'i \theta = pi \theta$ per ogni $i \in [1, n]$.

Spiegazione: la regola ha come premesse n formule atomiche $p'1, \dots, p'n$ e una singola implicazione la cui antecedente è la congiunzione $p1 \wedge \dots \wedge pn$. Se esiste una sostituzione θ tale che ciascuna $p'i$ unifica con la corrispondente pi (cioè $p'i\theta = pi\theta$), allora si può concludere $q\theta$, cioè la conseguenza dell'implicazione con θ applicata.

$q\theta$ è la notazione sintetica per $\text{SUBST}(\theta, q)$.

Esempio

Premesse: $\text{Re}(\text{John}), \text{Avido}(y), \text{Re}(x) \wedge \text{Avido}(x) \Rightarrow \text{Malvagio}(x)$.

Per applicare MPG serve unificare $\text{Re}(\text{John})$ con $\text{Re}(x)$ (dando x/John) e $\text{Avido}(y)$ con $\text{Avido}(x)$ (dando y/x , e con x/John ottenuto sopra, quindi y/John). La sostituzione complessiva è $\theta = \{x/\text{John}, y/\text{John}\}$, e si conclude:

Malvagio(John)

Le clausole guidano quindi la **ricerca della sostituzione** giusta, e permettono di ragionare **direttamente in FOL** senza passare per l'espansione esaustiva di tutte le costanti del vocabolario (a differenza della proposizionalizzazione).

Lifting

Il modus ponens proposizionale non pone restrizioni sulla forma della formula antecedente. Il **Modus Ponens Generalizzato** invece richiede che l'antecedente sia una **congiunzione di letterali positivi**, e generalizza la regola:

- consentendo un numero arbitrario di premesse atomiche (non solo due, come nel modus ponens classico);
- operando su formule con **variabili**, tramite unificazione.

Il termine "generalizzato" deriva proprio dall'aver "sollevato" (**lifted**) la regola dalla logica proposizionale a quella del prim'ordine — questo procedimento generale si chiama **lifting** e verrà applicato anche alla risoluzione (vedi sotto).

Unificazione

Definizione: l'unificazione è l'algoritmo chiave di tutte le tecniche di inferenza in FOL. Date due formule F_1 e F_2 :

$$\text{UNIFY}(F_1, F_2) = \theta \quad \text{tale che} \quad F_1\theta = F_2\theta$$

Il risultato è una sostituzione θ che, applicata a entrambe le formule, le rende **sintatticamente identiche**. Tale sostituzione, se esiste, è detta **unificatore**. Se ne esistono più di una, si preferisce calcolare e usare il **Most General Unifier (MGU)**, l'unificatore più generale (quello che vincola meno le variabili, lasciando aperte più possibilità).

Esempi passo-passo

1. $\text{UNIFY}(\text{Conosce}(\text{John}, x), \text{Conosce}(\text{John}, \text{Richard}))$: John coincide già; x deve unificare con Richard $\Rightarrow \theta = \{x/\text{Richard}\}$.
2. $\text{UNIFY}(\text{Conosce}(\text{John}, x), \text{Conosce}(y, \text{Richard}))$: John deve unificare con y ; x deve unificare con Richard $\Rightarrow \theta = \{x/\text{Richard}, y/\text{John}\}$.
3. $\text{UNIFY}(\text{Conosce}(\text{John}, x), \text{Conosce}(y, \text{MadreDi}(y)))$: John unifica con $y \Rightarrow y/\text{John}$; sostituendo nel secondo termine, x deve unificare con $\text{MadreDi}(y)$ che con y/John diventa $\text{MadreDi}(\text{John}) \Rightarrow \theta = \{x/\text{MadreDi}(\text{John}), y/\text{John}\}$.
4. $\text{UNIFY}(\text{Conosce}(\text{John}, x), \text{Conosce}(x, \text{Richard}))$: **fallisce**, perché la stessa variabile x dovrebbe assumere contemporaneamente due valori diversi (John a sinistra, Richard a destra) — è un conflitto di occorrenza/legame. Per evitare questo tipo di collisioni accidentali fra variabili di formule diverse, si applica la **standardizzazione separata** (*standardizing apart*): si rinominano le variabili di una formula in modo che non collidano mai con quelle di un'altra formula prima di tentare l'unificazione.

Clausole di Horn del prim'ordine

Analoghe a quelle proposizionali: sono disgiunzioni di letterali di cui **al più uno è positivo**. In pratica, nella pratica di modellazione:

- **Formule atomiche** (fatti): es. $Avido(x)$ (variabile intesa universalmente quantificata), oppure $Avido(John)$ (fatto ground).
- **Implicazioni** il cui antecedente è una congiunzione di letterali positivi: $Re(x) \wedge Avido(x) \Rightarrow Malvagio(x)$.

Non tutte le KB sono traducibili in clausole di Horn, ma molte lo sono — e a queste si può applicare il **forward chaining** (con MPG) per fare inferenza.

KB di esempio: “Sotto Casa”

Enunciato: *“La legge dice che è un reato per un negozio vendere alcolici a un minorenne. Marco, minorenne, possiede della birra. Tale birra gli è stata venduta dal minimarket Sotto Casa.”* Obiettivo: dimostrare che Sotto Casa è reo.

Vocabolario: - Costanti: Marco, SottoCasa - Predicati: Vende (ternario), Negozio, Supermarket, Birra, Alcolico, Minorenne, Possiede, Reo - Funzioni: nessuna

Formalizzazione:

C1) $Negozio(x) \wedge Vende(x,y,z) \wedge Alcolico(y) \wedge Minorenne(z) \Rightarrow Reo(x)$

(“è un reato per un negozio vendere alcolici a un minorenne”)

$\exists x Possiede(Marco,x) \wedge Birra(x)$

(“Marco possiede della birra”) — applicando **EI**, si introduce la costante di Skolem B (“la birra”):

C2) $Possiede(Marco, B)$

C3) $Birra(B)$

C4) $Possiede(Marco,x) \wedge Birra(x) \Rightarrow Vende(SottoCasa,x,Marco)$

(“se Marco possiede della birra, l’ha comprata al minimarket”)

C5) $Birra(x) \Rightarrow Alcolico(x)$

(“la birra è un alcolico”)

C6) $Minimarket(SottoCasa)$

C7) $Minimarket(x) \Rightarrow Negozio(x)$

C8) $Minorenne(Marco)$

Questa KB è composta da clausole di Horn del prim’ordine **senza funzioni**: questa particolare sottoclasse (Horn + no funzioni) è chiamata **DATALOG**.

Nota (citazione da slide, statistico George Box): “tutti i modelli sono sbagliati, alcuni sono utili” — la modellazione in clausole di Horn qui proposta non è l’unica possibile, ma è una semplificazione utile.

Forward Chaining in FOL

L’algoritmo è concettualmente simile al caso proposizionale, ma richiede cautele aggiuntive per via delle variabili:

- Un fatto è considerato una **rinomina** di un altro se sono identici a meno dei nomi delle variabili (es. $Fratello(John,x)$ e $Fratello(John,y)$ sono rinomine l’uno dell’altro).
- In proposizionale un fatto inferito si aggiunge alla KB solo se non è già presente; in FOL si aggiunge solo se **non è una rinomina** di un fatto già presente.

Esempio (continuazione KB Sotto Casa): da C2, C3, C4 si applica MPG unificando x con B ($\theta = \{x/B\}$), ottenendo:

Vende(SottoCasa, B, Marco)

Il processo continua costruendo un **grafo AND-OR**: si combina con C5 per ottenere Alcolico(B), con C6/C7 per ottenere Negozio(SottoCasa), con C8 (Minorenne(Marco)) e infine con C1 (unificando $x/\text{SottoCasa}$, y/B , z/Marco) si conclude:

Reo(SottoCasa)

Punto chiave: **non si propagano solo valori di verità** (come in proposizionale) ma anche **le sostituzioni**, che fanno da collante fra le diverse applicazioni successive di MPG lungo il grafo di inferenza.

Proprietà: - **Correttezza**: garantita, perché si basa sulla regola di inferenza corretta (MPG, *Generalized Modus Ponens*). - **Completezza**: se la KB è DATALOG (Horn, senza funzioni), l'algoritmo è completo e **termina**. Se la KB contiene funzioni, per il teorema di Herbrand l'algoritmo può **non terminare** se la risposta cercata non è implicata dalla KB.

Backward Chaining (BC) in FOL

Come nel caso proposizionale, il ragionamento è **guidato dall'obiettivo** (goal-driven):

1. L'obiettivo viene inserito in uno **stack**.
2. Iterativamente si estrae un obiettivo dallo stack e si cercano clausole la cui **testa** (conseguente) sia **unificabile** con l'obiettivo:
 - se l'obiettivo unifica con un **fatto** (clausola a corpo vuoto), viene semplicemente rimosso (risolto);
 - altrimenti, per ogni clausola applicabile si avvia un procedimento **ricorsivo** che inserisce nello stack le premesse della clausola (con le variabili opportunamente sostituite secondo l'unificatore trovato).
3. Quando lo stack è vuoto: **successo**. Se non si possono applicare altre inferenze: **fallimento**.

Esempio (KB Sotto Casa), traccia sintetica: - Stack iniziale: Reo(SottoCasa). - Unifica con la testa di C1 ($\theta_1 = \{x/\text{SottoCasa}\}$) $\Rightarrow$ Stack: Negozio(SottoCasa), Vende(SottoCasa, y, z), Alcolico(y), Minorenne(z). - Negozio(SottoCasa) unifica con la testa di una clausola tipo Supermarket(x) $\Rightarrow$ Negozio(x), che a sua volta si risolve con il fatto Supermarket(SottoCasa) (analogo a C6/C7). - Alcolico(y) unifica con la testa di C5 (Birra(x) $\Rightarrow$ Alcolico(x), con $\theta_2 = \{y/x\}$), che si risolve con C3) Birra(B) ($\theta_2 = \{x/B\}$). - Vende(SottoCasa, y, z) (con y ormai legata a B) unifica con la testa di C4, che genera i sotto-obiettivi risolti da C2) Possiede(Marco, B) e C3) Birra(B). - Minorenne(z) (con z legata a Marco tramite le sostituzioni composte) unifica con il fatto C8) Minorenne(Marco). - Stack vuoto $\Rightarrow$ **successo**, Reo(SottoCasa) dimostrato.

Composizione delle sostituzioni: l'algoritmo BC applica ripetutamente la composizione $\text{COMPOSE}(\theta_1, \theta_2) = \theta_3$, con la proprietà:

$$\text{SUBST}(\text{COMPOSE}(\theta_1, \theta_2), F) = \text{SUBST}(\theta_2, \text{SUBST}(\theta_1, F))$$

cioè applicare la sostituzione composta equivale ad applicare prima θ_1 e poi θ_2 in sequenza. È necessario concatenare correttamente le sostituzioni via via trovate lungo la catena di risoluzione degli obiettivi per ottenere l'inferenza corretta finale.

Valutazione di BC: - **Corretto**. - **Incompleto**, perché implementa una strategia depth-first: può incorrere in **loop infiniti** e generare stati ripetuti. - Come nel caso proposizionale, è generalmente **più efficiente** del forward chaining (perché esplora solo ciò che serve a dimostrare l'obiettivo, non tutte le conseguenze possibili della KB).

Risoluzione in FOL: lifting e refutazione

Nel caso proposizionale, la regola di **risoluzione** unita all'algoritmo di **refutazione** costituisce una procedura di inferenza **completa** (vedi Modulo 5). Questa combinazione può essere "liftata" (sollevata) anche a FOL:

- La KB va tradotta in **CNF** (Conjunctive Normal Form: congiunzione di clausole, ciascuna disgiunzione di letterali).
- Le variabili nelle clausole sono intese **implicitamente quantificate universalmente**.
- Ogni KB FOL può essere tradotta in una KB CNF **inferenzialmente equivalente**: la CNF è insoddisfacibile se e solo se lo è la KB FOL originaria. Questo è ciò che rende lecita l'applicazione della procedura di refutazione.

Procedura per tradurre FOL in CNF

Esempio guida: $\forall x [\forall y \text{ Animale}(y) \Rightarrow \text{Ama}(x,y)] \Rightarrow [\exists y \text{ Ama}(y,x)]$ ("tutti coloro che amano gli animali sono amati da qualcuno").

Passo 1 — Elimina l'implicazione ($A \Rightarrow B \equiv \neg A \vee B$):

$$\forall x \neg [\forall y \text{ Animale}(y) \Rightarrow \text{Ama}(x,y)] \vee [\exists y \text{ Ama}(y,x)]$$

$$\forall x \neg [\forall y \neg \text{Animale}(y) \vee \text{Ama}(x,y)] \vee [\exists y \text{ Ama}(y,x)]$$

Passo 2 — Sposta la negazione all'interno (usando $\neg \forall \equiv \exists \neg$ e De Morgan):

$$\forall x [\exists y \text{ Animale}(y) \wedge \neg \text{Ama}(x,y)] \vee [\exists y \text{ Ama}(y,x)]$$

Passo 3 — Standardizzazione delle variabili: le due occorrenze di $\exists y$ sono variabili quantificate indipendenti (scope diversi), vanno rinominate per evitare ambiguità:

$$\forall x [\exists y \text{ Animale}(y) \wedge \neg \text{Ama}(x,y)] \vee [\exists z \text{ Ama}(z,x)]$$

Passo 4 — Skolemizzazione (vedi sezione dedicata sotto): elimina i quantificatori esistenziali.

Passo 5 — Cancella i quantificatori universali (rimasti impliciti, dato che in CNF tutte le variabili libere si intendono universali).

Passo 6 — Distribuisci $\vee$ su $\wedge$ per ottenere la forma a clausole (congiunzione di disgiunzioni).

Il risultato finale non è pensato per essere leggibile da un umano, ma per essere processato automaticamente da un algoritmo di inferenza.

Skolemizzazione

Perché serve

Al passo 4 sopra, non si può applicare direttamente la regola EI (istanziamento esistenziale con una singola costante) perché la formula non ha la forma semplice $\exists x F(x)$: l'esistenziale è annidato dentro lo scope di un quantificatore universale ($\forall x [\exists y \dots]$).

Se, sbagliando, si sostituisse $\exists y \text{ Animale}(y)$ con una singola costante A (come farebbe EI ingenuamente), si otterrebbe:

$$\forall x [\text{Animale}(A) \wedge \neg \text{Ama}(x,A)] \vee [\text{Ama}(B,x)]$$

che significa "**tutti** amano uno **stesso specifico** animale A" e "sono amati da uno **stesso specifico** essere B" — A e B sarebbero costanti fisse, uguali per ogni x. Questo è **sbagliato**: l'animale amato o l'amante possono variare da persona a persona.

Procedura (regola generale)

Si sostituisce ogni variabile quantificata esistenzialmente con una **funzione di Skolem** i cui argomenti sono **tutte le variabili universalmente quantificate nel cui scope ricade** l'esistenziale:

$\forall x_1, x_2, \dots [\exists y P(y, x_1, \dots) \dots \exists z Q(z, x_1, \dots)]$
 diventa
 $\forall x_1, x_2, \dots [P(S_1(x_1, x_2, \dots), \dots) \dots Q(S_2(x_1, x_2, \dots), \dots)]$

S_1, S_2 sono dette **funzioni di Skolem**.

Caso particolare: se l'esistenziale **non** ricade nello scope di alcun universale, la funzione di Skolem degenera in una **costante di Skolem** (è esattamente il caso della regola EI vista prima, con la costante B/C_1 nell'esempio "Sotto Casa" — lì infatti non c'era nessun $\forall$ esterno a $\exists x \text{ Possiede}(\text{Marco}, x) \wedge \text{Birra}(x)$).

Applicando la regola all'esempio guida: le variabili esistenziali y e z ricadono entrambe nello scope di $\forall x$, quindi diventano funzioni di x :

$\forall x [\text{Animale}(F(x)) \wedge \neg \text{Ama}(x, F(x))] \vee [\text{Ama}(G(x), x)]$

Dopo la skolemizzazione si completano i passi 5 e 6 (elimina $\forall$ residui, distribuisce $\vee$ su $\wedge$):

$[\text{Animale}(F(x)) \vee \text{Ama}(G(x), x)] \wedge [\neg \text{Ama}(x, F(x)) \vee \text{Ama}(G(x), x)]$

Esempio più semplice, passo-passo (per fissare l'intuizione)

Consideriamo: $\forall x \exists y \text{ persona}(x) \Rightarrow (\text{abita}(x, y) \wedge \text{paese}(y))$ ("ogni persona abita in un luogo che è in un paese").

Errore da non fare: applicare EI ingenuamente eliminerebbe $\exists y$ con una singola costante K :

$\forall x \text{ persona}(x) \Rightarrow (\text{abita}(x, K) \wedge \text{paese}(K))$

Questo fissa **un'unica costante K uguale per tutti gli x**, cioè affermerebbe che **tutte le persone abitano nello stesso posto** — chiaramente sbagliato: persone diverse possono abitare in paesi diversi (o anche nello stesso paese, ma non è imposto che sia sempre lo stesso).

Soluzione corretta: il luogo in cui una persona abita **dipende** (è funzione) dallo specifico individuo, quindi si introduce una funzione di Skolem $\text{luogo}(x)$:

$\forall x \text{ persona}(x) \Rightarrow (\text{abita}(x, \text{luogo}(x)) \wedge \text{paese}(\text{luogo}(x)))$

Ora il luogo può variare correttamente da persona a persona.

Funzioni di Skolem con più argomenti

Le funzioni di Skolem hanno come argomenti **tutte** le variabili universali nel cui scope ricadono, quindi possono avere arità > 1 . Esempio: in un rally, per ogni coppia di persone x, y esiste un'auto z di cui x è il pilota e y il navigatore:

$\forall x, y \exists z \text{ persona}(x) \wedge \text{persona}(y) \wedge \text{auto}(z) \wedge \neg(x=y) \Rightarrow (\text{pilota}(x, z) \wedge \text{navigatore}(y, z))$

Skolemizzando (z dipende sia da x che da y , quindi funzione a due argomenti $A(x, y)$):

$\forall x, y \text{ persona}(x) \wedge \text{persona}(y) \wedge \text{auto}(A(x, y)) \wedge \neg(x=y) \Rightarrow (\text{pilota}(x, A(x, y)) \wedge \text{navigatore}(y, A(x, y)))$

Caso limite: l'esistenziale "esterno" a tutti gli universali

Attenzione allo **scope**: le funzioni di Skolem dipendono dalle variabili universali nel cui scope ricade l'esistenziale, e lo scope dipende dall'**ordine di annidamento**, non dall'ordine di scrittura sulla carta.

- Se la formula è $\forall x, y \exists z \text{ formula}(x, y, z)$: z è dentro lo scope di x e $y \Rightarrow$ funzione di Skolem $S(x, y)$ con 2 argomenti.

- Se la formula è invece $\exists z [\forall x, y \text{ formula}(x, y, z)]$: qui è $\exists z$ a essere **più esterno**, quindi sono x e y a ricadere nello scope dell'esistenziale (e non viceversa!) — la formula dice che esiste un **unico** valore di z che vale per *tutti* i possibili x, y . In questo caso la funzione di Skolem **non ha argomenti**, cioè degenera in una **costante di Skolem** (si applica l'istanziamento esistenziale semplice, EI), esattamente come accadrebbe per la formula ancora più semplice $\exists z \text{ formula}(x, y, \dots, z)$ priva di universali che la "contengono".

? Domanda d'esame: Da cosa dipendono gli argomenti di una funzione di Skolem, e perché a volte si riduce a una costante? **Risposta:** gli argomenti sono esattamente le variabili quantificate universalmente il cui scope contiene (racchiude) il quantificatore esistenziale da eliminare — perché il valore "esistente" può, in linea di principio, variare al variare di quelle variabili universali già fissate più esternamente. Se non ci sono variabili universali che "contengono" l'esistenziale (l'esistenziale è il quantificatore più esterno, o comunque non è annidato dentro nessun $\forall$), allora il valore esistente è **unico e fisso**, non dipende da nulla, e la funzione di Skolem degenera in una semplice **costante di Skolem** — esattamente il caso della regola EI.

Binary resolution in FOL (lifting della risoluzione)

Formula generale (risoluzione binaria liftata):

$$l_1 \vee \dots \vee l_k, \quad m_1 \vee \dots \vee m_n$$

$$\text{SUBST}(\theta, l_1 \vee \dots \vee l_i - 1 \vee l_i + 1 \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_j - 1 \vee m_j + 1 \vee \dots \vee m_n)$$

dove θ è una sostituzione tale che, per una coppia di indici i, j , θ **unifica** l_i con $\neg m_j$ (cioè rende l_i e la negazione di m_j sintatticamente identici).

Esempio: $\text{Re}(\text{John})$ e $\neg \text{Re}(x)$ sono "opposte"; la sostituzione $\theta = \{x/\text{John}\}$ rende $\text{Re}(\text{John})\theta \equiv \neg \neg \text{Re}(x)\theta$, quindi le due clausole risolvono fra loro eliminando questi due letterali.

Osservazioni tecniche: - Le due clausole da risolvere **non devono condividere variabili** (da qui la necessità della standardizzazione separata, come nell'unificazione). - Va fatto anche il lifting della **fattorizzazione**: due letterali di una stessa clausola si riducono a uno solo non se sono sintatticamente uguali, ma se sono **unificabili** — e l'unificatore trovato va applicato all'intera clausola. - **Binary resolution + fattorizzazione** insieme costituiscono una regola di inferenza **completa**.

Dimostrazione per refutazione: l'esempio "Curiosity ha ucciso il gatto?"

Grazie al lifting della risoluzione, si può applicare la procedura di **refutazione** anche a FOL. Esempio classico:

- A) Tutti coloro che amano gli animali sono amati da qualcuno.
- B) Tutti coloro che uccidono un animale non sono amati da nessuno.
- C) Jack ama tutti gli animali.
- D) O Jack o Curiosity ha ucciso il gatto, il cui nome è Tuna.
- E) Gatto(Tuna).
- F) Tutti i gatti sono animali.

Query: **Curiosity ha ucciso il gatto?**

Formalizzazione FOL:

- A) $\forall x [\forall y \text{ Animale}(y) \Rightarrow \text{Ama}(x, y)] \Rightarrow [\exists y \text{ Ama}(y, x)]$
- B) $\forall x [\exists z \text{ Animale}(z) \wedge \text{Uccide}(x, z)] \Rightarrow [\forall y \neg \text{Ama}(y, x)]$
- C) $\forall x \text{ Animale}(x) \Rightarrow \text{Ama}(\text{Jack}, x)$
- D) $\text{Uccide}(\text{Jack}, \text{Tuna}) \vee \text{Uccide}(\text{Curiosity}, \text{Tuna})$
- E) $\text{Gatto}(\text{Tuna})$

F) $\forall x \text{ Gatto}(x) \Rightarrow \text{Animale}(x)$

Per dimostrare per **refutazione** che la risposta è “sì”, si aggiunge il **goal negato**:

G) $\neg \text{Uccide}(\text{Curiosity}, \text{Tuna})$

e si cerca di derivare una contraddizione (clausola vuota) da $\text{KB} \wedge \neg Q$. Se ci si riesce, allora $\text{KB} \models Q$.

Traduzione in CNF (applicando i 6 passi visti sopra, con skolemizzazione dove serve — nella formula A l'esistenziale $\exists y$ dipende da x , quindi diventa funzione di Skolem $G(x)$; analogamente in B $\exists z$ diventa $F(x)$):

A1) $\text{Animale}(F(x)) \vee \text{Ama}(G(x), x)$

A2) $\neg \text{Ama}(x, F(x)) \vee \text{Ama}(G(x), x)$

B) $\neg \text{Ama}(y, x) \vee \neg \text{Animale}(z) \vee \neg \text{Uccide}(x, z)$

C) $\neg \text{Animale}(x) \vee \text{Ama}(\text{Jack}, x)$

D) $\text{Uccide}(\text{Jack}, \text{Tuna}) \vee \text{Uccide}(\text{Curiosity}, \text{Tuna})$

E) $\text{Gatto}(\text{Tuna})$

F) $\neg \text{Gatto}(x) \vee \text{Animale}(x)$

G) $\neg \text{Uccide}(\text{Curiosity}, \text{Tuna})$

Catena di risoluzione (schema concettuale, unificando via via le variabili con le costanti opportune):

1. E) $\text{Gatto}(\text{Tuna})$ risolve con F) $\neg \text{Gatto}(x) \vee \text{Animale}(x)$ (unificando x/Tuna) $\Rightarrow \text{Animale}(\text{Tuna})$.
2. D) $\text{Uccide}(\text{Jack}, \text{Tuna}) \vee \text{Uccide}(\text{Curiosity}, \text{Tuna})$ risolve con G) $\neg \text{Uccide}(\text{Curiosity}, \text{Tuna}) \Rightarrow \text{Uccide}(\text{Jack}, \text{Tuna})$.
3. $\text{Animale}(\text{Tuna})$ risolve con B) $\neg \text{Ama}(y, x) \vee \neg \text{Animale}(z) \vee \neg \text{Uccide}(x, z)$ (unificando z/Tuna) $\Rightarrow \neg \text{Ama}(y, x) \vee \neg \text{Uccide}(x, \text{Tuna})$.
4. Questa risolve con $\text{Uccide}(\text{Jack}, \text{Tuna})$ (unificando x/Jack) $\Rightarrow \neg \text{Ama}(y, \text{Jack})$.
5. $\neg \text{Ama}(y, \text{Jack})$ risolve con A2) $\neg \text{Ama}(x, F(x)) \vee \text{Ama}(G(x), x)$ (unificando y/Jack da un lato e cercando di far coincidere Jack con $F(x)$... nello sviluppo delle slide, la catena prosegue unificando opportunamente fino a produrre $\neg \text{Animale}(F(\text{Jack})) \vee \text{Ama}(G(\text{Jack}), \text{Jack})$).
6. $\neg \text{Animale}(F(\text{Jack})) \vee \text{Ama}(G(\text{Jack}), \text{Jack})$ risolve con A1) $\text{Animale}(F(x)) \vee \text{Ama}(G(x), x)$ (istanziato in x/Jack , dando $\text{Animale}(F(\text{Jack})) \vee \text{Ama}(G(\text{Jack}), \text{Jack})$) $\Rightarrow$ per fattorizzazione/risoluzione si giunge a $\text{Ama}(G(\text{Jack}), \text{Jack})$.
7. Il ramo produce infine la **clausola vuota** (contraddizione), confermando che $\text{KB} \wedge \neg Q$ è insoddisfacibile, dunque **Curiosity ha ucciso il gatto** è conseguenza logica della KB.

(Lo schema esatto dell'albero binario di risoluzione, come presentato nelle slide originali, è un albero con foglie $\text{Gatto}(\text{Tuna})$, $\neg \text{Gatto}(\text{Tuna}) \vee \text{Animale}(\text{Tuna})$, $\text{Uccide}(\text{Jack}, \text{Tuna}) \vee \text{Uccide}(\text{Curiosity}, \text{Tuna})$, $\neg \text{Uccide}(\text{Curiosity}, \text{Tuna})$ e le clausole A1/A2/B/C, che si combinano a coppie fino a produrre il nodo radice $\text{Ama}(G(\text{Jack}), \text{Jack})$ seguito dalla clausola vuota — utile soprattutto per intuire il meccanismo, il dettaglio grafico esatto è secondario rispetto a comprendere che ogni passo è un'applicazione di risoluzione binaria con unificazione.)

Refutation-completeness

La risoluzione **non** è in grado di generare (enumerare) *tutte* le conseguenze logiche di una KB, ma è **refutation-complete**:

- Se una KB è **insoddisfacibile**, la risoluzione sarà sempre in grado di derivare, in un numero finito di passi, la clausola vuota (contraddizione).
- Di conseguenza, è in grado di derivare **tutte le risposte** a una query $Q(x)$ a partire da una KB, a patto di verificare che $\text{KB} \wedge \neg Q(x)$ sia insoddisfacibile — è esattamente lo schema della dimostrazione per refutazione già visto in proposizionale, ora esteso a FOL grazie al lifting.

Costruire una KB in FOL: ingegneria della conoscenza

Processo generale (*knowledge engineering*):

1. Identificare l'uso che si desidera fare della KB.
2. Raccogliere la conoscenza rilevante (in forma informale).
3. Definire un vocabolario di costanti, funzioni e predicati.
4. Formalizzare la conoscenza in formule FOL.

Quando poi si vuole interrogare la KB:

1. Descrivere in modo formale la specifica istanza del problema.
2. Interrogare la KB (query).

Riepilogo e punti chiave

- La logica proposizionale è dichiarativa, compositiva e non ambigua, ma **manca di espressività**: non permette rappresentazioni compatte né relazioni fra oggetti. FOL aggiunge **oggetti, relazioni (predicati), funzioni e quantificatori**, mantenendo le buone proprietà della proposizionale.
- Un **modello FOL** è la coppia $M=(D, I)$: dominio + interpretazione. Il valore di verità di una formula dipende sempre dal modello scelto, non è assoluto.
- **Termini** (costanti, variabili, applicazioni di funzione) denotano oggetti; le **formule atomiche** (predicato applicato a termini, o uguaglianza fra termini) esprimono fatti su di essi.
- $\forall$ si accompagna naturalmente a $\Rightarrow$ (per restringere il discorso a una sottoclasse), $\exists$ si accompagna naturalmente a $\wedge$ (per affermare l'esistenza di un caso con più proprietà congiunte). Usare la combinazione sbagliata è l'errore più comune in FOL.
- L'**ordine dei quantificatori annidati di tipo diverso conta**: $\forall x \exists y \neq \exists y \forall x$. Quantificatori dello stesso tipo invece commutano liberamente.
- L'**uguaglianza** serve a distinguere/identificare oggetti; senza $\neg(x=y)$ esplicito, FOL standard non assume l'unicità dei nomi (a differenza della *database semantics*, con Unique Names + Closed World + Domain Closure, usata in programmazione logica).
- L'inferenza in FOL può avvenire per **proposizionalizzazione** (UI/EI + algoritmo proposizionale, corretto e completo per il teorema di Herbrand ma inefficiente e a rischio di non terminazione se la KB contiene funzioni) o per **lifting delle regole di inferenza** (più efficiente).
- Il **Modus Ponens Generalizzato (MPG)** collega n premesse atomiche più un'implicazione con antecedente congiuntivo a una conclusione istanziata tramite sostituzione: è il motore del **forward chaining** e del **backward chaining** su clausole di Horn.
- L'**unificazione** ($UNIFY(F1, F2)=\theta$) trova la sostituzione (idealmente il Most General Unifier) che rende identiche due formule; può fallire per conflitti di variabile, risolvibili con la standardizzazione separata.
- **Forward chaining**: guidato dai dati, corretto, completo e terminante su KB DATALOG (Horn senza funzioni); propaga sia verità che sostituzioni lungo un grafo AND-OR.
- **Backward chaining**: guidato dall'obiettivo, corretto ma incompleto (depth-first, rischio di loop), generalmente più efficiente del forward chaining.
- La **risoluzione** proposizionale può essere "liftata" a FOL (risoluzione binaria + fattorizzazione, con unificazione al posto dell'uguaglianza sintattica): è **refutation-complete**, cioè capace di rilevare sempre l'insoddisfaccibilità di $KB \wedge \neg Q$ quando la query Q è davvero conseguenza logica.
- Per applicare la risoluzione, la KB va portata in **CNF**: elimina $\Leftrightarrow/\Rightarrow$, sposta le negazioni all'interno, standardizza le variabili, **skolemizza** gli esistenziali, elimina gli universali residui, distribuisce $\vee$ su $\wedge$.
- La **skolemizzazione** sostituisce ogni variabile esistenziale con una **funzione di Skolem** i cui argomenti sono le variabili universali nel cui scope essa ricade (o con una **costante di Skolem** se non ricade nello scope di alcun universale). È un passo cruciale: sbagliarlo (es. usare EI ingenuamente sotto un $\forall$) porta a formule che fissano erroneamente un unico valore condiviso da tutti gli oggetti, invece di un valore che può variare al variare degli altri parametri.

Modulo 7 — Tassonomie/Ontologie e Pianificazione automatica

Parte A: Tassonomie e Ontologie

1. Cos'è una tassonomia

Una **tassonomia** è un'organizzazione gerarchica di categorie o concetti. L'esempio classico è la tassonomia del mondo animale (regno, phylum, classe, ordine, famiglia, genere, specie), ma il meccanismo si applica a qualsiasi dominio.

Il mattone concettuale è la **relazione di sottoclasse** (relazione **Is-a**): se `PalloneCalcio` è sottoclasse di `Pallone`, allora **tutte le istanze di `PalloneCalcio` sono anche istanze di `Pallone`**. Una tassonomia, in sostanza, è proprio l'organizzazione delle categorie che risulta dall'applicare in modo sistematico un insieme di regole di sottoclasse: è quindi un **albero** (o comunque una struttura gerarchica) di concetti collegati da relazioni **Is-a**.

Perché è utile: permette di caratterizzare le categorie tramite **proprietà** che valgono per tutte le istanze, ad esempio:

$\text{Member}(X, \text{Pallone}) \Rightarrow \text{Sferico}(X)$

Grazie alla relazione di sottoclasse, le istanze **ereditano** le proprietà delle sovraclassi: non serve ripetere $\text{Member}(X, \text{PalloneCalcio}) \Rightarrow \text{Sferico}(X)$, perché vale già per **ereditarietà**. Questo è il vero valore aggiunto di una tassonomia rispetto a un semplice elenco di fatti: evita ridondanza e permette inferenza automatica di proprietà non esplicitate.

Esempio analogo dalle slide: la tassonomia delle conifere, dove `Pinophyta` $\equiv$ `Conifera` è la classe radice e si scompone in famiglie (`Pinaceae`, `Araucariaceae`, ..., `Taxaceae`, `Cycadophyta`).

2. Decomposizioni, disgiunzione, partizione

Quando si scompone una categoria `C` in sottocategorie, non basta l'intuizione che siano "separate": occorre **rendere esplicita** questa informazione al motore inferenziale con proprietà formali su un insieme di categorie $S = \{X_1, \dots, X_n\}$:

- **Disjoint(S)**: le categorie in S non hanno istanze in comune $\forall x_i, x_j \in S, x_i \neq x_j \Rightarrow \text{Intersection}(x_i, x_j) = \{\}$
- **ExhaustiveDec(S, C)**: ogni istanza di `C` appartiene ad almeno una delle categorie di S (decomposizione esaustiva) $\forall I (\text{Member}(I, C) \Leftrightarrow \exists x_i \text{ Is-a}(x_i, C) \wedge \text{Member}(I, x_i))$
- **Partition(S, C)**: S è sia disgiunto sia esaustivo rispetto a `C` $\text{Partition}(S, C) \Leftrightarrow \text{Disjoint}(S) \wedge \text{ExhaustiveDec}(S, C)$

Esempio: le famiglie delle Pinophyta costituiscono una partizione, perché per costruzione tassonomica sono disgiunte (nessuna specie appartiene a due famiglie) ed esaustive (ogni conifera appartiene a una famiglia).

3. Relazioni strutturali: Part-of e Bunch-of

Oltre a Is-a, un altro tipo naturale di conoscenza è quella **strutturale**, cioè “di cosa è fatta una cosa”:

- Part-of(Leg, Table): le gambe sono parte del tavolo
- Part-of(Head, Body), Part-of(Corolla, Flower), ecc.

Part-of è transitiva: $\text{Part-of}(X, Y) \wedge \text{Part-of}(Y, Z) \Rightarrow \text{Part-of}(X, Z)$.

Le relazioni strutturali comprendono anche predicati dipendenti dal dominio (On-top, Implements, ...) e permettono di catturare formalmente la struttura tipica di una categoria di oggetti, ad esempio:

$\text{Flower}(x) \Rightarrow \exists \text{Corolla, Stelo } \text{Part-of}(\text{Corolla}, x) \wedge \text{Part-of}(\text{Stelo}, x) \wedge \text{On-top}(\text{Corolla}, \text{Stelo})$

Bunch-of (mucchio) serve quando si vuole dire che un oggetto è composto da parti **senza specificare le relazioni fra queste**: $\text{Bunch-of}(\{\text{Mela1}, \text{Mela2}, \text{Mela3}\})$ indica il mucchio delle tre mele. Definizione: $\forall x \text{ In}(x, s) \Rightarrow \text{Part-of}(x, \text{Bunch-of}(s))$. Da notare: s non è una categoria ma un insieme di elementi, e $\text{Bunch-of}(s) = s$.

(Le slide trattano anche le **misure** — predicati per esprimere quantità puntuali, es. $\text{Durata}(d) = \text{Ore}(24)$, o ordinamenti come $\text{Difficoltà}(\text{SISINT}) > \text{Difficoltà}(\text{BASIDATI})$ — segnate come materiale di lettura, minore rilevanza per l'esame.)

4. T-box e A-box

Un'ontologia si struttura sempre in due parti:

- **T-box** (terminological box): la parte **generale/intensionale**, fatta di definizioni, specializzazioni e proprietà (le “regole” del dominio).
- **A-box** (assertional box): la parte **estensionale**, contiene fatti su istanze specifiche, che devono essere coerenti con la T-box.

Esempio: $\text{T-box} \rightarrow \text{Madre}(X) \Rightarrow \text{Donna}(X)$; $\text{A-box} \rightarrow \text{Madre}(\text{Anna})$.

Questa distinzione è cruciale perché separa lo **schema concettuale** (cosa può esistere e come si relaziona) dai **dati concreti** (cosa esiste effettivamente), un po' come lo schema di un database rispetto ai suoi record.

5. Problematiche delle tassonomie/ontologie

- **Norma ed eccezioni:** molte proprietà valgono “di default” ma ammettono eccezioni (gli uccelli di solito volano, ma struzzi, pinguini, kiwi, emù no; i cani di solito hanno il pelo, ma il cane nudo peruviano no). Le eccezioni possono **cancellare** proprietà ereditate: una sottoclasse SC1 eredita la proprietà P della superclasse C; una sotto-sottoclasse SC2 può specializzarsi aggiungendo una proprietà Q; un'altra sotto-sottoclasse SC3 può invece essere un'**eccezione al default** e non avere P, pur essendo comunque sottoclasse di C.
- **Polisemia:** una stessa parola può denotare concetti diversi e appartenere ad alberi tassonomici differenti. Esempio: “cane” è sia un animale sia una costellazione (Canis Major) sia una persona vile sia un dente d'arresto meccanico. Il motore inferenziale non deve mischiare le tassonomie: non si vuole concludere che la costellazione Canis Major “ha il pelo”. Il problema aperto è: **come identificare i concetti in modo univoco?** (è uno dei motivi per cui le ontologie del web semantico usano identificatori URI globalmente univoci, si veda oltre).

6. Dalla tassonomia all'ontologia

Una tassonomia, essendo un **albero** (ogni concetto ha un'unica super-classe diretta lungo la gerarchia Is-a), è un caso *particolare* e più limitato di struttura. Una base di conoscenza descrittiva può però assumere una forma più generale, in cui i concetti e le relazioni fra essi formano un **grafo** invece che un albero: questo insieme più generale di concetti e relazioni si chiama **ontologia** (o **rete semantica**).

Tassonomia vs Ontologia: la tassonomia cattura *solo* le relazioni gerarchiche Is-a (e quindi ha una struttura ad albero); un'ontologia è più espressiva e può includere anche relazioni Part-of, relazioni di dominio specifiche (es. haMoglie, ospitatoDa), e in generale una rete arbitraria di concetti collegati da relazioni diverse (struttura a grafo). **Le tassonomie sono quindi un caso speciale di ontologia.**

I sistemi che usano ontologie seguono tipicamente lo schema: la **realtà/dati** viene astratta nell'**ontologia**, un **motore inferenziale** lavora sui fatti rappresentati e permette di **interrogare** il sistema e ottenere risposte.

Tipi di interrogazione tipici su un'ontologia:

1. Istanza appartiene a categoria? (*Fido è un mammifero?*)
2. Istanza gode di proprietà? (*Fido può volare?*)
3. Differenza fra categorie? (*Che differenza c'è fra rocce magmatiche e sedimentarie?*)
4. Identificazione di istanze (*Quali alberghi a tre stelle di Rimini offrono supporto tecnico ai ciclisti?*)

Queste interrogazioni sono spesso inserite in sistemi più ampi: categorizzazione di immagini, risposta a domande in linguaggio naturale, diagnosi medica da risultati di laboratorio, ecc.

7. Esempio applicativo: PROV (provenance)

PROV è un'ontologia del web semantico (W3C) che rappresenta, integra e traccia la **provenienza dei dati** (chi/cosa ha creato/modificato/usato una risorsa), utile ad esempio per verificare il rispetto della privacy o la liceità d'uso di materiale audio/video. I concetti chiave sono **Agente** (persona, organizzazione, software), **Attività** (uso/creazione di dati), **Entità** (dato/documento). Le slide mostrano un caso d'uso (evoluzione di un file di statistiche di crimini modificato da più giornalisti) codificato con predicati come wasGeneratedBy, used, wasDerivedFrom, wasControlledBy. Su una rappresentazione di questo tipo si possono automatizzare interrogazioni come "su cosa si basa questa risorsa?", "chi l'ha fatta?", "quali risorse derivano dalle stesse fonti?" (contenuto contrassegnato come facoltativo, utile solo per intuire un caso reale d'uso).

8. Semantic Web e linguaggi per rappresentare ontologie

Il **Semantic Web** (termine coniato da Tim Berners-Lee, Turing Award 2016) è l'estensione del web tradizionale in cui i contenuti pubblicati sono arricchiti da **metadati** che abilitano interpretazione, inferenza, interrogazione ed elaborazione automatica. I linguaggi sono standardizzati dal **W3C**.

RDF (Resource Description Framework)

- È il modello/linguaggio di rappresentazione **base** su cui poggiano altri linguaggi (OWL, SKOS per ontologie; FOAF per applicazioni sociali).
- La conoscenza è espressa tramite **triple** *soggetto – predicato – oggetto* (statement): il predicato mette in relazione soggetto e oggetto.
- Soggetto, predicato e oggetto sono **IRI** (internationalized resource identifier, tipo URL) — questo risolve proprio il problema di identificazione univoca dei concetti visto sopra (§5): ogni concetto ha un identificatore globale non ambiguo.
- **RDFS** (RDF Schema) permette di costruire tassonomie appoggiandosi a RDF.
- Un insieme di triple RDF forma un **grafo RDF**.
- Sintassi concreta: RDF è tipicamente serializzato in **XML** (<rdf:RDF>...</rdf:RDF>), ma esistono anche notazioni alternative come **Turtle** e **N3**.
- **SPARQL** è il linguaggio di interrogazione per dati RDF (analogo a SQL per i DB relazionali).

OWL 2 (Web Ontology Language)

Linguaggio dichiarativo del semantic web pensato specificamente per **definire ontologie** tramite classi, proprietà, individui e valori. Le ontologie OWL possono essere pubblicate sul web e riferite da altre ontologie per costruire basi di conoscenza più ricche e componibili.

OWL modella la conoscenza con tre tipi di elementi:

- **Entità:** elementi atomici che riferiscono oggetti del mondo reale (individui, classi, proprietà) — corrispondono a costanti (individui), predicati unari (classi), predicati binari (proprietà) della logica del primo ordine.
- **Assiomi:** affermazioni di base (es. `ClassAssertion(:Persona :Maria) = "Maria è una Persona"`).
- **Espressioni:** combinazioni di entità che costruiscono descrizioni complesse (es. intersezione di "medico" e "donna" per ottenere "donna medico").

Costrutti principali (in sintassi *Functional-Style*, una delle possibili sintassi OWL insieme a RDF/XML, Turtle, Manchester):

- `Declaration(NamedIndividual(:Maria)),` `Declaration(Class(:Persona)),`
`Declaration(ObjectProperty(:donna))`
- `ClassAssertion(:Persona :Maria):` Maria è istanza di Persona
- `SubClassOf(:Madre :Donna):` relazione di sottoclasse (Is-a)
- `EquivalentClasses(:Persona :Umano):` equivalenza fra classi (sottoclasse reciproca)
- `DisjointClasses(:Donna :Uomo):` le classi non condividono istanze
- `ObjectPropertyAssertion(:haMoglie :Giovanni :Maria):` lega due individui con una proprietà
- `SubObjectPropertyOf(:haMoglie :haConiuge):` gerarchia fra proprietà
- `ObjectPropertyDomain / ObjectPropertyRange:` dominio e codominio di una proprietà
- Costruttori per classi complesse: `ObjectIntersectionOf`, `ObjectUnionOf`, `ObjectComplementOf`
- Quantificazione: `ObjectSomeValuesFrom` (esistenziale, es. `Genitore = chi ha almeno un figlio Persona`), `ObjectAllValuesFrom` (universale, es. `PersonaFelice = tutti i figli sono felici, vero anche vacuamente se non si hanno figli`)
- **Datatype:** per vincolare i valori delle proprietà a un insieme (es. `DataPropertyRange(:haEta xsd:nonNegativeInteger)`)

OWL 2 non è un framework per basi di dati, anche se il vocabolario a volte richiama quello dei DB. Differenze semantiche fondamentali:

- DB: **closed world assumption** (un fatto non presente è falso) — OWL 2: **open world assumption** (un fatto non presente è semplicemente *sconosciuto/mancante*, non falso)
- OWL non impone che le uniche proprietà di un individuo siano quelle della sua classe
- Classi e proprietà possono avere definizioni multiple, distribuite anche su documenti diversi

OWL 2 non è un linguaggio di programmazione ma un linguaggio di **rappresentazione dichiarativa**: su di esso si applicano programmi di *reasoning* per rispondere a query.

FOAF (Friend Of A Friend): ontologia per collegare persone sul web, con classi come Agent, Person, Group, Organization e proprietà come name, knows, age, mbox.

9. Costruire un'ontologia: metodologia pratica

Data l'esigenza tipica (esporre dati di un DB in un formalismo su cui fare inferenza), il procedimento è:

1. **Identificazione dei concetti:** elencare i sostantivi rilevanti nel DB, dare a ciascuno etichetta e descrizione, poi (in una seconda passata) identificare le sottoclassi. Esempio (dominio canili): Cane (specie), Bassotto (sottospecie), Cucciolo (età ≤ 6 mesi), Orfano, TagliaMedia, Struttura (canile), ecc.
2. **Identificazione delle proprietà:** elencare le relazioni del DB (tipicamente verbi), con etichetta e descrizione. Esempio: `ospitatoDa`, `coloreManto`, `haDisabilità`, `haTatuaggio`, `haMicrochip`.

3. **Riuso:** verificare se esistono ontologie già definite riutilizzabili (anche parzialmente), es. la *Wildlife Ontology* della BBC.
4. **Scrittura formale:** codificare quanto sopra in un linguaggio per ontologie (RDF o OWL) — si ottiene la **T-box**.
5. **Annotazione dei dati:** si popola la **A-box** con le istanze concrete.
6. **Validazione e raffinamento** della base di conoscenza.
7. **Costruzione di un sistema di interrogazione:** applicazioni che sfruttano ontologia e dati annotati appoggiandosi a motori inferenziali (esempio nelle slide: un sistema che sostituisce consigli statici testuali su quale cane adottare con un sistema interrogabile basato sull'ontologia).

Strumenti citati: Protégé (editor/IDE per KB, Stanford), reasoner (CEL, FaCT++, HermiT, Pellet, RacerPro), Alignment API, OWLTools, Sesame, Tripliser, W3C RDF Validator, Apache Jena, AllegroGraph (triplestore).

Applicazioni citate: Linked (Open) Data, DBpedia, Creative Commons (licenze in RDF), FOAF, Press Association (notizie annotate semanticamente), beni culturali, musei, ambito legale/medico (es. Genoma), geografico, meteorologico, curricula transnazionali.

10. Relazioni fra ontologie

Quando esistono più ontologie (magari sviluppate indipendentemente) che descrivono domini simili o sovrapposti, è utile classificare formalmente il tipo di relazione fra loro:

- **Identical:** O1 e O2 sono **la stessa** ontologia (es. un'ontologia e la sua copia su un disco mirror — identità letterale, non solo concettuale).
- **Equivalent:** O1 e O2 condividono **vocabolario e assiomatizzazione** ma sono espresse in **linguaggi differenti** (es. la stessa concettualizzazione scritta una volta in **SKOS** e una volta in **RDF**). Concettualmente dicono la stessa cosa, cambia solo la "lingua" formale.
- **Extension:** O1 **estende** O2 quando tutti i simboli definiti in O2 sono preservati in O1 insieme a proprietà e relazioni, ma **non vale il viceversa** (O1 è O2 + qualcosa in più).
- **Weakly-Translatable:** data una ontologia sorgente Osource e una destinazione Odest, è possibile tradurre espressioni di Osource in espressioni di Odest, ma **con perdita di informazione**. Esempio: Osource ha Fruit → {Agrume → {Citron, Orange, Pamplemousse}, Pomme, Poire}; Odest ha solo Fruit → {Apple, Lemon, Orange}. Il mapping Pomme→Apple, Citron→Lemon, Orange→Orange, Fruit→Fruit funziona ma si perde l'informazione su Poire, Pamplemousse e sulla sotto-categorizzazione di Agrume.
- **Strongly-Translatable:** Osource è fortemente traducibile in Odest quando (a) il vocabolario è **totalmente mappabile**, (b) l'assiomatizzazione di Osource **vale** in Odest, (c) **non c'è perdita di informazione**, (d) **non si introducono inconsistenze**. È una traduzione "fedele", anche se Odest può comunque contenere concetti (es. Pamplemousse) che non hanno corrispondente in Osource — la condizione riguarda la direzione Osource→Odest, non il viceversa.
- **Approx-Translatable:** Osource è Weakly-Translatable in Odest e possono essere introdotte **inconsistenze**. Accade tipicamente quando concetti che dovrebbero corrispondere sono solo *affini*, non identici. Esempio dato: il coriandolo, a seconda della tradizione culinaria, è visto come affine al prezzemolo (si usano le foglie) o al pepe (si usano i semi) — è la stessa pianta ma con proprietà ontologiche differenti a seconda del riferimento.

L'esercizio pratico di costruire l'allineamento (*ontology alignment / matching*) fra due ontologie O1 e O2 consiste proprio nell'individuare queste corrispondenze fra concetti, che **in generale sono imperfette** (slide con esempio "Cosa/Oggetto, Macchina/Locomotore/Veicolo/Treno/Automobile..." per illustrare come concetti simili ma non identici in due tassonomie diverse possano essere allineati parzialmente).

Un contesto applicativo citato: gli standard **FIPA** (Foundation for Physical/Intelligent Agents) per agenti software prevedono un *ontology agent* dedicato che offre servizi di discovery di ontologie pubbliche, traduzione fra ontologie diverse, e risposta a query sulle differenze fra termini/ontologie — utile quando due agenti comunicano e devono condividere/riconciliare i rispettivi vocabolari concettuali.

? Domanda d'esame: Qual è la differenza fra una tassonomia e un'ontologia?

Una tassonomia è una struttura **gerarchica ad albero** costruita esclusivamente tramite relazioni Is-a (sottoclasse): ogni concetto eredita le proprietà del proprio "genitore" nella gerarchia. Un'ontologia è un concetto più generale: un **grafo** di concetti collegati da relazioni di natura diversa (Is-a, Part-of, relazioni di dominio come haMoglie o ospitatoDa, ecc.), non necessariamente organizzato ad albero. Ogni tassonomia è quindi un caso particolare (più semplice) di ontologia, ma non vale il viceversa.

? Domanda d'esame: Perché Is-a e Part-of non bastano a rappresentare qualunque tipo di conoscenza?

Perché sono relazioni essenzialmente **statiche**, adatte a descrivere *cosa è qualcosa* (categorizzazione) o *di cosa è fatto* qualcosa (composizione), ma non catturano il concetto di **azione** e di **cambiamento nel tempo**: non permettono di rappresentare che un agente compie un'azione che trasforma lo stato del mondo (esempio delle slide: l'inferenza ontologica pura non basta a decidere/rappresentare l'attraversare la strada). Per questo serve un formalismo diverso, orientato alle azioni e agli stati: il **Situation Calculus**, che è il ponte verso la Parte B — la pianificazione.

Parte B: Pianificazione automatica

1. Dalle ontologie alla pianificazione: perché serve un nuovo formalismo

Le relazioni Is-a e Part-of descrivono conoscenza statica. Molte applicazioni reali dei sistemi di inferenza, invece, richiedono di **decidere quali azioni eseguire** per raggiungere un obiettivo: questo è il compito della **pianificazione** (planning).

Pianificare significa costruire una **sequenza di azioni** che, applicata a partire da uno stato iniziale, soddisfa un certo obiettivo (goal). Un problema di pianificazione specifica: gli elementi di interesse del mondo (stato), le azioni disponibili, e l'obiettivo da raggiungere.

Il mondo è descritto da un insieme di variabili, tipicamente nel linguaggio **PDDL** (*Planning Domain Definition Language*, lo standard de facto per descrivere problemi di pianificazione). Uno **stato** è una congiunzione di **atomi ground** (senza variabili né funzioni), es. $At(Camion1, Bari) \wedge At(Camion2, Lecce)$. Le azioni sono descritte in modo **schematico** (parametriche) e hanno un impatto **limitato** sul mondo: in un dato stato solo un sottoinsieme di azioni è applicabile, e di queste ne viene scelta ed eseguita **una sola per volta**. Un aspetto pratico rilevante: se l'azione scelta viene davvero eseguita nel mondo reale, **potrebbe non essere possibile fare backtracking** (a differenza della ricerca "sulla carta").

2. Differenza rispetto alla ricerca classica

Nella ricerca nello spazio degli stati "classica" (vista in altri moduli, es. A*), si cerca un cammino fra stato iniziale e stato goal muovendosi fra stati rappresentati in modo essenzialmente atomico o con poca struttura interna, valutando ogni azione/operatore individualmente rispetto allo stato corrente.

Nella pianificazione, invece:

- Gli stati e le azioni hanno una **rappresentazione logica strutturata** (predicati, precondizioni, effetti), non atomica: questo permette ragionamenti più espressivi (es. dedurre che un'azione non è applicabile senza dover enumerare esplicitamente lo stato).
- L'obiettivo è tipicamente **complesso** e viene tipicamente **scomposto in sottoobiettivi** (si veda sotto), sfruttando la struttura logica per decidere l'ordine e l'interazione fra le azioni necessarie a soddisfare ciascun sottoobiettivo.

- Si presta esplicitamente attenzione al problema di **rappresentare gli effetti delle azioni** (assiomi di applicabilità/effetto, frame problem — vedi sotto), aspetto che nella ricerca “classica” è dato per scontato tramite la funzione successore.

3. Il Situation Calculus: fondamento logico della pianificazione

Il **Situation Calculus** è la rappresentazione logica (in FOL) su cui si basa storicamente il ragionamento su azioni. Introduce i concetti di:

- **Azione:** qualcosa che viene compiuto e influenza il mondo. In logica non è un predicato ma una **funzione** che restituisce un oggetto del dominio (un’azione è quindi un oggetto “intangibile”). Esempio: $\text{Move}(R, L1, L2)$ denota l’azione di spostare R da L1 a L2.
- **Situazione:** lo stato risultante dall’esecuzione di una sequenza di azioni.
- **Fluente:** proprietà/relazione che può **cambiare valore** (fluire) con l’esecuzione delle azioni; ha sempre come parametro la situazione, es. $\text{Holds}(\text{At}(R, \text{Loc}), s)$, $\text{Adjacent}(R1, R2, s)$.
- **Predicato/funzione atemporale (eterno):** il cui valore **non** dipende dalle azioni eseguite.

Il tempo non è rappresentato esplicitamente, ma scandito dalla sequenza di eventi: si parte da una situazione iniziale $S0$ e si applicano azioni $a1, a2, a3...$ generando $S1, S2, S3...$

La funzione $\text{Do}(\text{Azioni}, S)$ restituisce la situazione raggiunta applicando la sequenza Azioni a partire da S:

$\text{Do}([], s) = s$

$\text{Do}([a \mid \text{resto}], s) = \text{Do}(\text{resto}, \text{Risultato}(a, s))$

Nota fondamentale: **due situazioni sono identiche solo se derivano dallo stesso stato iniziale tramite la stessa sequenza di azioni** — una situazione è cioè identificata dalla sua “storia”: $\text{Do}(\text{Azioni1}, S1) = \text{Do}(\text{Azioni2}, S2) \Leftrightarrow (\text{Azioni1}=\text{Azioni2}) \wedge (S1=S2)$.

Tramite Do un agente può fare **proiezione**, cioè ragionare sugli effetti futuri delle azioni: verificare se un piano attraversa solo situazioni con certe proprietà (vincolo da mantenere lungo tutto il percorso, es. “non rimanere mai a secco di carburante”), oppure pianificare per raggiungere una situazione con una proprietà finale specifica (goal, es. “assemblare la bicicletta”).

Rappresentazione delle azioni tramite due tipi di assiomi:

- **Assioma di Applicabilità:** $\forall \text{params}, s \text{ Applicable}(\text{Action}(\text{params}), s) \Leftrightarrow \text{Precond}(\text{params}, s)$ — un’azione è applicabile in una situazione se e solo se valgono le sue precondizioni. Esempio: $\text{Applicable}(\text{go}(X,Y), S) \Leftrightarrow \text{At}(X,S) \wedge \text{Adjacent}(X,Y)$.
- **Assioma di Effetto:** $\forall \text{params}, s \text{ Applicable}(\text{Action}(\text{params}), s) \Rightarrow \text{Effects}(\text{params}, \text{Result}(\text{Action}(\text{params}), s))$ — specifica cosa diventa vero nello stato risultante. Esempio: $\text{Applicable}(\text{go}(X,Y), S) \Rightarrow \text{At}(Y, \text{Result}(\text{go}(X,Y), S))$.

Esempio guida — mondo dei blocchi: l’azione $\text{Move}(X,Y,Z)$ sposta il blocco X da Y a Z.

- Applicabilità: $\text{Applicable}(\text{Move}(x,y,z), s) \Leftrightarrow \text{Clear}(x,s) \wedge \text{Clear}(z,s) \wedge \text{On}(x,y,s) \wedge x \neq z \wedge y \neq z \wedge x \neq \text{Table}$ (X e Z devono essere liberi in cima, X deve essere effettivamente su Y, non si può spostare un blocco su se stesso, né “sopra” il tavolo con la stessa semantica di un blocco).
- Effetto: $\text{Applicable}(\text{Move}(x,y,z), s) \Rightarrow \text{On}(x,z, \text{Result}(\dots)) \wedge \text{Clear}(y, \text{Result}(\dots))$.

Da stato iniziale + assiomi di applicabilità + assiomi di effetto (la “KB”) si possono **derivare** nuovi fatti sullo stato risultante, es. dopo $\text{Move}(A,B,D)$ si deriva $\text{On}(A,D, \dots)$ e $\text{Clear}(B, \dots)$.

4. Il Frame Problem

Frame problem (problema della cornice): le azioni hanno tipicamente un impatto **limitato** sul mondo — come rappresentare tutto ciò che **non** viene modificato da un'azione?

Il problema è che dalla sola conoscenza di stato iniziale + assiomi di applicabilità/effetto **non si può derivare** ciò che rimane invariato: si può ragionare su cosa è cambiato, ma nulla dice esplicitamente al sistema che il resto è rimasto com'era. Esempio: dopo $\text{Move}(A, B, D)$, sapevamo $\text{On}(B, C, s)$ nello stato iniziale, ma nulla garantisce che il sistema possa inferire $\text{On}(B, C, s')$ nello stato risultante s' , a meno di dirglielo esplicitamente.

Due soluzioni proposte nelle slide:

1. **Enumerare esplicitamente ciò che non cambia** (assiomi di frame): per ogni azione e per ogni fluente non toccato da quell'azione si scrive un assioma tipo $\forall \text{params}, \text{vars}, s \text{ fluent}(\text{vars}, s) \wedge \text{params} \neq \text{vars} \Rightarrow \text{fluent}(\text{vars}, \text{Result}(\text{Action}(\text{params}), s))$ **Problema pratico:** bisogna scrivere un assioma di frame per **ogni combinazione azione × fluente non toccato** — con molte proprietà (colore, materiale, ecc.) il numero di assiomi esplode rapidamente, rendendo l'approccio poco scalabile.
2. **Evitare l'enumerazione con un assioma di stato successore:** si introduce un unico schema generale che dice al sistema che ciò che non è esplicitamente indicato come effetto rimane implicitamente vero: $\text{Azione applicabile} \Rightarrow (\text{fluente vero nella situazione risultante} \Leftrightarrow (\text{l'azione lo rende vero} \vee (\text{era vero} \wedge \text{l'azione non l'ha reso falso})))$ Questo compatta in un solo schema, per ciascun fluente, sia gli effetti positivi sia la "persistenza per default" di ciò che non è toccato — evitando l'esplosione combinatoria della soluzione 1.

Infine, il Situation Calculus è completato da assiomi di **unique action names**: azioni con nomi diversi sono oggetti diversi ($A_i(x, \dots) \neq A_j(y, \dots)$), e due usi dello stesso nome d'azione sono uguali se e solo se hanno argomenti uguali ($A(x_1, \dots, x_n) = A(y_1, \dots, y_n) \Rightarrow x_1=y_1 \wedge \dots \wedge x_n=y_n$) — condizioni tecniche necessarie perché l'inferenza logica funzioni correttamente senza ambiguità.

5. Perseguire goal complessi: scomposizione in sottogoal

L'approccio tipico alla pianificazione di un obiettivo complesso è **scomporlo in sottoobiettivi** e perseguirli, in linea di principio, uno alla volta.

Esempio (mondo dei blocchi): stato iniziale con A e B sul tavolo, C sopra B (letto dalle slide: "A B" sul tavolo con C sopra uno di essi); goal = "A su B su C" (pila A-B-C). Il goal si scompone in due sottogoal:

- Sottogoal 1: A su B
- Sottogoal 2: B su C

L'idea è risolvere prima un sottogoal, poi l'altro, in sequenza.

6. L'anomalia di Sussman: quando i sottogoal interagiscono

Non sempre il perseguimento dei sottoobiettivi è sequenzializzabile in modo indipendente. Questo è il fenomeno noto come **anomalia di Sussman**, un classico esempio di *interazione fra sotto-obiettivi* nella pianificazione.

Stato iniziale: C sul tavolo da solo, A sopra B (pila A/B) sul tavolo. Goal: pila A su B su C.

Il problema è che i due sottogoal ("A su B" e "B su C") sono **in conflitto** se perseguiti nell'ordine sbagliato:

- Se perseguo prima **"B su C"**: per liberare B devo spostare A altrove, quindi **disfo** la pila A/B che già avevo parzialmente — per ottenere B su C devo disfare la struttura A/B.

- Se perseguo prima **“A su B”**: metto A sopra B, ma poi per ottenere “B su C” dovrei spostare l’intera pila A/B, e quindi devo di nuovo **disfare** ciò che avevo appena costruito (togliere A da sopra B per poter mettere B su C, per poi rimettere A sopra B).

In entrambi i casi, risolvere un sottogoal **“disfa”** (annulla) i progressi fatti per l’altro: la scomposizione **sequenziale e indipendente** in sottogoal non funziona qui, perché i sottogoal **non sono indipendenti** — ci sono interazioni fra loro (l’ordine di esecuzione conta, e un ordine “ingenuo” porta a lavoro ripetuto o a cicli).

Soluzione — interleaving dei passi: la soluzione efficiente richiede di **intercalare** (fare interleaving) i passi provenienti dai due sottopiani, invece di completare un sottogoal per intero prima di iniziare l’altro. Sequenza corretta (una delle possibili):

1. Sposto C dal tavolo... (passo verso “B su C”, cioè libero/posiziono C)
2. Sposto A da sopra B (passo di preparazione)
3. Sposto B su C (passo di “B su C”)
4. Sposto A su B (passo di “A su B”)

In sintesi: si esegue prima l’azione che porta B su C (dopo aver tolto A da B), e **solo dopo** si posiziona A su B, ottenendo la pila finale A-B-C senza dover disfare nulla. Questo dimostra che un planner efficace deve poter **intrecciare** le sequenze di azioni relative a sottogoal diversi, non semplicemente concatenarle.

7. Limiti pratici del Situation Calculus

- Il Situation Calculus ha permesso di usare la **logica del primo ordine (FOL)** per formalizzare rigorosamente il problema della pianificazione, ed è stato storicamente **fondamentale** per definirne le basi teoriche.
- Nella **pratica**, però, **non è molto usato**, perché **non esistono euristiche efficienti** che guidino efficacemente la ricerca della soluzione nello spazio (molto grande) delle sequenze di azioni possibili. I planner moderni usano rappresentazioni più specializzate (es. PDDL con algoritmi dedicati, planning graph, euristiche ad hoc) proprio per rendere la ricerca trattabile.
- La pianificazione (planning) come argomento a sé stante viene approfondita nel corso magistrale **“IA e Laboratorio”** della laurea in Intelligenza Artificiale e Sistemi Informatici — qui è trattata solo nei suoi fondamenti concettuali e logici.

? Domanda d’esame: Cos’è l’anomalia di Sussman e cosa dimostra?

È l’esempio classico (mondo dei blocchi: stato iniziale A su B, C isolato; goal: pila A su B su C) che mostra come la scomposizione di un obiettivo complesso in sottoobiettivi **non sia in generale risolvibile perseguendo i sottoobiettivi in sequenza indipendente**: risolvere un sottogoal (es. “B su C”) può richiedere di disfare i progressi fatti per un altro sottogoal (es. “A su B”) già raggiunto, e viceversa. Dimostra quindi che i sottoobiettivi possono **interagire** fra loro, e che un planner corretto deve poter fare **interleaving** dei passi provenienti da piani diversi invece di eseguirli a blocchi separati e sequenziali.

? Domanda d’esame: Cos’è il frame problem e come si risolve?

È il problema di rappresentare esplicitamente **ciò che un’azione NON modifica**, dato che dalla sola conoscenza di stato iniziale, assiomi di applicabilità e assiomi di effetto non si può derivare automaticamente la persistenza delle proprietà non toccate da un’azione. Si risolve in due modi: (1) enumerando esplicitamente un assioma di frame per ogni coppia azione/fluento non toccato (soluzione corretta ma che esplode combinatoriamente al crescere di azioni e proprietà); (2) tramite un **assioma di stato successore** unico per fluente, che stabilisce per default che un fluente resta vero nella situazione risultante se lo era prima e l’azione non lo ha reso falso (o se l’azione lo rende vero) — soluzione compatta che evita l’enumerazione esplicita.

Riepilogo e punti chiave

Parte A — Tassonomie e Ontologie

- **Tassonomia** = organizzazione gerarchica (ad albero) di concetti tramite relazioni **Is-a** (sottoclasse): le istanze di una sottoclasse ereditano le proprietà della superclasse, evitando ridondanza.
- Su un insieme di sottocategorie si possono definire formalmente le proprietà **Disjoint**, **ExhaustiveDec**, **Partition** per rendere esplicita al motore inferenziale l'organizzazione voluta.
- Relazioni **strutturali** (Part-of, transitiva; Bunch-of per aggregati senza struttura interna) affiancano le relazioni Is-a per descrivere la composizione degli oggetti.
- Ogni ontologia si divide in **T-box** (schema/definizioni, intensionale) e **A-box** (istanze/fatti, estensionale).
- Problemi aperti delle tassonomie: gestire **eccezioni** alla norma (cancellazione di proprietà ereditate) e la **polisemia** (una parola → concetti/alberi tassonomici diversi).
- Un'ontologia generalizza la tassonomia: da albero (solo Is-a) a **grafo** (relazioni arbitrarie, incluse Part-of e relazioni di dominio). Ogni tassonomia è un'ontologia, non viceversa.
- Il **Semantic Web** standardizza (W3C) linguaggi per ontologie: **RDF** (triple soggetto-predicato-oggetto, IRI come identificatori univoci, grafo RDF, sintassi XML/Turtle/N3), **RDFS**, **OWL 2** (classi, proprietà, individui, assiomi come SubClassOf/EquivalentClasses/DisjointClasses, quantificatori esistenziale/universale), **SPARQL** (query), **FOAF** (ontologia sociale).
- Costruire un'ontologia: identificare concetti → identificare proprietà → riusare ontologie esistenti → formalizzare (T-box) → annotare dati (A-box) → validare → costruire sistema di interrogazione.
- OWL 2 **non** è un DB: usa **open world assumption** (a differenza della closed world assumption dei DB) e ammette conoscenza incompleta/distribuita.
- **Relazioni fra ontologie**: Identical (stessa ontologia), Equivalent (stesso contenuto, linguaggio diverso, es. SKOS vs RDF), Extension (una include l'altra + di più), Weakly-Translatable (traduzione con perdita), Strongly-Translatable (traduzione fedele, senza perdita né inconsistenze), Approx-Translatable (traduzione debole con possibili inconsistenze per concetti solo affini). Il matching/allineamento fra ontologie è in generale imperfetto.
- Is-a e Part-of **non bastano** a rappresentare le **azioni**: serve un formalismo dedicato → ponte verso la pianificazione.

Parte B — Pianificazione automatica

- **Pianificare** = costruire una sequenza di azioni che porta da uno stato iniziale a uno stato che soddisfa il goal; si differenzia dalla ricerca classica per la rappresentazione logica strutturata di stati/azioni (precondizioni/effetti) e per l'attenzione a scomposizione e interazione dei sottoobiettivi.
- Il **Situation Calculus** fornisce il fondamento logico (FOL): Azione (funzione), Situazione, Fluente (proprietà parametrizzata sulla situazione), predicati atemporal; funzione $Do(Azioni, S)$ per il concatenamento di azioni; assiomi di **Applicabilità** (precondizioni) e di **Effetto** (conseguenze), illustrati con l'esempio del mondo dei blocchi ($Move(X, Y, Z)$).
- Il **frame problem** riguarda la rappresentazione di ciò che un'azione **non** modifica: risolvibile per enumerazione esplicita (poco scalabile) o con l'**assioma di stato successore** (soluzione compatta standard).
- La scomposizione di un goal in **sottogoal** è l'approccio base alla pianificazione, ma i sottogoal possono **interagire**: l'**anomalia di Sussman** è l'esempio canonico in cui risolvere un sottogoal disfa i progressi di un altro, e la soluzione richiede **interleaving** dei passi dei diversi (sotto-)piani anziché una sequenza rigida.
- Il Situation Calculus è stato storicamente fondamentale ma è **poco usato in pratica** per mancanza di euristiche efficienti; la pianificazione vera e propria è approfondita in corsi successivi (IA e Laboratorio, magistrale).

Modulo 8: Machine Learning — Classificazione, Alberi di Decisione, Reti Neurali

Materiale parzialmente tratto dalle slide di Cristina Baroglio, in parte associate al libro *Introduction to Data Mining* di Tan, Steinbach e Kumar.

Il problema della classificazione

Le classi non sono “naturali”

Un punto concettuale che l’insegnamento sottolinea fin da subito: **le classi non esistono in natura**. Non sono insite negli esempi, non sono raggruppamenti naturali inequivocabili: sono **definite a tavolino** da chi costruisce il sistema, in funzione dello scopo per cui si vuole costruire il predittore. Lo stesso insieme di oggetti (es. dei fiori) può essere raggruppato per colore, per forma, per “vero/finto” — dipende da cosa interessa a chi progetta il classificatore. Questo è un’osservazione importante perché ricorda che **il sistema non ha altra conoscenza oltre ai dati** e impara esattamente quello che c’è nei dati per come sono etichettati, non quello che il progettista *pensa* di aver messo nei dati.

Definizione del problema

Dati: - **Esempi** (istanze/record): gli oggetti da classificare (es. fiori, animali) - **Categorie o classi**: le etichette di appartenenza (es. mammifero/rettile/pesce)

Obiettivo: costruire una **rappresentazione astratta (modello)** che permetta di associare correttamente nuove istanze alla classe (o alle classi) di appartenenza.

Si parla di **apprendimento supervisionato** quando gli esempi da cui si astraggono le definizioni delle classi hanno già associata la classe corretta (l’etichetta).

Il problema si scompone in tre sotto-problemi: 1. **Rappresentazione dei dati** 2. **Analisi dei dati e costruzione delle definizioni** (il modello) 3. **Utilizzo della conoscenza acquisita**

Schema generale: training set, modello, test set

```
DATI TRAINING SET  --induzione-->  MODELLO: classe = f(descrizione)  --deduzione-->  CLASSE
                                     ↑
DATI TEST SET      ----->| (usato per la VALUTAZIONE del modello)
```

- **Training (learning) set:** la collezione di dati usati per l'apprendimento. Ogni istanza è una tupla (x, y) dove x è una tupla di valori di attributi descrittivi e y è la classe di appartenenza.
- Il processo di **induzione** (dai dati al modello) costruisce il modello partendo dal training set.
- Il processo di **deduzione** applica il modello a nuove descrizioni per ottenerne la classe.
- Il **test set** serve a valutare la bontà del modello appreso, applicandolo a dati non usati in fase di apprendimento e confrontando la classe predetta con quella reale.
- Una volta validato, il modello viene messo in **uso** per classificare nuovi dati.

Esempio di training set (dataset zoo-like): ogni riga è un animale (istanza), le colonne sono attributi descrittivi (temperatura corporea, copertura della pelle, viviparo, ecc.) e l'ultima colonna è la **classe target** (es. mammifero, rettile, pesce, uccello, anfibio).

Come classe si usano tipicamente attributi binari (sì/no, vero/falso) oppure etichette nominali categoriche (es. mammifero, pesce, ...).

Modello predittivo vs modello descrittivo

Modello predittivo	Modello descrittivo
Usato per predire la classe di istanze ignote, non viste in fase di apprendimento Es. data la descrizione di una salamandra si applicano le regole apprese per decidere la classe	Usato come strumento esplicativo che evidenzia quali caratteristiche distinguono le categorie Esprime sinteticamente delle descrizioni senza dover ragionare sugli esempi grezzi (es. "i mammiferi hanno sangue caldo e di solito non sono acquatici")

Valutazione: matrice di confusione, accuratezza, error rate

La qualità di un modello si valuta sperimentalmente: lo si usa per classificare le istanze del test set e si confronta l'esito con la classe reale.

Matrice di confusione (caso a 2 classi):

	classe predetta 1	classe predetta 2
classe reale 1	f11 (corretto)	f12 (errore)
classe reale 2	f21 (errore)	f22 (corretto)

- **Accuratezza** = $(f11 + f22) / (f11 + f22 + f12 + f21)$ — predizioni corrette su predizioni totali
- **Error rate** = $(f12 + f21) / (f11 + f22 + f12 + f21)$ — predizioni sbagliate su predizioni totali

Esempio numerico (lattina vs altro oggetto): accuratezza = $(18+19)/40 = 92.5\%$, error rate = $3/40 = 7.5\%$.

Matrice dei costi

Non tutti gli errori hanno lo stesso peso. La **matrice dei costi** assegna un costo a ciascuna cella della matrice di confusione (costi degli errori e costi delle valutazioni corrette, che possono anche essere negativi, cioè un guadagno).

Esempio classico: è più grave dire a un malato che è sano (falso negativo, costoso) o dire a una persona sana che è malata (falso positivo, meno grave ma comunque un costo)? Il costo complessivo si calcola sommando, cella per cella, $\text{costo}_{ij} \times \text{frequenza}_{ij}$ della matrice di confusione (es. Costo = $-1 \times 18 + 50 \times 2 + 10 \times 1 + 0 \times 19 = 92$).

? **Domanda d'esame:** perché un'accuratezza del 99.9% non è automaticamente sinonimo di "ottimo classificatore"? **Risposta:** perché l'accuratezza va sempre letta insieme alla distribuzione delle classi nel dataset. Se il learning set è fortemente sbilanciato (es. 9990 istanze di classe A e solo 10 di classe B), un classificatore banale che risponde sempre "classe A" (regola `if TRUE then A`) ottiene un'accuratezza del 99.9% pur non avendo imparato nulla di utile: non ha individuato nessun pattern discriminante, si limita a sfruttare lo sbilanciamento delle classi. Bisogna quindi guardare anche altre metriche (matrice di confusione completa, per-classe) e la composizione del dataset.

Insidie pratiche nella costruzione del classificatore

Alcuni errori tipici nella raccolta/uso dei dati (esempio guida: riconoscere "frutta" da altre cose usando solo foto di mele → il modello impara a riconoscere solo mele, non la frutta in generale). Cause tipiche: - **Dati difficili da reperire** → si usano dataset di comodo, non rappresentativi - **Fretta / risorse limitate** disponibili (es. usare solo un archivio fotografico locale) - **Bias mentale** di chi costruisce il dataset (se vivo circondato da mele, non mi viene naturale includere uva o ciliegie) - **Attribuire all'algoritmo capacità umane:** un umano generalizza usando moltissima conoscenza di base senza accorgersene (es. sa che un frutto nasce dall'impollinazione di un fiore); un algoritmo non ha questa conoscenza, impara solo dai dati che vede.

Applicazioni ad alto impatto in cui questi errori sono particolarmente rischiosi: riconoscimento facciale, concessione di mutui, assicurazioni sanitarie, applicazioni militari (riconoscimento automatico di bersagli), agricoltura di precisione (see and spray). Da qui il tema della **responsabilità etica**: nel caso della guida autonoma (variante del *trolley problem*), chi è responsabile della decisione presa da un agente autonomo? Si parla di *problem of the many hands*: la responsabilità è distribuita fra molti attori (progettisti, produttori, chi addestra il modello, chi lo utilizza), rendendo difficile individuare un singolo responsabile.

Rote learning (apprendimento meccanico) — un "non-modello"

Prima di introdurre gli alberi di decisione, le slide presentano il *rote learning* come caso limite/degenere di apprendimento: - **Strategia:** memorizzare semplicemente tutte le istanze del training set. Non esiste un vero modello: il sistema "ricorda" i casi ma non generalizza. - **Uso:** data una nuova istanza, cerca un'istanza identica in memoria; se la trova, restituisce la classe corrispondente. - Se non trova un'istanza identica, cerca istanze **simili** (misura di distanza = misura di similitudine, "vicino = simile"): - se le istanze simili trovate hanno tutte la stessa classe, quella è l'output; - se le classi sono diverse, serve una strategia di composizione, ad esempio: - **votazione a maggioranza:** ogni corrispondenza vale un voto, vince la classe più votata; - **votazione pesata:** il peso del voto dipende dalla distanza fra le istanze (più vicino = peso maggiore). I pesi vengono calcolati, usati e poi dimenticati (non sono persistenti, a differenza dei pesi di una rete neurale).

Questo approccio anticipa concettualmente algoritmi come k-NN, ma qui serve soprattutto a introdurre il concetto di **generalizzazione**: algoritmi di apprendimento diversi producono modelli di tipo diverso (alberi di decisione → albero; sistemi a regole → insiemi di regole if-then; reti neurali → matrici di numeri; apprendimento per rinforzo → distribuzioni di probabilità e matrici di numeri).

Alberi di decisione

Cosa sono

Gli alberi di decisione (*decision tree*, DT) sono strumenti di supporto alle decisioni basati su modelli strutturati ad albero, usati storicamente per definire strategie mirate al raggiungimento di un obiettivo (es. la struttura di navigazione di un negozio online: "ti interessa un libro, un film o un gioco?").

Applicati alla classificazione: si induce da esempi un albero di decisione che, dato un nuovo record, permette di predirne la classe seguendo un percorso di test sui valori degli attributi fino a una foglia.

Struttura

- **Nodi interni:** rappresentano un **test** su un attributo descrittivo; da ogni nodo interno partono rami corrispondenti ai possibili esiti/valori del test.
- **Foglie:** rappresentano una **decisione**, cioè l'assegnazione a una classe.
- Un percorso dalla radice a una foglia corrisponde a una sequenza di scelte/test e porta a una conclusione (classe).

Esempio concreto — dataset **Iris** (3 classi: *setosa*, *versicolor*, *virginica*; attributi continui: petal width, petal length, sepal width, sepal length):

```

Petal width ≤ 0.6?
├─ sì → IRIS SETOSA
└─ no → Petal width ≤ 1.7?
      ├─ no → IRIS VIRGINICA
      └─ sì → Petal length ≤ 4.9?
            ├─ sì → IRIS VERSICOLOR
            └─ no → Petal width ≤ 1.5?
                  ├─ sì → IRIS VIRGINICA
                  └─ no → IRIS VERSICOLOR
  
```

Algoritmi per apprendere alberi di decisione

Le slide citano diversi algoritmi: **Algoritmo di Hunt** (lo schema ricorsivo di base, su cui si fondano i successivi), **ID3**, **C4.5**, **CART**.

L'algoritmo di Hunt

L'albero viene costruito **ricorsivamente**, suddividendo il learning set in sottoinsiemi via via più "puri" (cioè sempre più concentrati su una singola classe).

Notazione: - D_t = sottoinsieme del learning set associato al nodo t - $y = \{y_1, y_2, \dots, y_c\}$ = insieme delle etichette di classe possibili

Procedura ricorsiva (per ogni nodo t da elaborare):

1. **Caso base:** se tutte le istanze in D_t appartengono alla stessa classe y_t , allora il nodo t diventa una **foglia** etichettata con la classe y_t .
2. **Passo ricorsivo:** altrimenti, si sceglie un attributo tra quelli descrittivi e si produce un **nodo figlio per ogni possibile valore** dell'attributo. A ciascun nodo figlio si associano le istanze del padre per le quali l'attributo assume il valore corrispondente. Si richiama la procedura ricorsivamente su ciascun figlio.

Note importanti sull'algoritmo: - *Nota 1:* se una combinazione di valori non è rappresentata da nessuna istanza del training set, a quel ramo/nodo si associa una **classe di default** (se prevista). - *Nota 2:* se tutte le istanze associate a un nodo hanno **tuple identiche** (stessi valori di attributi) ma classi diverse (situazione di **non-determinismo** nei dati), il nodo non può essere ulteriormente scisso: diventa una foglia, etichettata con la classe più rappresentata (maggioranza). - *Nota 3:* resta da definire **quando fermarsi** nella costruzione dell'albero (criteri di arresto). - *Nota 4:* resta da definire **come si sceglie l'attributo di split** ad ogni passo (misure di impurità, vedi sotto).

Strategia greedy e problemi aperti

La costruzione dell'albero segue una **strategia greedy**: ad ogni passo si seleziona l'attributo di split che massimizza (localmente) una misura di riferimento, senza backtracking. I problemi da risolvere sono: 1. Come specificare la condizione di test sui diversi tipi di attributo (binari, nominali, ordinali, continui)? 2. Come determinare quale sia lo split migliore? 3. Quando fermarsi nella costruzione dell'albero?

Tipi di split in base al tipo di attributo

- **Attributi binari** (es. "viviparo: sì/no"): lo split produce esattamente 2 figli, uno per ciascun valore.
- **Attributi nominali** (valori su un insieme finito di etichette $\{L_1, \dots, L_n\}$, senza ordine):
 - *split multivalore*: un figlio per ciascun valore possibile dell'attributo (es. "residenza": Asti, Torino, Ivrea, ...).
 - *split binario*: un figlio corrisponde a un valore (o a un sottoinsieme di valori), l'altro raccoglie il resto. Se l'attributo ha k valori possibili, esistono $2^{(k-1)} - 1$ possibili raggruppamenti binari alternativi.
- **Attributi ordinali** (es. small < medium < large < extralarge): si possono fare split binari o multivalore, ma il raggruppamento dei valori **deve rispettare l'ordinamento** (es. {small, medium} vs {large, extralarge} è corretto; {small, large} vs {medium, extralarge} è scorretto perché "spezza" l'ordine).
- **Attributi continui**:
 - *split binario*: si individua un valore soglia v e si testa $A \leq v$ vs $A > v$ (es. "Altezza ≤ 1.7 ").
 - *split multivalore*: si individuano più soglie v_i e si producono test del tipo $v_i \leq A < v_{i+1}$. Questo richiede di **discretizzare** la variabile continua individuando un numero finito di intervalli significativi.

Quale attributo scegliere per lo split? Misure di impurità

Non tutti gli ordini di split sono equivalenti: la scelta dell'attributo (e dell'ordine degli split) incide sulla dimensione e la qualità dell'albero risultante.

Criterio generale (Rasoio di Occam): a parità di prestazioni (stessa accuratezza/error rate), si preferiscono alberi più compatti, cioè che richiedono un numero minore di test. A parità di assunzioni, la spiegazione più semplice è da preferire.

Idea intuitiva: sono preferiti gli split che producono nodi figli con **minore confusione**, cioè con **grado di purezza maggiore** (più le istanze di un nodo appartengono alla stessa classe, meno "confuso" è il nodo, più è facile prevedere correttamente la classe di un elemento estratto a caso da quel nodo).

Per formalizzare questa idea si definisce, per un nodo t , $p(i|t)$ = probabilità che un elemento estratto casualmente dall'insieme associato al nodo t sia di classe i . Su questa distribuzione di probabilità si costruiscono le **misure di impurità**:

Entropia

$$\text{Entropia}(t) = - \sum_{i=0}^{c-1} p(i|t) \cdot \log_2 p(i|t)$$

(con la convenzione $0 \cdot \log_2(0) = 0$).

Proprietà (caso a 2 classi): - Distribuzioni (0,1) o (1,0): **purezza massima**, nessuna confusione → Entropia = $-0 \cdot \log_2(0) - 1 \cdot \log_2(1) = 0$ (valore minimo). - Distribuzione (0.5, 0.5): **massima confusione** → Entropia = $-0.5 \cdot \log_2(0.5) - 0.5 \cdot \log_2(0.5) = 1$ (valore massimo per 2 classi).

L'entropia è quindi 0 quando il nodo è puro, ed è massima quando le classi sono equidistribuite.

Gini index

Citato tra le misure alternative di impurità insieme a entropia ed errore di classificazione (le slide non ne riportano la formula esplicita, ma è la misura tipicamente usata dall'algoritmo **CART**; nella pratica standard: $\text{Gini}(t) = 1 - \sum p(i|t)^2$).

Errore di classificazione (misclassification error)

Citato come terza misura alternativa di impurità disponibile per la selezione dello split.

Calcolo del guadagno (gain) di uno split

Per confrontare split alternativi si calcola quanto uno split riduce l'impurità rispetto al nodo genitore:

$$\text{guadagno} = I(\text{parent}) - \sum_{j=1}^k \frac{N(v_j)}{N} \cdot I(v_j)$$

dove: - $I(\text{parent})$ = impurità del nodo genitore - k = numero di nodi figli prodotti dallo split - N = numero di record nel nodo genitore - $N(v_j)$ = numero di record del nodo figlio j -mo (quelli che nell'attributo scelto per lo split hanno tutti lo stesso valore v_j) - $I(v_j)$ = impurità del nodo figlio j -mo

In pratica: dall'impurità del nodo padre si sottrae la **media pesata** (pesata sulla numerosità) delle impurità dei nodi figli. Si sceglie lo split che **massimizza il guadagno** (equivalentemente, che minimizza l'impurità residua media).

Information gain

Quando la misura di impurità usata è l'entropia, il guadagno prende il nome di **information gain**:

$$\text{gain} = \text{entropia}(\text{parent}) - \sum_{j=1}^k \frac{N(v_j)}{N} \cdot \text{entropia}(v_j)$$

Attenzione — limite dell'information gain (e di Gini): queste misure tendono a **favorire attributi con molti valori diversi** rispetto ad attributi con pochi valori. Caso estremo: un identificatore univoco (es. numero di matricola) annulla completamente l'entropia dei figli (ogni figlio conterrebbe una sola istanza, quindi puro), dando un information gain massimo — ma **non è affatto un attributo significativo** per generalizzare, anzi porta a overfitting totale. Una possibile soluzione è restringersi a **split binari**.

? Domanda d'esame: perché un identificatore univoco (es. matricola) non va mai scelto come attributo di split, nonostante massimizzi l'information gain? **Risposta:** perché produce tanti nodi figli quante sono le istanze, ciascuno con una sola istanza e quindi entropia zero (purezza massima) — l'information gain risulta massimo. Ma un simile split non generalizza affatto: il "modello" ottenuto equivale a un dizionario che memorizza ogni singola istanza (overfitting estremo), incapace di classificare correttamente istanze nuove mai viste, che quasi certamente avranno un valore di matricola non presente nel training set. Questo mostra il limite intrinseco di entropia/Gini, che favoriscono attributi con molti valori distinti indipendentemente dalla loro reale capacità discriminante/generalizzante.

Criteri di arresto (cenni)

Le slide pongono esplicitamente la domanda "quando si termina la costruzione dell'albero?" (Nota 3 dell'algoritmo di Hunt) come uno dei problemi aperti della strategia greedy, insieme alla scelta dello split migliore. Il caso base naturale è quando un nodo è già puro (tutte le istanze della stessa classe) o quando non è più possibile scindere il nodo (istanze identiche ma classi diverse, gestito come da Nota 2).

Reti neurali

Ispirazione biologica

Le reti neurali (Neural Networks, NN) si ispirano al modo in cui i **neuroni biologici** agiscono e interagiscono fra loro. È importante però ricordare che i **neuroni artificiali non sono modelli fedeli** dei neuroni biologici: ne catturano solo alcuni principi essenziali (connessioni pesate, soglia di attivazione, propagazione del segnale).

Tra i primi modelli proposti: il **Perceptron** di Rosenblatt e la *B-machine* di Turing.

Il neurone artificiale (Perceptron)

Un **perceptron** è un elemento computazionale dotato di una piccola memoria (i pesi), in grado di calcolare una funzione di attivazione codificata al suo interno, applicata a una combinazione pesata dei valori in ingresso:

$$\text{net} = \sum_{i=1}^n w_i \cdot X_i$$

$$Y = f(\text{net})$$

- $x_1, \dots, x_n$: valori in ingresso (attributi descrittivi dell'istanza)
- $w_1, \dots, w_n$: pesi delle connessioni
- net : combinazione lineare pesata degli input
- f : funzione di attivazione

Funzioni di attivazione: - **Funzione gradino (step)**: la funzione originariamente usata da Rosenblatt; produce valori discreti per Y (0 oppure 1). Non è derivabile, il che la rende poco adatta a tecniche di apprendimento basate sul gradiente. - **Funzione sigmoide**:

$$Y = f(\text{net}) = \frac{1}{1 + e^{-\alpha(\text{net} - \theta)}}$$

dove θ è la soglia (bias, un parametro preimpostato) e α controlla la pendenza della curva. Il vantaggio della sigmoide rispetto al gradino è di essere **derivabile** (fondamentale per l'apprendimento via discesa del gradiente/backpropagation). Al crescere di α , la sigmoide tende alla forma della funzione gradino (approssimazione sempre più "brusca").

Passata forward e interpretazione geometrica

Il perceptron elabora dati visti come punti in uno **spazio N-dimensionale** (spazio degli input), dove N è il numero di attributi descrittivi (feature) dell'istanza. Data un'istanza, questa viene propagata attraverso il neurone che calcola Y tramite la funzione di attivazione: il processo può essere visto come una forma di classificazione binaria (l'istanza ricade nella categoria 1 o nella categoria 0, a seconda dell'output del perceptron).

Cosa codifica un perceptron? Un **test lineare**, cioè un **iperpiano** nello spazio degli input:

$$w_1 x_1 + w_2 x_2 + \dots = 0$$

- Ciò che cade **sopra** l'iperpiano (definito dai pesi) fa attivare il neurone (output 1): riconosciuto come appartenente alla classe obiettivo.
- Ciò che cade **sotto** non fa attivare il neurone (output 0): istanze negative.

I pesi sulle connessioni in ingresso definiscono quindi **posizione e pendenza** dell'iperpiano nello spazio degli input. I pesi **caratterizzano il neurone** e ne costituiscono la conoscenza: sono **persistenti** (a differenza dei pesi "usa e getta" della votazione pesata nel rote learning).

Per un perceptron con 2 input, l'iperpiano è la retta $x_1 = -(w_2/w_1) \cdot x_2$; il neurone si attiva per i punti tali che $w_1 \cdot x_1 + w_2 \cdot x_2 > 0$.

Caratteristiche del perceptron

- Adatto a compiti di tipo numerico
- Risolve solo problemi **linearmente separabili**
- La conoscenza è data dai pesi, che sono persistenti
- Apprendimento **da esempi, supervisionato**
- "Imparare" = individuare la posizione corretta dell'iperpiano nello spazio degli input

Apprendimento del perceptron

Essendo un problema di tipo numerico, si può usare l'**errore** (differenza fra valore desiderato d e valore ottenuto o) come segnale per guidare l'apprendimento. Regola di aggiornamento del peso sulla connessione dall'input j -mo:

$$w_j^{(k+1)} = w_j^{(k)} + \eta \cdot (d - o) \cdot x_j$$

Il peso viene aggiornato sommando l'errore, moltiplicato per un **fattore di scalamento** η (learning rate) e per il valore della componente j -ma dell'input.

Teorema di Novikoff: se il problema è **linearmente separabile**, questo algoritmo di apprendimento **converge** alla soluzione; se il problema non è linearmente separabile, l'algoritmo **non converge**.

Passata backward: dopo la passata forward il perceptron produce un output o ; essendo l'apprendimento supervisionato, si conosce l'output desiderato d . L'errore ($d - o$) è l'informazione usata per modificare i pesi, rendendo il comportamento futuro del perceptron più vicino a quello desiderato.

Epoca di apprendimento: l'elaborazione (forward + backward, aggiornamento pesi) di **tutte** le istanze del learning set costituisce un'epoca. L'addestramento richiede tipicamente molte epoche.

Intuizione geometrica della convergenza: ogni esempio "tira" l'iperpiano affinché il proprio caso venga classificato correttamente; l'iperpiano si sposta nello spazio delle istanze; il fattore di scalamento η evita che l'apprendimento "inseguo" solo l'ultimo esempio visto, forzando una generalizzazione più stabile.

Limiti del singolo perceptron: lo XOR

Il limite principale del perceptron fu dimostrato con un esempio semplicissimo: lo **XOR** (or esclusivo).

A1	A2	XOR
0	0	0
1	0	1
0	1	1
1	1	0

Interpretando le coppie (A1, A2) come coordinate di punti nel piano e l'output XOR come l'appartenenza sopra/sotto un ipotetico iperpiano separatore: i punti positivi (output 1) e negativi (output 0) **non sono linearmente separabili** da nessuna retta — non esiste un'unica retta che separi correttamente le due classi. Un singolo perceptron non può quindi apprendere lo XOR.

Intuizione della soluzione: un solo iperpiano non basta, ma se se ne avessero **due** disponibili, con la capacità di combinare le loro risposte, il problema si risolverebbe. Con più iperpiani a disposizione si possono addirittura circoscrivere regioni chiuse arbitrariamente complesse. Questa intuizione porta naturalmente all'introduzione delle reti **multi-strato**.

Reti neurali: definizione generale e topologia

Una **rete neurale** è un **approssimatore universale di funzioni**, di natura distribuita, costituita da un insieme di neuroni collegati fra loro secondo una **topologia** (che dipende dal modello di rete). I neuroni possono implementare funzioni diverse e possono essere realizzati in software o hardware.

Esempi di topologia: - **A strati (layered)**: i neuroni sono organizzati in livelli successivi (input, hidden, output).

- **A vicinato**: connessioni basate su prossimità/adiacenza fra neuroni, senza una stratificazione rigida.

Multi-Layer Perceptron (MLP)

Il **MLP** è un modello di rete neurale con topologia **a strati**, di tipo **feed-forward** (il flusso di calcolo procede in una sola direzione, dall'input verso l'output, senza cicli):

- **Livello di input**: di solito i neuroni implementano la **funzione identità** (si limitano a "passare" il valore dell'attributo corrispondente).
- **Livello/i hidden**: i neuroni sono perceptron veri e propri, che usano tipicamente la **sigmoide** (o altre funzioni derivabili, es. varianti dello scalino). Si possono avere **più livelli hidden**.
- **Livello di output**: combina i risultati prodotti dai neuroni hidden.

Tipicamente la rete è **fully connected** (ogni neurone di un livello è connesso a tutti i neuroni del livello successivo).

Codifica delle classi nell'output di un MLP

Il modo di codificare le classi nel livello di output dipende dal numero di classi del problema:

- **Riconoscimento di 1 classe** (es. "è un'istanza della classe X oppure no"): basta **1 solo neurone di output**. Se si attiva, l'istanza è riconosciuta come appartenente alla classe X.
- **Distinzione fra 2 classi** (X vs Y, classificazione binaria): basta ancora **1 neurone di output**, la cui attivazione viene associata convenzionalmente a una delle due classi (es. attivo = classe X, non attivo = classe Y).
- **Distinzione fra 3 o più classi**: qui si aprono due strade:
 - *Codifica binaria compatta*: userebbero almeno $\lceil \log_2(\text{numero classi}) \rceil$ neuroni (es. con 2 neuroni si rappresentano fino a 4 combinazioni/classi in binario).
 - *Codifica "one-hot" (un neurone per classe)*: è **l'alternativa più usata in pratica**. Si impiega un neurone di output per ciascuna classe; il neurone corrispondente alla classe corretta deve attivarsi, tutti gli altri devono restare a zero.

* Esempio con 2 classi codificate one-hot: output 10 → Classe A, output 01 → Classe B.

? Domanda d'esame: perché per un problema a più classi si preferisce spesso un neurone di output per classe (codifica one-hot) invece della codifica binaria compatta ($\log_2$ dei neuroni)? **Risposta:** con la codifica compatta, il significato di ciascun neurone di output è "distribuito" e non direttamente interpretabile: un piccolo errore nell'attivazione di un solo neurone può far scivolare l'output verso il codice binario di una classe completamente diversa e sbagliata (gli output non hanno una gerarchia naturale di "vicinanza semantica"). Con la codifica one-hot, invece, ogni neurone rappresenta direttamente e in modo indipendente il grado di attivazione/confidenza verso una specifica classe: è più robusta a piccoli errori (l'errore di solito si concentra su neuroni "vicini" in termini di somiglianza fra classi), più facile da interpretare e da addestrare con la delta rule/backpropagation standard (ogni neurone di output ha un proprio target 0/1 chiaro).

Cosa imparano i diversi hidden layer

Con più livelli hidden, si osserva empiricamente una sorta di **gerarchia di astrazione**: - il **primo layer** traccia dei confini (separazioni lineari elementari, come un singolo perceptron); - il **secondo layer** combina questi confini per costruire delle **forme**; - il **terzo layer** combina le forme per creare **forme arbitrariamente complesse**.

Questo spiega intuitivamente perché aggiungere strati hidden aumenti la capacità espressiva della rete rispetto al singolo perceptron (che può solo tracciare un iperpiano).

Potere espressivo dell'MLP

Gli **MLP a 3 livelli** i cui neuroni usano come funzione di attivazione la sigmoide sono **approssimatori universali di funzioni**, a patto di poter utilizzare un numero di neuroni hidden sufficientemente grande (senza limiti a priori).

La conoscenza appresa dalla rete è memorizzata nella **matrice dei pesi** che definisce la forza di tutte le connessioni.

Apprendimento dell'MLP: backpropagation

Una rete MLP impara in modo **supervisionato**, inducendo la matrice dei pesi a partire da un insieme di esempi etichettati (nel caso della classificazione). Per ogni istanza si effettuano due passate:

1. **Passata forward**: l'istanza viene sottoposta alla rete, che la elabora strato per strato e produce un output.
2. **Passata backward**: l'errore commesso viene utilizzato per modificare i pesi, procedendo "a ritroso" a partire dalle connessioni più vicine ai neuroni di output verso quelle degli strati precedenti.

Anche qui, l'elaborazione dell'intero learning set costituisce un'**epoca di apprendimento**, e il training richiede tipicamente molte epoche.

Discesa del gradiente

L'errore complessivo E della rete dipende dalla matrice dei pesi w . Imparare significa **cercare la configurazione di pesi w che minimizza E** . Si usa la tecnica della **discesa del gradiente** (greedy, cerca un minimo — non necessariamente globale):

$$\Delta w_{ji} = -\eta \cdot \frac{\partial E(w)}{\partial w_{ji}}$$

Ogni peso w_{ji} (sulla connessione dal neurone j -mo al neurone i -mo) viene modificato sottraendo la derivata parziale dell'errore rispetto a quel peso, scalata da un fattore di apprendimento η .

Il **gradiente** $\nabla f = (\partial f / \partial x_1, \dots, \partial f / \partial x_n)$ indica la direzione di massima crescita di una funzione; usato con segno negativo, permette di muoversi verso un minimo. Analogamente $\nabla E = (\partial E / \partial w_1, \dots, \partial E / \partial w_n)$ indica come modificare i pesi per ridurre l'errore.

Errore globale dell'MLP

$$E = \frac{1}{2} \sum_{i=1}^p (t_i - y_i)^2$$

dove p = numero di neuroni di output, t_i = valore desiderato (target) del neurone di output i -mo, y_i = valore effettivamente prodotto dal neurone di output i -mo.

Il problema della distribuzione del credito/biasimo (credit assignment)

L'errore è calcolabile direttamente solo per i neuroni del **livello di output** (per i quali si conosce il target t). Ma come si distribuisce il "biasimo" per l'errore fra i pesi di tutti gli strati precedenti (hidden)? Questo è il problema centrale risolto dall'algoritmo di **backpropagation**.

Delta rule generalizzata per i neuroni di output:

$$\Delta w_{ji} = \eta \cdot \delta_j \cdot x_{ji}$$

$$\delta_j = y_j \cdot (1 - y_j) \cdot (t_j - y_j)$$

dove j è un neurone di output, i un neurone del livello precedente connesso a j , x_{ji} il valore inviato da i a j , y_j l'output prodotto da j , t_j il target desiderato per j . Il termine $y_j(1-y_j)$ deriva dalla derivata della funzione sigmoide.

Delta rule per i neuroni hidden — l'idea è distribuire l'errore all'indietro fra le connessioni, proporzionalmente alla forza dei pesi correnti:

$$\Delta w_{ki} = \eta \cdot \delta_k \cdot x_{ki}$$

$$\delta_k = y_k \cdot (1 - y_k) \cdot \sum_{j \in I_k} \delta_j \cdot w_{kj}$$

dove k è un neurone hidden e I_k è l'insieme dei neuroni del livello successivo a cui k è connesso. In pratica il δ di un neurone hidden è calcolato "propagando all'indietro" (da cui il nome *backpropagation*) i δ dei neuroni a cui è connesso nel livello successivo, pesati per la forza delle rispettive connessioni.

Oltre la classificazione: la regressione

Una rete neurale (non necessariamente MLP) può risolvere sia problemi di **classificazione** sia problemi di **regressione**. La regressione, in statistica, è il problema di definire la relazione fra un insieme di variabili indipendenti e una variabile dipendente; più in generale, è il problema di costruire un'approssimazione di una **funzione continua** $y = f(x)$ la cui forma analitica non è nota e va indotta da esempi.

Il learning set ha la stessa struttura vista per la classificazione (variabili indipendenti $X_1 \dots X_n$ e variabile dipendente target), ma il target è un valore continuo anziché una classe discreta. Una rete neurale può anche imparare **funzioni a più valori** (più variabili dipendenti): basta prevedere un neurone di output per ciascun valore prodotto dalla funzione.

Calcolo dell'errore in regressione: non essendoci classi, non si può più usare la matrice di confusione né misure come precision/recall o accuratezza/error rate. Si usa invece l'**Errore Quadratico Medio (Mean Squared Error, MSE)**:

$$MSE(y) = \frac{\sum_{i=1}^n (y_o - y_d)^2}{n}$$

Somma dei quadrati degli errori (differenza fra output ottenuto y_o e output desiderato y_d) su tutte le istanze di test, divisa per il numero di istanze. Si usa il quadrato per evitare che errori di segno opposto si annullino a vicenda nella somma.

Riepilogo e punti chiave

- **Classificazione:** le classi non sono naturali ma definite dal progettista; si apprende un modello dal *training set* (induzione) e lo si valuta sul *test set* (deduzione), tramite matrice di confusione, accuratezza $((f_{11}+f_{22})/\text{tot})$, error rate, ed eventualmente matrice dei costi quando gli errori hanno peso diverso. Attenzione a dataset sbilanciati: un'accuratezza alta non implica un buon modello.
- **Alberi di decisione:** nodi interni = test su attributi, foglie = decisioni/classi. Si costruiscono ricorsivamente con l'**algoritmo di Hunt**: se il nodo è puro diventa foglia, altrimenti si sceglie un attributo di split e si ricorre sui figli. La scelta dell'attributo migliore è **greedy** e si basa su misure di impurità — **entropia** $(-\sum p_i \log_2 p_i)$, **Gini**, **errore di classificazione** — combinate nella formula del **guadagno** (impurità del padre meno media pesata delle impurità dei figli); con l'entropia si parla di **information gain**. Attenzione al bias verso attributi con molti valori (es. identificatori univoci) — un limite noto di entropia e Gini. Split diversi a seconda del tipo di attributo (binario, nominale, ordinale, continuo). Principio guida: Rasoio di Occam, preferire alberi compatti.
- **Reti neurali / perceptron:** ispirate (in modo non fedele) al neurone biologico. Il perceptron calcola $\text{net} = \sum w_i \cdot x_i$ e applica una funzione di attivazione (gradino, poi sostituita dalla **sigmoide**, derivabile) per produrre $y = f(\text{net})$. Geometricamente codifica un **iperpiano** (test lineare) nello spazio degli input: risolve solo problemi **linearmente separabili** (limite dimostrato con lo **XOR**). Apprende per correzione dell'errore $(d-o)$ con un fattore di scalamento η ; converge solo se il problema è linearmente separabile (Novikoff).
- **MLP:** rete feed-forward a strati (input, uno o più hidden, output), tipicamente fully-connected, con hidden layer che usano la sigmoide. Con almeno 3 livelli e neuroni sigmoide a sufficienza è un **approssimatore universale di funzioni**. La codifica delle classi in output usa 1 neurone per problemi a 1-2 classi, e tipicamente **un neurone per classe (one-hot)** per problemi multiclasse. L'apprendimento avviene per **backpropagation**: passata forward per calcolare l'output, passata backward per propagare l'errore e aggiornare i pesi via **discesa del gradiente**, minimizzando l'errore globale $E = \frac{1}{2} \sum (t_i - y_i)^2$; la *delta rule* per l'output usa $\delta_j = y_j(1-y_j)(t_j-y_j)$, quella per gli hidden propaga i δ dei neuroni successivi pesati sulle connessioni. Le reti neurali risolvono anche problemi di **regressione** (funzione continua), valutati con l'**MSE** anziché con le metriche di classificazione.